UNIT-5

MATPLOTLIB

1Q) Define Matplotlib.

Ans:

Matplotlib is a **Python library used to create data visualizations such as graphs, charts, and plots.**

2Q) Explain Key features of Matplotlib.

Ans:

- **Supports multiple chart types** – Can create line plots, bar charts, scatter plots, pie charts, histograms, and many more.
- **Highly customizable** – Allows customization of colors, labels, titles, markers, fonts, and plot styles.
- **Exports plots in multiple formats** – Saves graphs as PNG, PDF, SVG, JPEG, and other formats.
- **Works with NumPy and Pandas** – Integrates seamlessly with NumPy arrays, Pandas DataFrames, and Jupyter Notebooks for data analysis and visualization.

3Q) Explain Simple Line Plot with uses, applications and code with output.

Ans:

Simple Line Plot

Definition

A **Simple Line Plot** is a graph in which data points are connected by straight lines. It is used to show **changes, trends, or patterns** over a continuous interval, such as time.

Key Uses

- Shows how a value changes over time.
- Displays trends and patterns in data.
- Compares two or more datasets.

Real-Life Examples

- Tracking daily temperature.
- Monitoring monthly sales of a company.
- Recording the number of steps walked each day.
- Showing stock price changes over time.

Syntax

plt.plot(x, y)

- **x** → Values for the x-axis
- **y** → Values for the y-axis

Example Program

import matplotlib.pyplot as plt

Data

x = [1, 2, 3, 4, 5]

y = [10, 12, 8, 14, 20]

Plot

plt.plot(x, y, label='Sales', color='blue', marker='o')

Title and labels

```
plt.title("Simple Line Plot")

plt.xlabel("Time (Days)")

plt.ylabel("Sales")


# Legend and grid

plt.legend()

plt.grid(True)


# Display the plot

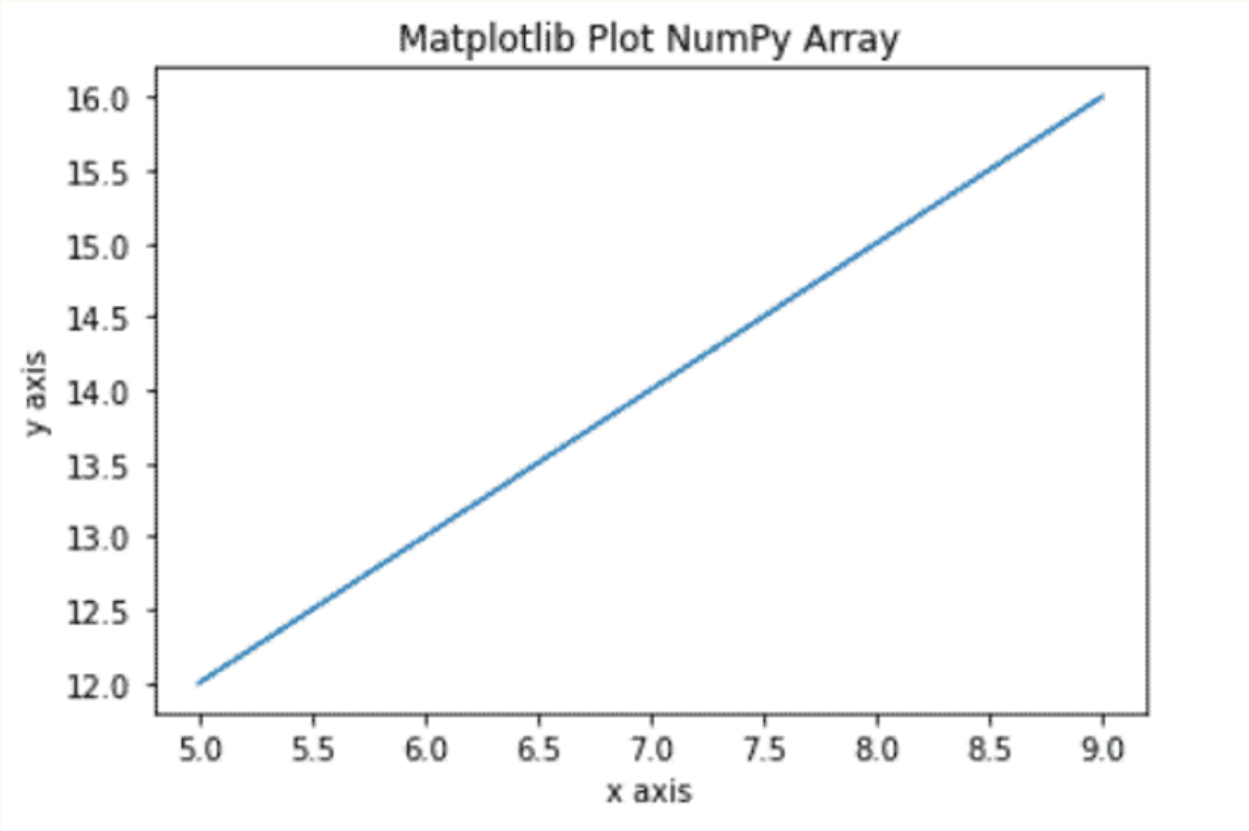
plt.show()
```

Output

A line graph with:

- Blue line
- Circular (`o`) markers
- X-axis labeled **Time (Days)**
- Y-axis labeled **Sales**
- Legend showing **Sales**

- Grid in the background



Explanation of the Code

1. Import Matplotlib

import matplotlib.pyplot as plt

Imports the `pyplot` module and gives it the alias `plt`.

2. Create Data

x = [1, 2, 3, 4, 5]

y = [10, 12, 8, 14, 20]

- `x` contains the values for the x-axis.
- `y` contains the corresponding values for the y-axis.

3. Draw the Line Plot

plt.plot(x, y, label='Sales', color='blue', marker='o')

- `plt.plot()` → Draws the line graph.
- `x` → X-axis values.
- `y` → Y-axis values.
- `label='Sales'` → Gives a name to the plotted line for the legend.
- `color='blue'` → Sets the line color to blue.
- `marker='o'` → Displays a circular marker at each data point.

4. Add Title

plt.title("Simple Line Plot")

Adds the title **"Simple Line Plot"** to the graph.

5. Label the X-axis

plt.xlabel("Time (Days)")

Labels the x-axis as **Time (Days).**

6. Label the Y-axis

plt.ylabel("Sales")

Labels the y-axis as **Sales.**

7. Display the Legend

plt.legend()

Shows the legend using the label **Sales.**

8. Display Grid

```
plt.grid(True)
```

Adds grid lines to improve readability.

9. Show the Plot

```
plt.show()
```

Displays the graph on the screen.

Important Functions Used

Function	Purpose
`plt.plot()`	Draws a line plot
`plt.title()`	Adds a title
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis

`plt.legend()`	Displays the legend
`plt.grid(True)`	Adds grid lines
`plt.show()`	Displays the plot

Exam Definition (2 Marks)

A Simple Line Plot is a graph that connects data points with straight lines to show trends, changes, or patterns over a continuous interval such as time.

4Q) Explain Simple Scatter Plot with uses, applications and code with output.

Ans:

Scatter Plot

Definition

A **Scatter Plot** is a graph that displays individual data points as dots to show the relationship between two numerical variables. Each dot represents one observation in the dataset.

Unlike a line plot, **the points are not connected by lines**.

Key Features

- Displays the relationship between two variables.
- Each point represents one data observation.
- No lines connect the points.
- Helps identify correlations, clusters, and outliers.
- Useful in regression and trend analysis.

Key Uses

- Shows the relationship between two variables.
- Identifies positive, negative, or no correlation.
- Detects clusters and outliers.
- Used in data analysis and machine learning.

Real-Life Examples

- Hours studied vs Exam scores
- Age vs Speed
- Height vs Weight
- Advertising cost vs Sales

Syntax

```
plt.scatter(x, y)
```

- **x** → Values for the x-axis
- **y** → Values for the y-axis

Example Program

```
import matplotlib.pyplot as plt


# Data
x = [5, 7, 8, 7, 2, 17, 2, 9, 4, 11]
y = [99, 86, 87, 88, 100, 86, 103, 87, 94, 78]


# Plot
plt.scatter(x, y, color='green')


# Title and labels
plt.title("Simple Scatter Plot")
plt.xlabel("Age")
plt.ylabel("Speed")


# Grid
plt.grid(True)


# Display plot
```

plt.show()

Explanation of the Code

1. Import Matplotlib

import matplotlib.pyplot as plt

Imports the `pyplot` module.

2. Create Data

x = [5, 7, 8, 7, 2, 17, 2, 9, 4, 11]

y = [99, 86, 87, 88, 100, 86, 103, 87, 94, 78]

- `x` contains the x-axis values (Age).
- `y` contains the y-axis values (Speed).

3. Draw the Scatter Plot

plt.scatter(x, y, color='green')

- `plt.scatter()` creates a scatter plot.
- Each `(x, y)` pair is shown as a **green dot**.
- No lines connect the dots.

4. Add Title

plt.title("Simple Scatter Plot")

Adds the title **"Simple Scatter Plot"**.

5. Label the X-axis

plt.xlabel("Age")

Labels the x-axis as **Age**.

6. Label the Y-axis

plt.ylabel("Speed")

Labels the y-axis as **Speed**.

7. Add Grid

plt.grid(True)

Displays grid lines.

8. Display the Plot

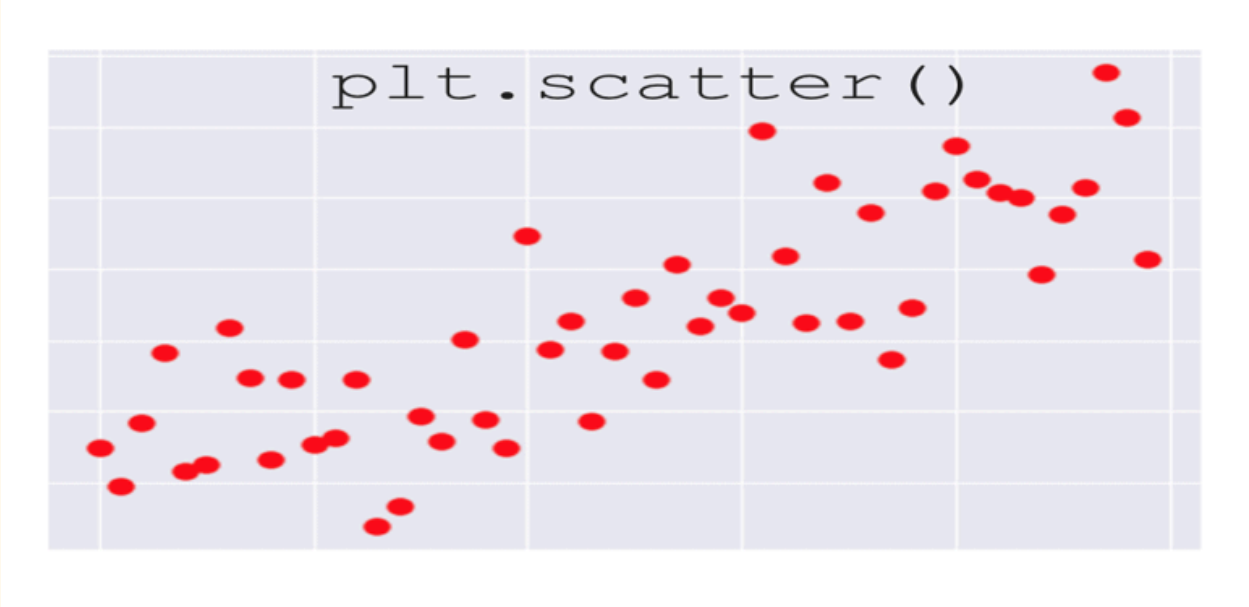
plt.show()

Shows the scatter plot.

Output

The output is a **Scatter Plot** containing **10 green dots**.

- X-axis → **Age**
- Y-axis → **Speed**
- Title → **Simple Scatter Plot**
- Green dots represent each `(Age, Speed)` pair.
- Grid lines are displayed.
- **No lines connect the points.**



Data Points Plotted

Age	Speed
5	99
7	86
8	87

7 88

2 100

17 86

2 103

9 87

4 94

11 78

Approximate Output

Speed

105 | ●

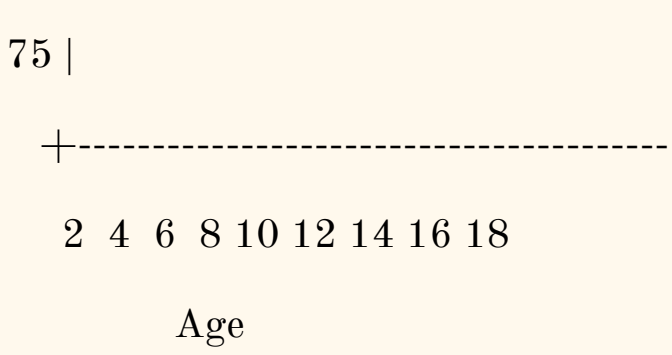
100 | ● ●

95 | ●

90 | ● ● ●

85 | ● ●

80 | ●



Important Functions Used

Function	Purpose
`plt.scatter()`	Creates a scatter plot
`plt.title()`	Adds a title
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis
`plt.grid(True)`	Displays grid lines

`plt.show()`	Displays the graph

Exam Definition (2 Marks)

A Scatter Plot is a graph that represents data points as dots to show the relationship between two numerical variables. It is used to identify correlations, patterns, clusters, and outliers in the data.

5Q) Explain Pie Chart with uses, applications and code with output.

Ans:

Pie Chart

A Pie Chart is a circular graph divided into slices, where each slice represents a category's proportion of the whole. It is mainly used to display percentage or proportional data.

The **entire circle represents 100%** of the data.

Key Characteristics

- The whole pie represents **100%**.
- Each slice represents one category.
- The size of each slice is proportional to its value.
- Can display labels and percentages.

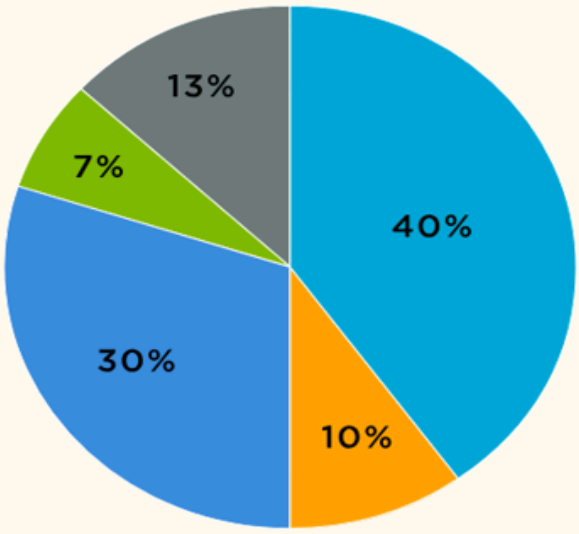
- Supports colors, exploded slices, and shadows.

Key Uses

- Shows part-to-whole relationships.
- Displays percentage distribution.
- Compares proportions of different categories.

Real-Life Examples

- Market share of smartphone brands.
- Household budget distribution.
- Student marks distribution.
- Sales contribution of different products.



Syntax

plt.pie(sizes)

Example Program

```
import matplotlib.pyplot as plt


# Data
labels = ['Apples', 'Bananas', 'Cherries', 'Dates']
sizes = [30, 25, 25, 20]
colors = ['red', 'yellow', 'pink', 'brown']
explode = (0.1, 0, 0, 0)


# Plot
plt.pie(
    sizes,
    labels=labels,
    colors=colors,
    explode=explode,
    autopct='%1.1f%%',
    shadow=True,
    startangle=140
```

)

plt.title("Fruit Distribution")

plt.axis('equal')

plt.show()

Explanation of the Code

1. Import Matplotlib

import matplotlib.pyplot as plt

Imports the `pyplot` module.

2. Create Data

labels = ['Apples', 'Bananas', 'Cherries', 'Dates']

sizes = [30, 25, 25, 20]

colors = ['red', 'yellow', 'pink', 'brown']

explode = (0.1, 0, 0, 0)

- `labels` → Names of the categories.
- `sizes` → Values of each category.

- `colors` → Color of each slice.
- `explode` → Moves the first slice slightly outward for emphasis.

3. Draw the Pie Chart

```
plt.pie(
    sizes,
    labels=labels,
    colors=colors,
    explode=explode,
    autopct='%1.1f%%',
    shadow=True,
    startangle=140
)
```

Parameters:

- `sizes` → Determines the size of each slice.
- `labels=labels` → Displays category names.
- `colors=colors` → Assigns different colors.
- `explode=explode` → Separates the first slice.
- `autopct='%1.1f%%'` → Displays percentages with one decimal place.
- `shadow=True` → Adds a shadow effect.

- `startangle=140` → Rotates the chart by 140°.

4. Add Title

plt.title("Fruit Distribution")

Adds the title **"Fruit Distribution"**.

5. Make the Pie Circular

plt.axis('equal')

Ensures the pie chart is drawn as a **perfect circle** instead of an ellipse.

6. Display the Chart

plt.show()

Displays the pie chart.

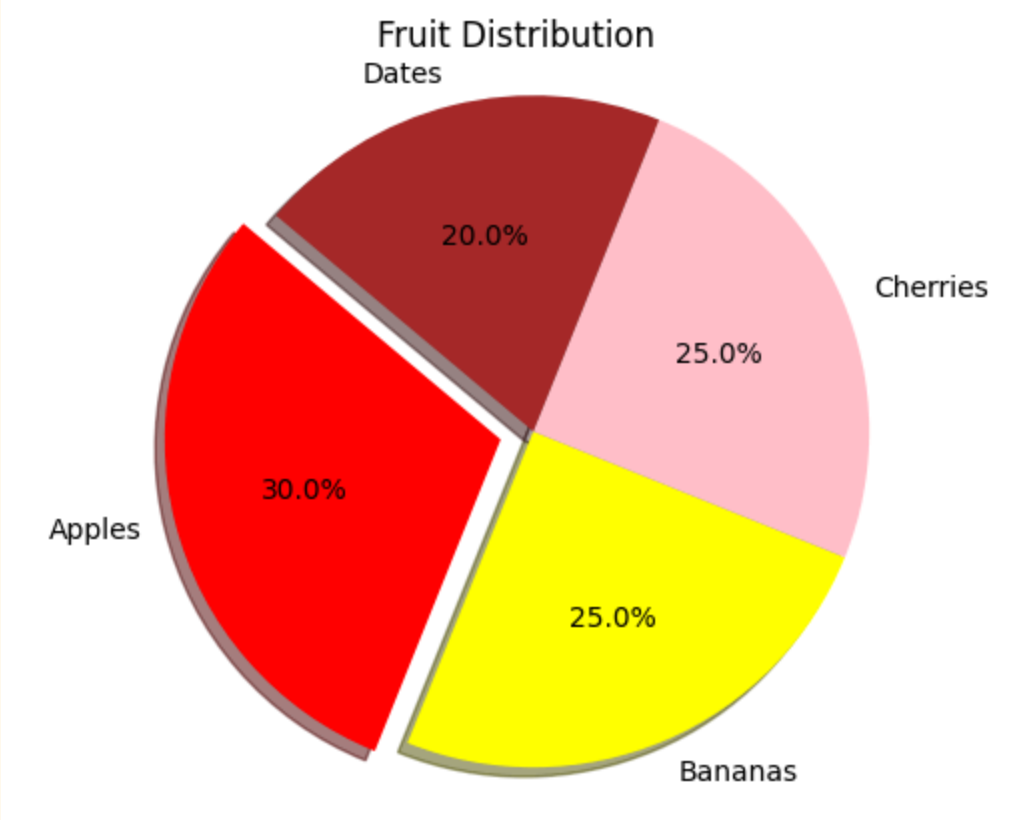
Output

The output is a **circular pie chart** showing the distribution of four fruits.

Features of the output:

- **Red slice** → Apples (30%)

- **Yellow slice** → Bananas (25%)
- **Pink slice** → Cherries (25%)
- **Brown slice** → Dates (20%)
- The **Apples slice is slightly separated** from the pie.
- Each slice displays its percentage.
- A shadow appears behind the pie.
- The title "**Fruit Distribution**" is displayed.



Data Represented

Fruit	Value	Percentage

Apples	30	30.0%
Bananas	25	25.0%
Cherries	25	25.0%
Dates	20	20.0%

Important Functions/Parameters Used

Function/Parameter	Purpose
`plt.pie()`	Creates a pie chart
`labels`	Displays category names
`sizes`	Specifies the value of each slice

`colors`	Assigns colors to slices
`explode`	Separates a slice for emphasis
`autopct`	Displays percentage values
`shadow=True`	Adds a shadow effect
`startangle`	Rotates the starting position of the pie
`plt.title()`	Adds a title
`plt.axis('equal')`	Makes the pie chart perfectly circular

`plt.show()`	Displays the chart

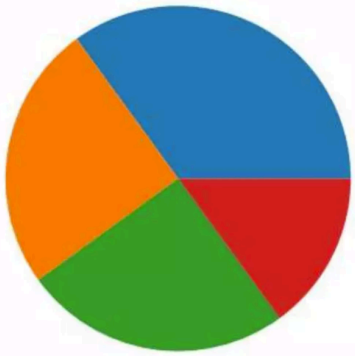
Creating Pie Charts

With Pyplot, you can use the pie() function to draw pie charts:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([35,25,25,15])

plt.pie(y)
plt.show()
```



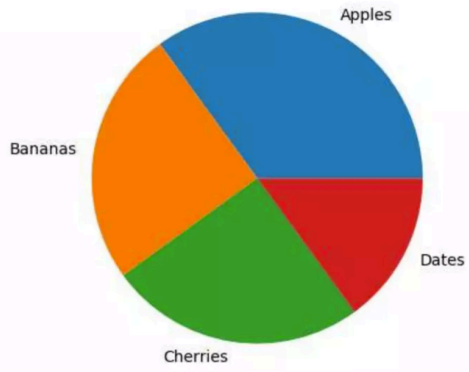
Labels

Add labels to the pie chart with the label parameter.

The label parameter must be an array with one label for each wedge:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas",
"Cherries", "Dates"]
plt.pie(y, labels=mylabels)
plt.show()
```

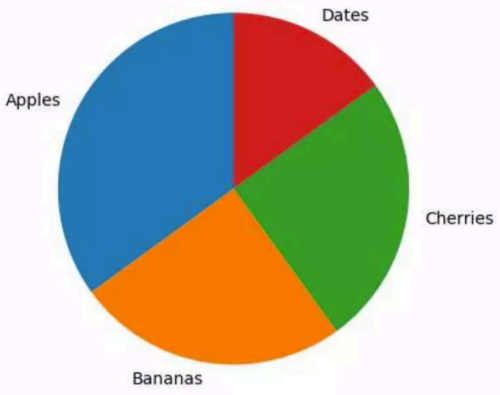


Start Angle

As mentioned the default start angle is at the x-axis, but you can change the start angle by specifying a startangle parameter.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas",
"Cherries", "Dates"]
plt.pie(y, labels=mylabels,startangle=90)
plt.show()
```

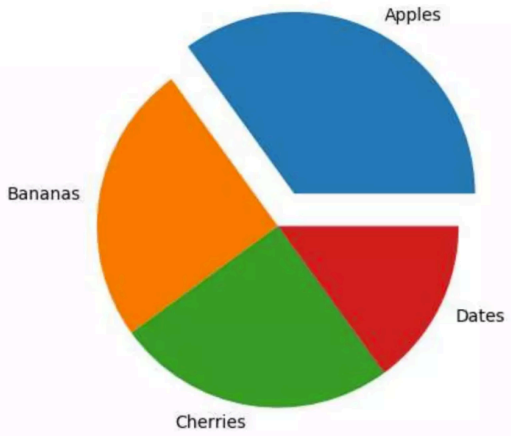


Explode

Maybe you want one of the wedges to stand out? The explode parameter allows you to do that.

The explode parameter, if specified, and not None, must be an array with one value for each wedge

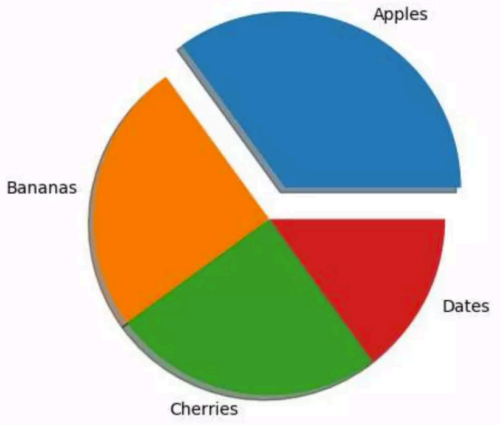
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas", "Cherries",
"Dates"]
myexplode = [0.2, 0, 0, 0]
plt.pie(y, labels=mylabels,
startangle=90,explode=myexplode)
plt.show()
```



Shadow

Add a shadow to the pie chart by setting the shadows parameter to True:

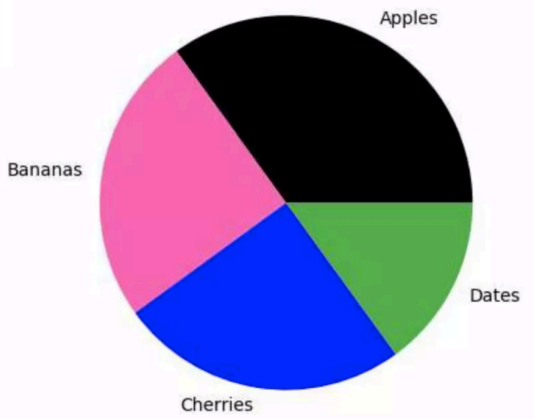
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas", "Cherries",
"Dates"]
myexplode = [0.2, 0, 0, 0]
plt.pie(y, labels=mylabels, startangle=90,
explode=myexplode, shadow=True)
plt.show()
```



Colors

You can set the color of each wedge with the colors parameter.

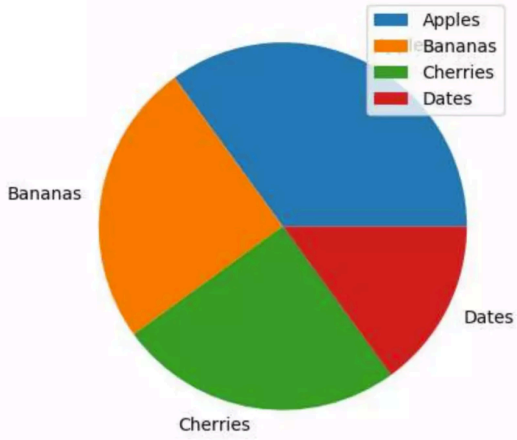
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas", "Cherries",
"Dates"]
mycolors = ["black", "hotpink", "b", "#4CAF50"]
myexplode = [0.2, 0, 0, 0]
plt.pie(y, labels=mylabels, startangle=90,
explode=myexplode, colors= mycolors)
plt.show()
```



Legend

To add a list of explanation for each wedge, use the legend() function:

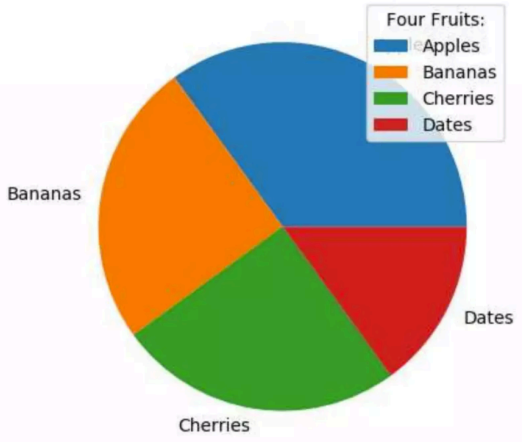
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas", "Cherries",
"Dates"]
plt.pie(y, labels=mylabels)
plt.legend()
plt.show()
```



Legend With Header

To add a header to the legend, add the title parameter to the legend function.

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas", "Cherries",
"Dates"]
plt.pie(y, labels=mylabels)
plt.legend(title='Four Fruits:')
plt.show()
```



Exam Definition (2 Marks)

A Pie Chart is a circular graph divided into slices, where each slice represents a category's proportion or percentage of the whole. It is used to display part-to-whole relationships and percentage distributions.

6Q) Explain Bar Graph with uses, applications and code with output.

Ans:

Bar Graph (Bar Chart)

Definition

A **Bar Graph (Bar Chart)** is a graph that represents data using **rectangular bars**, where the height (or length) of each bar is proportional to the value it represents. It is mainly used to **compare values among different categories**.

Key Characteristics

- Represents data using **rectangular bars**.
- Bars can be **vertical or horizontal**.
- The height (or length) of each bar represents the value.
- Bars are separated by equal spaces.
- Different colors can be used for better visualization.

Key Uses

- Compares values of different categories.
- Displays categorical data clearly.
- Shows highest and lowest values easily.
- Useful for comparing sales, marks, population, etc.

Real-Life Examples

- Comparing students' marks in different subjects.
- Monthly sales of a shop.
- Population of different cities.
- Number of employees in different departments.

Purpose

Used to **compare values across different categories**.

Syntax

plt.bar(x, height)

- **x** → Category names or positions.
- **height** → Values of each category.

Example Program

import matplotlib.pyplot as plt

Data

subjects = ['Math', 'Science', 'English', 'Social']

marks = [85, 90, 75, 80]

```
# Plot
plt.bar(subjects, marks, color='skyblue')

# Title and labels
plt.title("Student Marks")
plt.xlabel("Subjects")
plt.ylabel("Marks")

# Grid
plt.grid(axis='y')

# Display plot
plt.show()
```

Explanation of the Code

1. Import Matplotlib

import matplotlib.pyplot as plt

Imports the `pyplot` module.

2. Create Data

subjects = ['Math', 'Science', 'English', 'Social']

marks = [85, 90, 75, 80]

- `subjects` contains the category names.
- `marks` contains the corresponding values.

3. Draw the Bar Graph

plt.bar(subjects, marks, color='skyblue')

- `plt.bar()` creates a vertical bar graph.
- `subjects` are displayed on the x-axis.
- `marks` determine the height of each bar.
- `color='skyblue'` makes all bars sky blue.

4. Add Title

plt.title("Student Marks")

Displays the title **"Student Marks"**.

5. Label the X-axis

plt.xlabel("Subjects")

Labels the x-axis as **Subjects.**

6. Label the Y-axis

plt.ylabel("Marks")

Labels the y-axis as **Marks.**

7. Add Grid

plt.grid(axis='y')

Adds horizontal grid lines for easier comparison.

8. Display the Graph

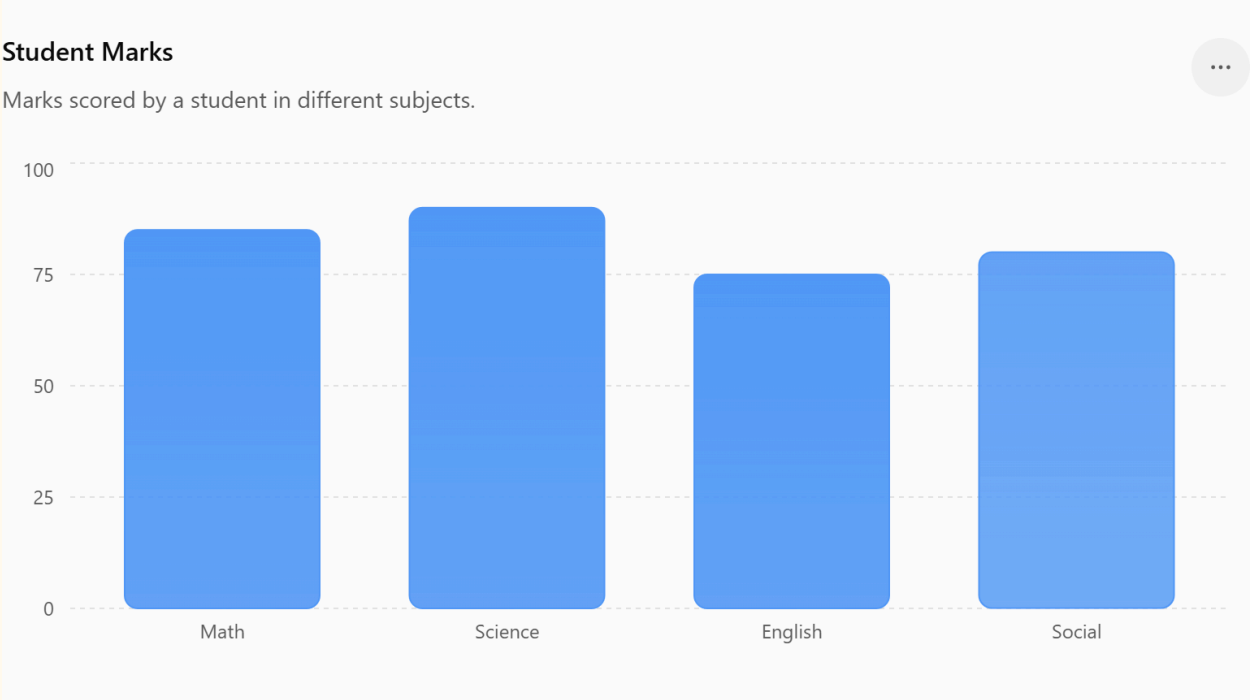
plt.show()

Displays the bar graph.

Output

Student Marks

Marks scored by a student in different subjects.



Output Description

The output is a **vertical bar graph** with the following features:

- **Title:** *Student Marks*
- **X-axis:** Subjects (Math, Science, English, Social)
- **Y-axis:** Marks
- Four **sky-blue vertical bars** represent the marks.
- Horizontal grid lines are displayed.

Subject	Marks

Math	85
Science	90
English	75
Social	80

Important Functions Used

Function	Purpose
`plt.bar()`	Creates a vertical bar graph
`plt.title()`	Adds a title
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis
`plt.grid(axis='y')`	Adds horizontal grid lines

`plt.show()`	Displays the graph

Exam Definition (2 Marks)

A Bar Graph (Bar Chart) is a graph that uses rectangular bars to represent data, where the height or length of each bar is proportional to the value it represents. It is used to compare values across different categories.

7Q) Demonstrate the use of annotations to label important points in a graph, use a scatter plot for illustration.

Ans:

Definition of Annotation

Annotation is the process of adding **text, labels, arrows, or notes** to specific points on a graph to highlight or explain important data points.

In Matplotlib, annotations help make graphs **more informative and easier to understand** by drawing attention to significant observations such as the highest value, lowest value, outliers, or special events.

Why are Annotations Used?

Annotations are used to:

- Highlight important data points.
- Explain special events or observations.
- Make graphs easy to understand.
- Improve the readability of graphs.
- Point out maximum, minimum, or unusual values.

Real-Life Applications

- Highlighting the highest stock price.
- Showing the highest exam score.
- Marking the maximum temperature.
- Identifying outliers in scientific experiments.
- Showing the best-performing product in sales data.

Function Used

plt.annotate(text, xy, xytext, arrowprops)

Syntax

```
plt.annotate(
    "Text",
    xy=(x, y),
    xytext=(x_text, y_text),
    arrowprops=dict(facecolor='black', shrink=0.05)
```

)

Parameters

Parameter	Description
`text`	The text displayed on the graph.
`xy`	Coordinates of the point to annotate.
`xytext`	Position where the text will appear.
`arrowprops`	Draws an arrow from the text to the point.

Example Program

```
import matplotlib.pyplot as plt


# Data
x = [1, 2, 3, 4, 5, 6]
```

```
y = [12, 18, 15, 25, 20, 30]

# Scatter Plot
plt.scatter(x, y, color='blue')

# Annotate the highest point
plt.annotate(
    "Highest Point",
    xy=(6, 30),
    xytext=(4.5, 33),
    arrowprops=dict(facecolor='red', shrink=0.05)
)

# Annotate another important point
plt.annotate(
    "Good Score",
    xy=(4, 25),
    xytext=(2.8, 27),
    arrowprops=dict(facecolor='green', shrink=0.05)
```

```
)

# Title and Labels
plt.title("Scatter Plot with Annotations")
plt.xlabel("Student Number")
plt.ylabel("Marks")

# Grid
plt.grid(True)

# Display Plot
plt.show()
```

Explanation of the Code

Step 1: Import Matplotlib

```
import matplotlib.pyplot as plt
```

Imports the `pyplot` module for plotting graphs.

Step 2: Create Data

x = [1,2,3,4,5,6]

y = [12,18,15,25,20,30]

- `x` represents student numbers.
- `y` represents their marks.

Step 3: Create Scatter Plot

plt.scatter(x, y, color='blue')

Creates a scatter plot using blue dots.

Step 4: Add First Annotation

plt.annotate(

"Highest Point",

xy=(6,30),

xytext=(4.5,33),

arrowprops=dict(facecolor='red', shrink=0.05)

)

Explanation

- `"Highest Point"` → Text displayed.
- `xy=(6,30)` → Point being highlighted.
- `xytext=(4.5,33)` → Position of the text.
- `arrowprops` → Draws a red arrow pointing to the data point.

Step 5: Add Second Annotation

```
plt.annotate(
    "Good Score",
    xy=(4,25),
    xytext=(2.8,27),
    arrowprops=dict(facecolor='green', shrink=0.05)
)
```

This adds another label with a green arrow pointing to the point `(4,25)`.

Step 6: Add Title

```
plt.title("Scatter Plot with Annotations")
```

Adds the graph title.

Step 7: Add Axis Labels

```
plt.xlabel("Student Number")
plt.ylabel("Marks")
```

Labels the X-axis and Y-axis.

Step 8: Display Grid

```
plt.grid(True)
```

Shows grid lines.

Step 9: Display the Graph

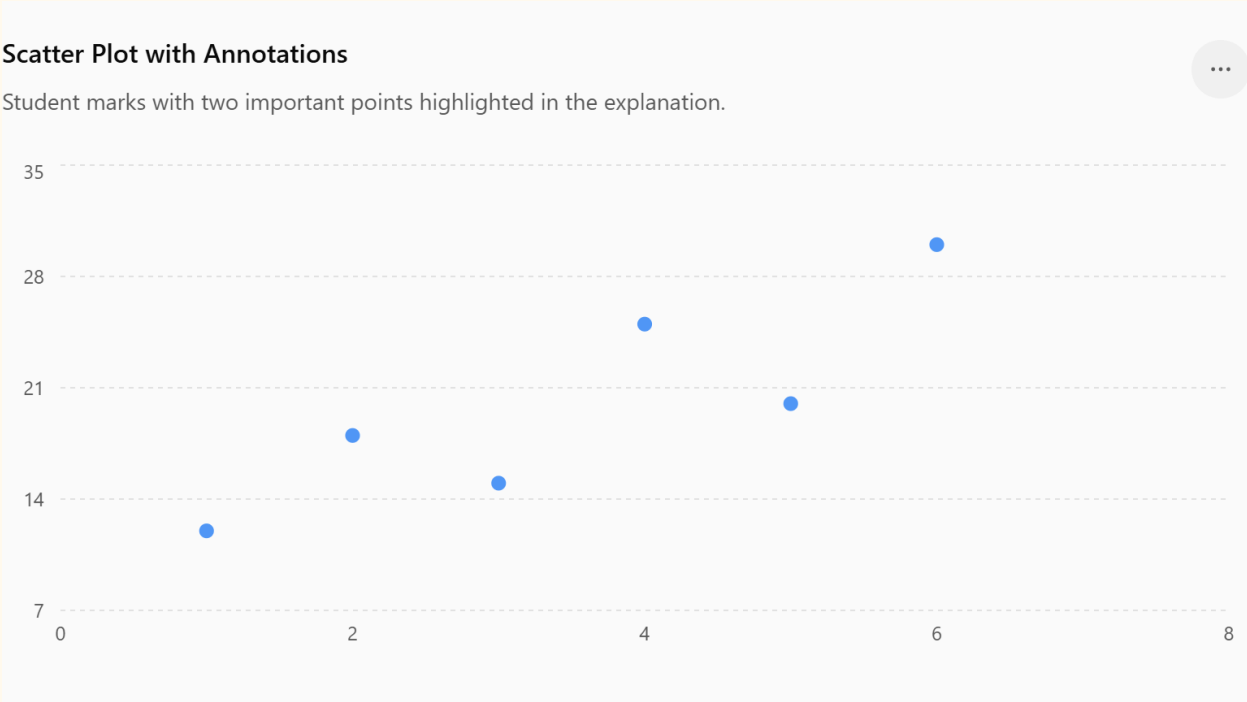
plt.show()

Displays the graph.

Output

Scatter Plot with Annotations

Student marks with two important points highlighted in the explanation.



Expected Output Description

The output is a **scatter plot** containing six blue points.

- **Title:** *Scatter Plot with Annotations*
- X-axis → Student Number
- Y-axis → Marks

- Blue dots represent students' marks.
- The point **(6, 30)** is labeled "**Highest Point**" with a **red arrow** pointing to it.
- The point **(4, 25)** is labeled "**Good Score**" with a **green arrow** pointing to it.
- Grid lines are displayed for easier reading.

Important Functions Used

Function	Purpose
`plt.scatter()`	Creates a scatter plot
`plt.annotate()`	Adds labels and arrows to important points
`xy`	Specifies the point to annotate
`xytext`	Specifies the position of the annotation text
`arrowprops`	Draws an arrow from the text to the point

`plt.title()`	Adds a title
`plt.xlabel()`	Labels the X-axis
`plt.ylabel()`	Labels the Y-axis
`plt.grid(True)`	Displays grid lines
`plt.show()`	Displays the graph

Advantages of Using Annotations

- Highlights important data points.
- Makes graphs more informative.
- Improves readability.
- Helps identify maximum, minimum, and outlier values.
- Useful in presentations, reports, and research papers.

Applications of Annotations

- Business sales analysis
- Stock market analysis
- Weather forecasting
- Medical reports
- Scientific research
- Machine learning and data analysis

Viva Questions

1. What is an annotation?
Annotation is the process of adding text, labels, or arrows to highlight important points on a graph.

2. Which function is used to add annotations in Matplotlib?
`plt.annotate()`

3. What does `xy` represent?
The coordinates of the point to be annotated.

4. What does `xytext` represent?
The position where the annotation text is displayed.

5. Why are annotations useful?
They highlight important points and make graphs easier to understand.

Exam Definition (2 Marks)

Annotation is the process of adding text labels and arrows to specific data points in a graph to highlight important information. In Matplotlib, annotations are created using the `plt.annotate()` function.

9Q) Explain the steps to create multiple subplots in matplotlib with examples.

Ans:

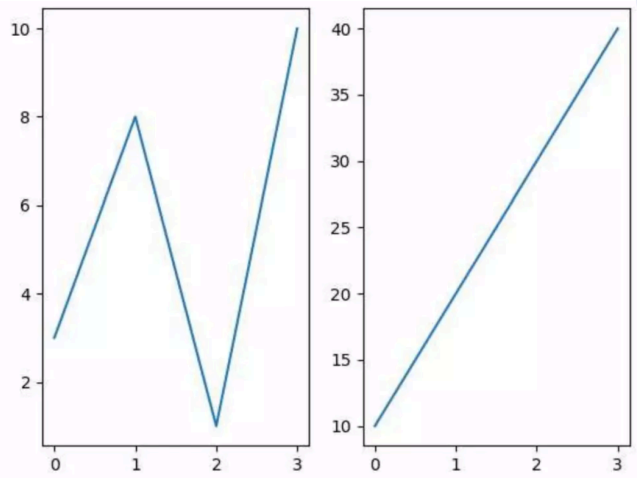
Display Multiple Plots

With the subplot() function you can draw multiple plots in one figure:

- The subplot() function takes three arguments that describes the layout of the figure.
- The layout is organized in rows and columns, which are represented by the first and second argument.
- The third argument represents the index of the current plot.

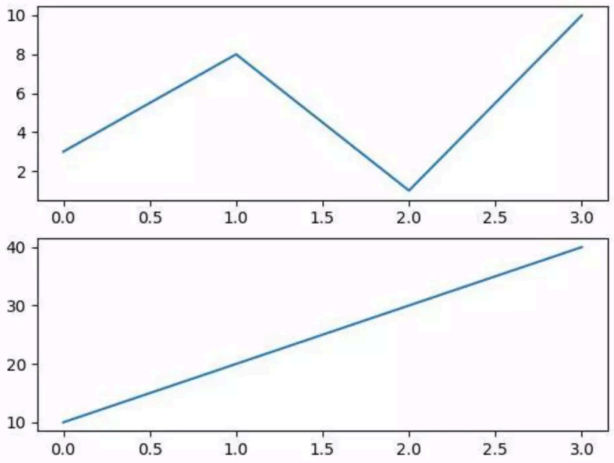
Display Multiple Plots

```
import matplotlib.pyplot as plt
import numpy as np
#plot 1
x = np.array([0,1,2,3])
y = np.array([3,8,1,10])
plt.subplot(1,2,1)
plt.plot(x,y)
#plot 2
x = np.array([0,1,2,3])
y = np.array([10,20,30,40])
plt.subplot(1,2,2)
plt.plot(x,y)
plt.show()
```



Display Multiple Plots

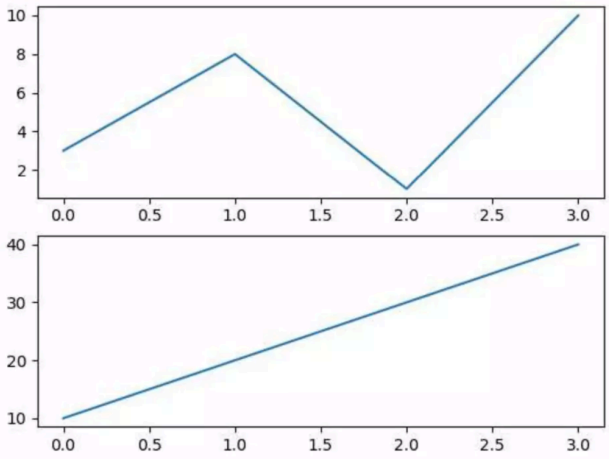
```
import matplotlib.pyplot as plt
import numpy as np
#plot 1
x = np.array([0,1,2,3])
y = np.array([3,8,1,10])
plt.subplot(2,1,1)
plt.plot(x,y)
#plot 2
x = np.array([0,1,2,3])
y = np.array([10,20,30,40])
plt.subplot(2,1,2)
plt.plot(x,y)
plt.show()
```



Super Title

You can add a title to the entire figure with the suptitle() function:

```
import matplotlib.pyplot as plt
import numpy as np
#plot 1
x = np.array([0,1,2,3])
y = np.array([3,8,1,10])
plt.subplot(2,1,1)
plt.plot(x,y)
#plot 2
x = np.array([0,1,2,3])
y = np.array([10,20,30,40])
plt.subplot(2,2,1)
plt.plot(x,y)
plt.suptitle("My Data")
plt.show()
```



Definition

A **subplot** is a way of displaying **multiple graphs in a single figure (window)**. Instead of opening separate windows for different graphs, Matplotlib allows us to display multiple graphs together. This makes it easier to compare different datasets or different types of charts in one place.

Why do we Use Subplots?

Subplots are used to:

- Compare two or more graphs.
- Display multiple datasets in a single figure.
- Save screen space.
- Improve the readability of graphs.
- Analyze related data more effectively.

Function Used

Matplotlib provides the `subplot()` function to create multiple graphs within the same figure.

Syntax:

plt.subplot(rows, columns, position)

Where:

- **rows** → Number of rows in the figure.

- **columns** → Number of columns in the figure.
- **position** → Position of the current subplot.

Steps to Create Multiple Subplots

Step 1: Import the Matplotlib Library

First, import the `matplotlib.pyplot` module, which contains the plotting functions.

Step 2: Prepare the Data

Create the data that will be displayed on the graphs. The data may consist of x-axis values and one or more sets of y-axis values.

Step 3: Create the First Subplot

Use the `plt.subplot()` function to divide the figure into the required number of rows and columns and select the first position.

Step 4: Plot the First Graph

Draw the first graph using any plotting function such as `plt.plot()`, `plt.bar()`, or `plt.scatter()`. Add a title if required.

Step 5: Create the Next Subplot

Call `plt.subplot()` again with the same number of rows and columns but specify the next position.

Step 6: Plot the Second Graph

Draw the second graph and add the necessary title, labels, or formatting.

Step 7: Display the Figure

Finally, use `plt.show()` to display all the subplots together in a single figure.

How `plt.subplot()` Works

The `subplot(rows, columns, position)` function divides the figure into a grid.

For example:

- `plt.subplot(1,2,1)` divides the figure into **1 row and 2 columns** and selects the **first (left) subplot**.
- `plt.subplot(1,2,2)` selects the **second (right) subplot**.
- `plt.subplot(2,2,3)` divides the figure into **2 rows and 2 columns** and selects the **third (bottom-left) subplot**.

Advantages of Subplots

- Displays multiple graphs in a single figure.
- Makes comparison between graphs easier.
- Saves screen space.
- Organizes graphs neatly.
- Useful for reports, presentations, and data analysis.

Applications of Subplots

Subplots are widely used in:

- Comparing students' marks.
- Monthly sales analysis.
- Weather forecasting.
- Scientific experiments.
- Stock market analysis.
- Machine learning and data visualization.

Important Functions Used

- `plt.subplot()` – Creates multiple subplots.
- `plt.plot()` – Draws a line graph.
- `plt.bar()` – Draws a bar chart.
- `plt.scatter()` – Draws a scatter plot.
- `plt.title()` – Adds a title.
- `plt.xlabel()` – Labels the x-axis.
- `plt.ylabel()` – Labels the y-axis.
- `plt.show()` – Displays all subplots.

Conclusion

Subplots are an important feature of Matplotlib that allow multiple graphs to be displayed in a single figure. They help compare different datasets easily, save space, and improve the presentation of data. By

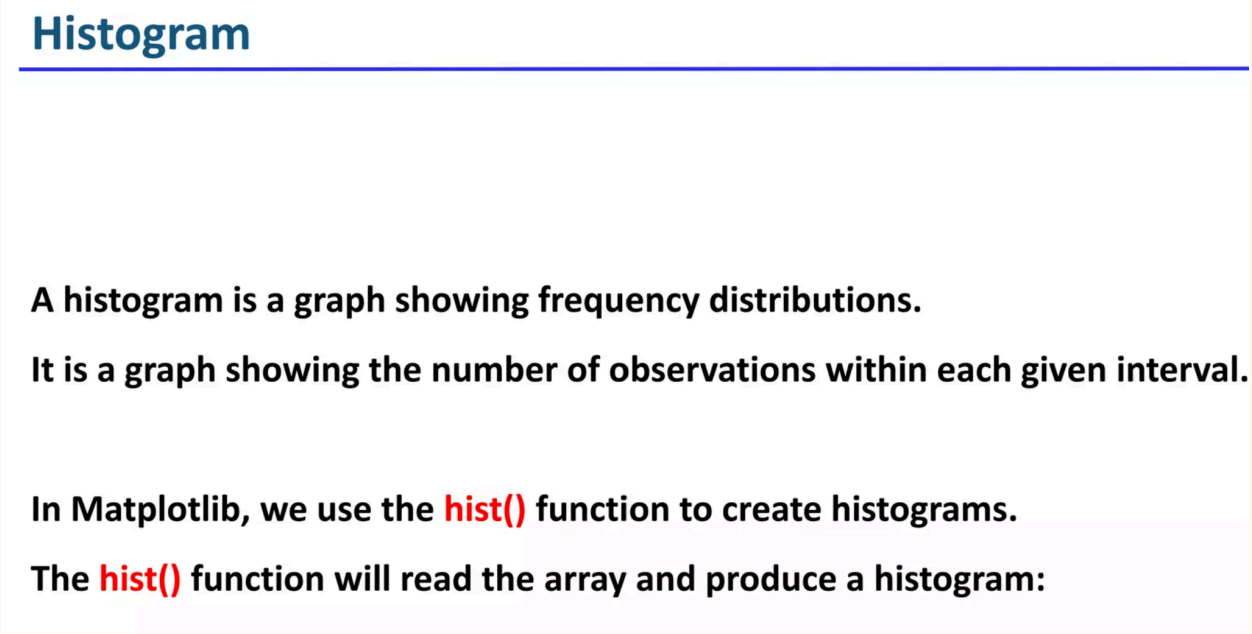
using the `plt.subplot()` function, different types of graphs such as line, bar, scatter, and pie charts can be arranged in an organized layout.

Exam Definition (2 Marks)

Subplots are multiple graphs displayed in a single figure. They are created using the `plt.subplot(rows, columns, position)` function, which divides the figure into rows and columns and places each graph at a specified position.

10Q) Explain Histogram with steps in matplotlib with example.

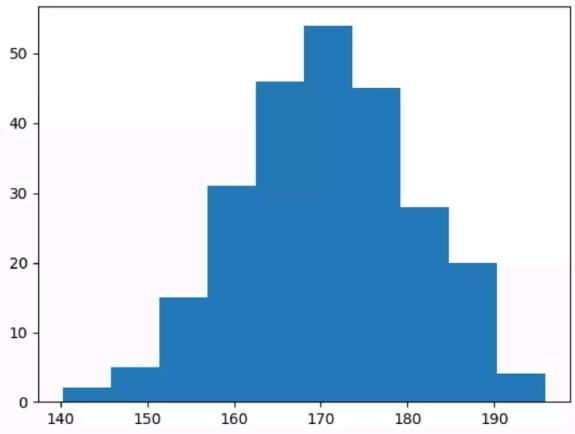
Ans:



Histogram

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.normal(170,10,250)
plt.hist(x)
plt.show()
```



Definition

A **Histogram** is a graphical representation of the **distribution (frequency)** of numerical data. It groups data into **intervals called bins** and shows how many values fall into each interval using bars.

Unlike a bar chart, the bars in a histogram **touch each other** because the data is continuous.

Key Features of a Histogram

- Used to display the **frequency distribution** of numerical data.
- Data is divided into **bins (intervals)**.
- The height of each bar represents the **frequency** of values in that bin.
- Bars **touch each other**, indicating continuous data.
- Helps identify the shape and spread of the data.

Purpose of a Histogram

A histogram is used to:

- Analyze the distribution of data.
- Find the frequency of values.
- Identify patterns, peaks, and gaps.
- Detect outliers.
- Understand whether data is symmetric or skewed.

Real-Life Applications

Histograms are commonly used in:

- Exam score analysis
- Age distribution of people
- Salary distribution of employees
- Daily rainfall measurements
- Heights and weights of students
- Manufacturing quality control

Function Used

plt.hist(data, bins=number_of_bins)

Syntax

plt.hist(data, bins=5)

Parameters

- **data** → List or array of numerical values.
- **bins** → Number of intervals into which the data is divided.

Steps to Create a Histogram

Step 1: Import the Matplotlib Library

import matplotlib.pyplot as plt

Imports the `pyplot` module.

Step 2: Create the Data

marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75,

78, 80, 82, 85, 88, 90, 92, 95]

Stores students' marks.

Step 3: Create the Histogram

plt.hist(marks, bins=5)

Divides the marks into **5 intervals (bins)** and draws the histogram.

Step 4: Add Title and Labels

plt.title("Histogram of Student Marks")

plt.xlabel("Marks")

plt.ylabel("Number of Students")

Adds the title and axis labels.

Step 5: Display the Histogram

plt.show()

Displays the histogram.

Complete Program

```
import matplotlib.pyplot as plt


# Student Marks
marks = [45, 50, 55, 60, 62, 65, 68, 70, 72,
         75, 78, 80, 82, 85, 88, 90, 92, 95]


# Histogram
plt.hist(marks, bins=5, color='skyblue', edgecolor='black')


# Title and Labels
plt.title("Histogram of Student Marks")
plt.xlabel("Marks")
```

```
plt.ylabel("Number of Students")


# Display
plt.show()
```

Explanation of the Program

Import Library

```
import matplotlib.pyplot as plt
```

Imports the plotting library.

Create Data

```
marks = [...]
```

Stores the marks of students.

Create Histogram

```
plt.hist(marks, bins=5)
```

Creates a histogram by dividing the data into **5 equal intervals.**

Add Title

```
plt.title("Histogram of Student Marks")
```

Adds the graph title.

Add Axis Labels

plt.xlabel("Marks")

plt.ylabel("Number of Students")

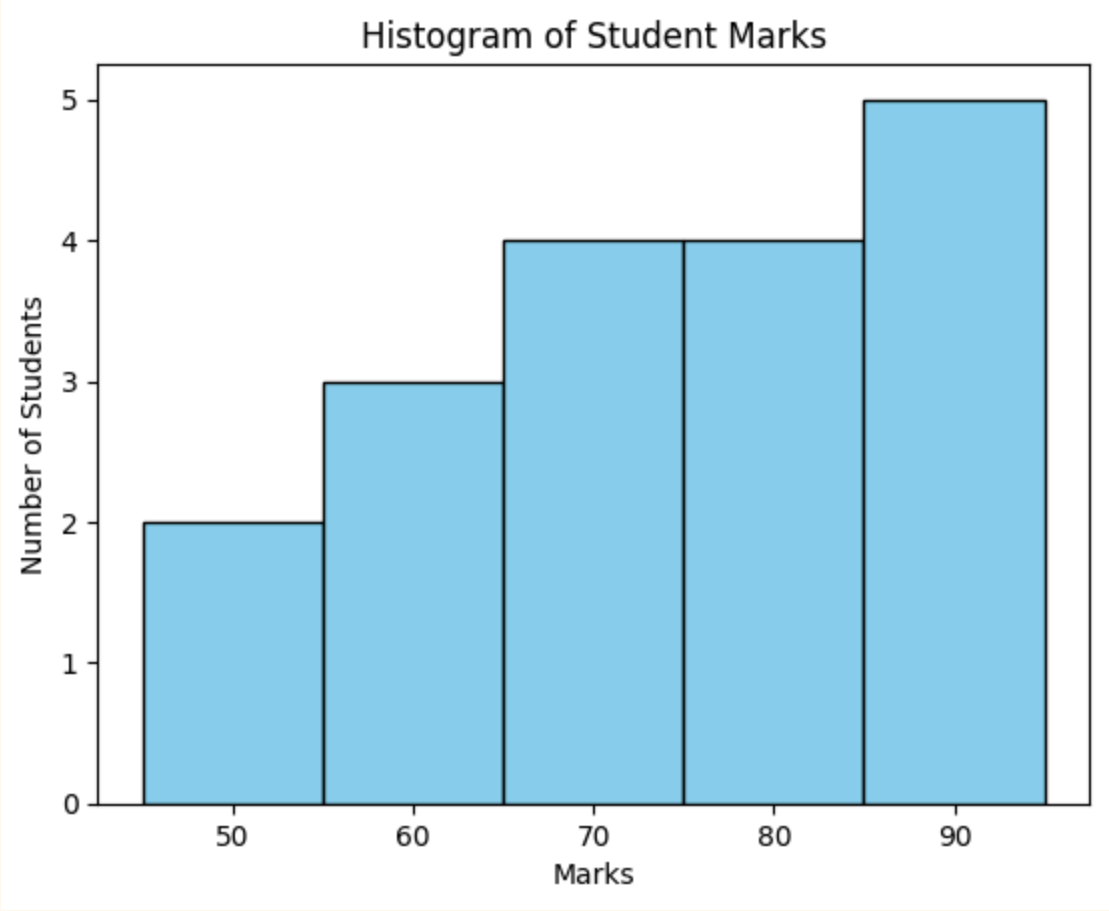
Labels the x-axis and y-axis.

Display the Graph

plt.show()

Displays the histogram.

Output:



Output Description

The output is a **histogram** showing the distribution of students' marks.

- The **x-axis** represents the **marks**.
- The **y-axis** represents the **number of students (frequency)**.
- The data is divided into **5 bins**.
- The height of each bar indicates how many students fall within that range of marks.
- The bars are **touching each other** because the data is continuous.

Difference Between Histogram and Bar Chart

Histogram	Bar Chart
Used for continuous numerical data	Used for categorical data
Bars touch each other	Bars have gaps between them
Shows frequency distribution	Compares different categories
X-axis contains intervals (bins)	X-axis contains category names

Advantages of Histogram

- Easy to understand data distribution.
- Shows the frequency of values.
- Helps identify patterns and trends.
- Detects outliers and gaps.
- Useful in statistical analysis.

Limitations of Histogram

- Suitable only for numerical data.
- Choosing too many or too few bins can affect interpretation.
- Exact individual values cannot be identified.

Applications

Histograms are widely used in:

- Education (exam marks)
- Banking (income distribution)
- Healthcare (patient age distribution)
- Weather forecasting
- Business and sales analysis
- Manufacturing and quality control
- Data Science and Machine Learning

Important Functions Used

Function	Purpose
`plt.hist()`	Creates a histogram
`bins`	Divides data into intervals
`plt.title()`	Adds a title
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis
`plt.show()`	Displays the graph

Viva Questions

1. What is a histogram?

A histogram is a graph that shows the frequency distribution of numerical data.

2. Which function is used to create a histogram in Matplotlib?
`plt.hist()`

3. What are bins?
Bins are intervals used to group continuous data in a histogram.

4. Why do the bars touch each other in a histogram?
Because a histogram represents continuous numerical data.

5. What is the difference between a histogram and a bar chart?
A histogram is used for continuous data with touching bars, while a bar chart is used for categorical data with gaps between bars.

Exam Definition (2 Marks)

A histogram is a graph that represents the frequency distribution of continuous numerical data by dividing it into intervals called bins. In Matplotlib, a histogram is created using the `plt.hist()` function.

11Q) Explain How Matplotlib Configurations and Style Sheets Are Used to Maintain Consistency Across Plots, and Demonstrate Their Application.

Ans:

Definition

Matplotlib configurations and **style sheets** are features that allow us to apply the **same appearance and formatting** to multiple plots. They help

maintain a consistent look by setting common properties such as colors, fonts, line styles, grid styles, and figure sizes.

What are Matplotlib Configurations?

Matplotlib **configurations** are settings that control the appearance of plots. These settings can be changed to customize graphs according to our needs.

Configurations help to:

- Set the default figure size.
- Change font size and font style.
- Set default colors.
- Change line width.
- Enable or disable grids.
- Customize axes and labels.

These settings apply to all plots created after the configuration is set.

What are Style Sheets?

A **style sheet** is a collection of predefined formatting settings used to give plots a consistent appearance.

Instead of changing each property manually, we can apply a style sheet using a single command.

Matplotlib provides many built-in style sheets.

Some commonly used styles are:

- `default`
- `ggplot`
- `seaborn-v0_8`
- `dark_background`
- `classic`
- `Solarize_Light2`
- `fivethirtyeight`

Why are Style Sheets Used?

Style sheets are used to:

- Maintain the same appearance across multiple plots.
- Improve readability.
- Save time by avoiding repeated formatting.
- Produce professional-looking graphs.
- Ensure consistency in reports and presentations.

Function Used

Apply a Style Sheet

plt.style.use('style_name')

Example

plt.style.use('ggplot')

This applies the **ggplot** style to all plots.

Viewing Available Styles

print(plt.style.available)

This displays all the built-in style sheets available in Matplotlib.

Example 1: Using a Style Sheet

Program

import matplotlib.pyplot as plt

Apply style

plt.style.use('ggplot')

Data

x = [1,2,3,4,5]

y = [10,20,15,25,30]

Plot

```
plt.plot(x, y, marker='o')


plt.title("Sales Growth")

plt.xlabel("Year")

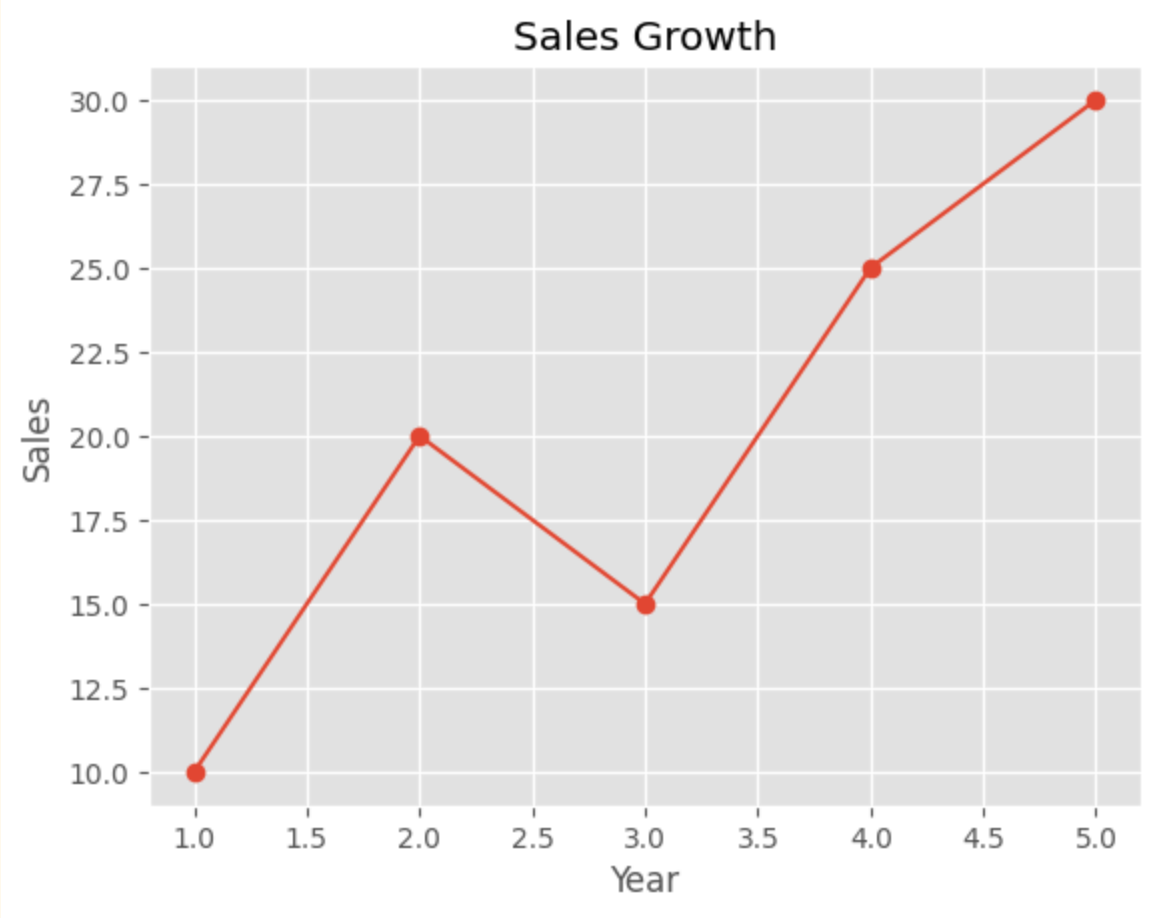
plt.ylabel("Sales")


plt.show()
```

Explanation

- `plt.style.use('ggplot')` applies the **ggplot** style.
- `plt.plot()` creates the line graph.
- `marker='o'` displays circular markers.
- `title()`, `xlabel()`, and `ylabel()` add labels.

Output



Output Description

The graph uses the **ggplot** style.

- Gray background with grid lines.
- Styled axes and fonts.
- Circular markers.
- Professional appearance.
- Consistent formatting.

Example 2: Using Matplotlib Configurations

Program

```
import matplotlib.pyplot as plt


# Configuration settings

plt.rcParams['figure.figsize'] = (6,4)

plt.rcParams['font.size'] = 12

plt.rcParams['lines.linewidth'] = 3


# Data

x = [1,2,3,4,5]

y = [2,4,6,8,10]


# Plot

plt.plot(x,y)

plt.title("Line Graph")

plt.xlabel("X-axis")

plt.ylabel("Y-axis")
```

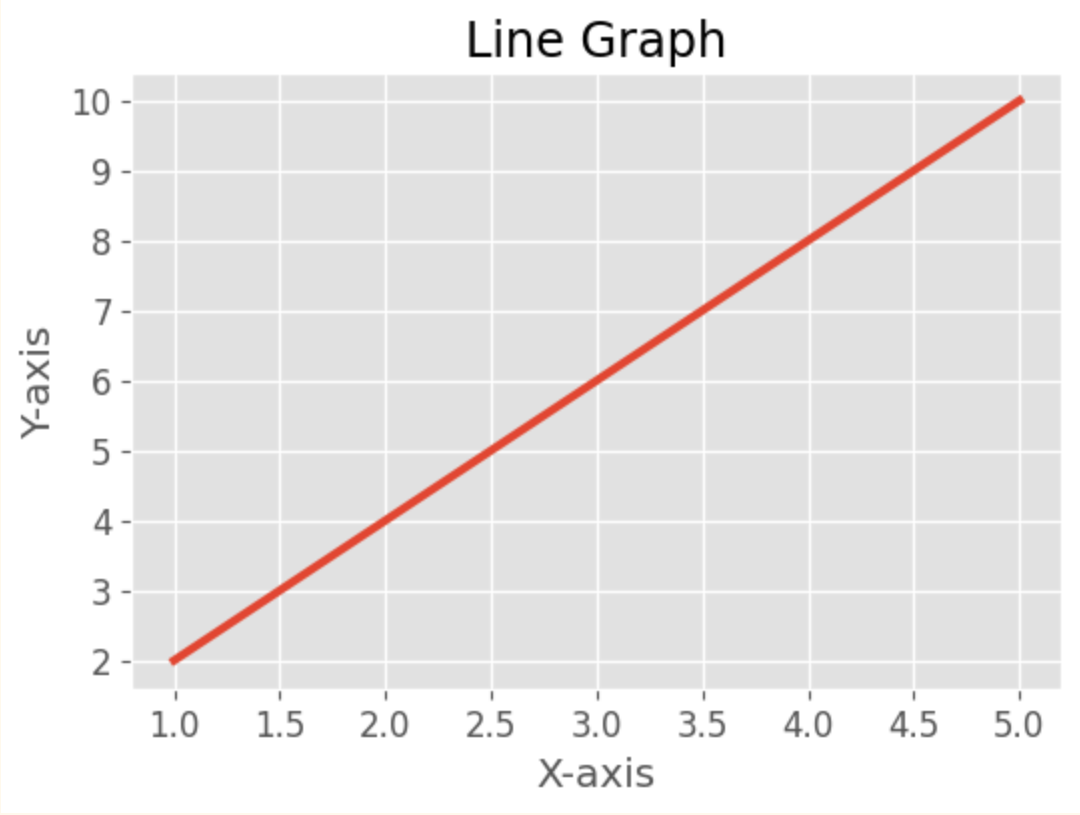
```
plt.grid(True)

plt.show()
```

Explanation

- `figure.figsize` sets the size of the figure.
- `font.size` changes the default font size.
- `lines.linewidth` changes the thickness of the line.
- `grid(True)` displays grid lines.

Output



Output Description

The graph displays:

- Larger figure size.
- Bigger text.
- Thick line.
- Grid lines.
- Uniform appearance.

Common Configuration Settings

Configuration	Purpose
`figure.figsize`	Sets figure size
`font.size`	Changes font size
`lines.linewidth`	Changes line thickness
`axes.titlesize`	Changes title size
`axes.labelsize`	Changes axis label size

`grid.color`	Changes grid color

Advantages of Configurations and Style Sheets

- Gives all graphs the same appearance.
- Saves time.
- Improves readability.
- Creates professional-looking graphs.
- Makes reports and presentations attractive.
- Reduces repeated formatting code.

Applications

Configurations and style sheets are widely used in:

- Data analysis.
- Business reports.
- Scientific research.
- Dashboards.
- Machine Learning projects.
- Academic presentations.

Important Functions

Function	Purpose
`plt.style.use()`	Applies a style sheet
`plt.style.available`	Lists available styles
`plt.rcParams`	Changes default configuration settings
`plt.plot()`	Draws a line graph
`plt.grid()`	Displays grid lines
`plt.show()`	Displays the graph

Viva Questions

1. What is a style sheet in Matplotlib?

A style sheet is a predefined set of formatting options that gives plots a consistent appearance.

2. Which function is used to apply a style sheet?
`plt.style.use()`

3. What is `rcParams`?
`rcParams` is a dictionary used to change the default configuration settings of Matplotlib.

4. Why are style sheets used?
To maintain a consistent appearance across multiple plots.

5. Name any four built-in Matplotlib styles.

- `ggplot`
- `classic`
- `dark_background`
- `Solarize_Light2`

Exam Definition (2 Marks)

Matplotlib configurations and style sheets are used to maintain a consistent appearance across multiple plots by controlling properties such as colors, fonts, line styles, grid styles, and figure size. Style sheets are applied using `plt.style.use()`, while configuration settings are changed using `plt.rcParams`.

12Q) Generate a line plot with the following style properties:

- Line style must be dotted.
- Line color should be blue.
- Show the legend at the lower right location.
- X-axis label should be "Period Number".
- Y-axis label should be "Attainment Value".

Ans:

Program

```
import matplotlib.pyplot as plt


# Data

period = [1, 2, 3, 4, 5]

attainment = [60, 70, 75, 85, 90]


# Line Plot

plt.plot(period, attainment,

        linestyle=':',

        color='blue',

        label='Attainment')
```

```
# Title and Labels

plt.title("Attainment over Periods")

plt.xlabel("Period Number")

plt.ylabel("Attainment Value")


# Legend

plt.legend(loc='lower right')


# Grid

plt.grid(True)


# Display the graph

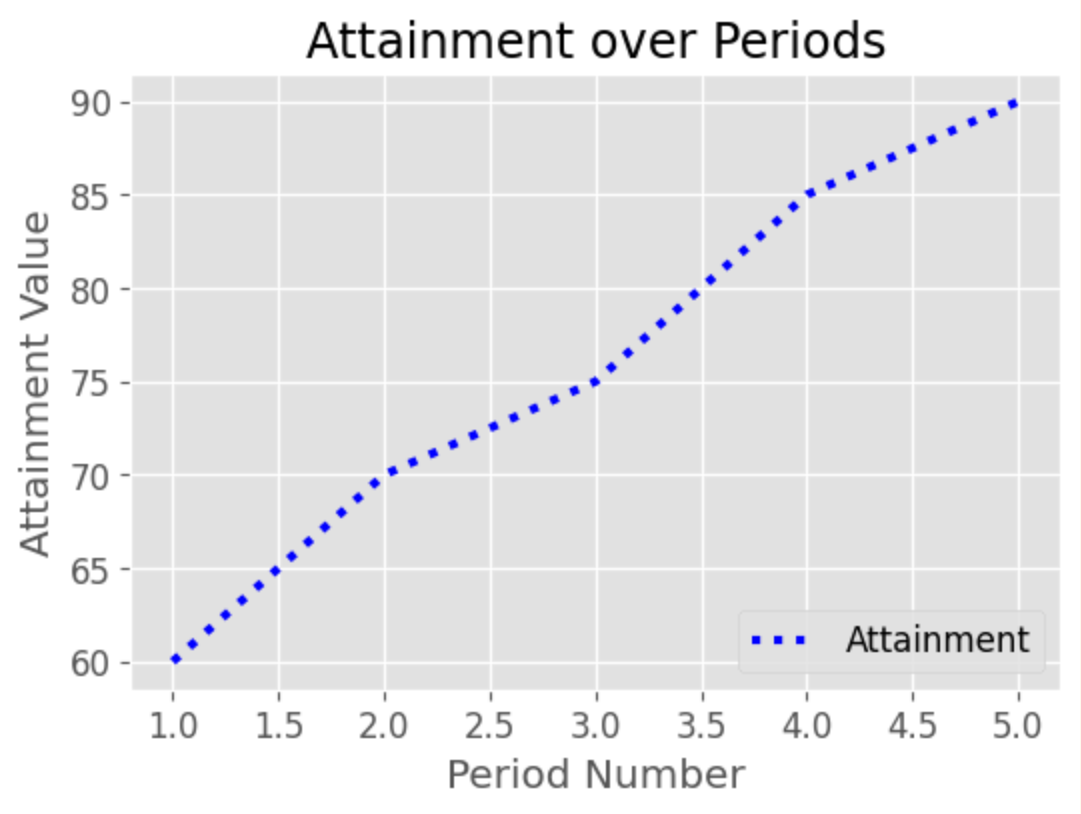
plt.show()
```

Explanation

- `linestyle=':'` → Draws the line in **dotted** style.
- `color='blue'` → Sets the line color to **blue**.
- `label='Attainment'` → Gives a label for the legend.

- `plt.legend(loc='lower right')` → Displays the legend at the **lower-right corner**.
- `plt.xlabel("Period Number")` → Labels the x-axis.
- `plt.ylabel("Attainment Value")` → Labels the y-axis.
- `plt.grid(True)` → Displays grid lines.
- `plt.show()` → Displays the graph.

Output



Output Description

- A **blue dotted line** connects the data points.
- The **x-axis** is labeled "**Period Number**".
- The **y-axis** is labeled "**Attainment Value**".

- The **legend** appears at the **lower-right** corner.
- Grid lines improve readability.
- The title "**Attainment over Periods**" is displayed at the top.

13Q) How to represent time series and geographical data using Matplotlib?

Ans:

Matplotlib is a powerful Python library used for data visualization. It can represent different types of data, including **time series data** and **geographical data**. These visualizations help users understand trends, patterns, and locations more effectively.

1. Time Series Data

Definition

Time series data is a sequence of data points collected or recorded at regular intervals of time such as daily, monthly, or yearly.

It is commonly represented using a **line plot** because it clearly shows how values change over time.

Applications of Time Series Data

- Stock market prices
- Monthly sales

- Temperature records
- Rainfall analysis
- Population growth
- Website traffic

Steps to Represent Time Series Data

1. Import Matplotlib.
2. Create time values and corresponding data values.
3. Use `plt.plot()` to create a line graph.
4. Add title and axis labels.
5. Display the graph using `plt.show()`.

Program

```
import matplotlib.pyplot as plt


# Sample Data

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]

sales = [120, 150, 170, 160, 190, 210]


# Plot

plt.plot(months, sales, marker='o', color='blue')
```

```
# Title and Labels

plt.title("Monthly Sales")

plt.xlabel("Months")

plt.ylabel("Sales")


# Grid

plt.grid(True)


# Display

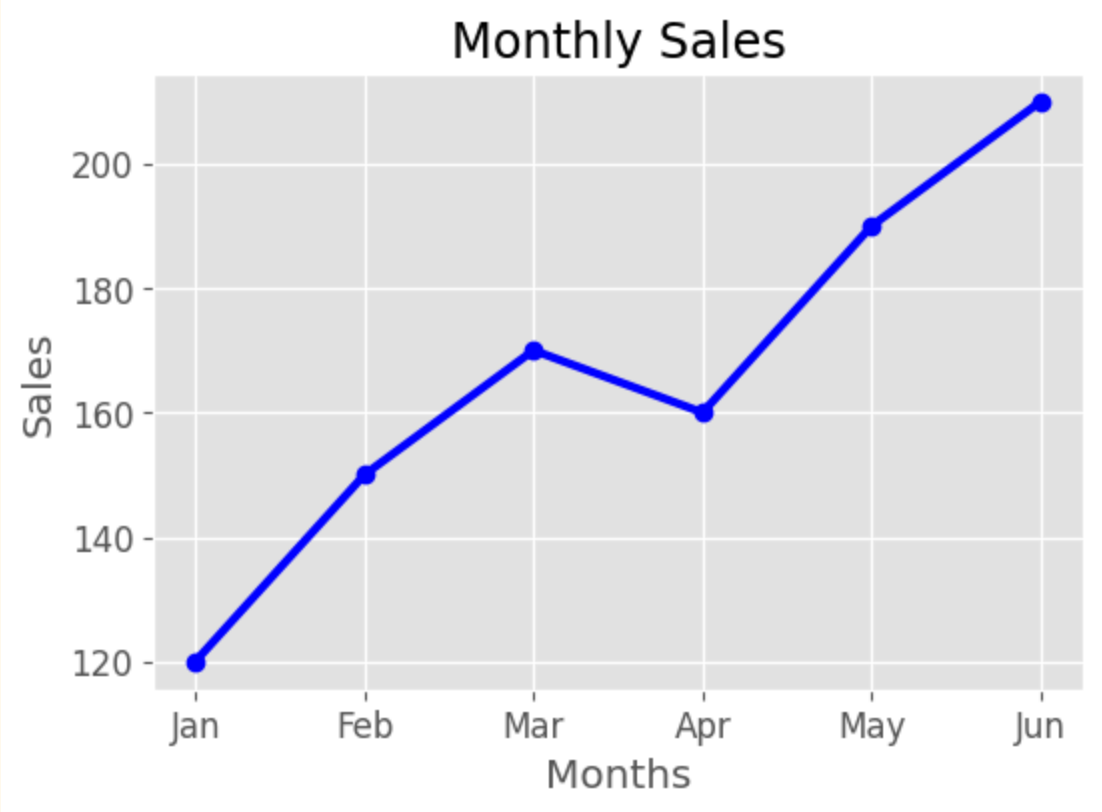
plt.show()
```

Explanation of the Code

- `import matplotlib.pyplot as plt`
  Imports the Matplotlib plotting module.
- `months`
  Stores the months on the x-axis.
- `sales`
  Stores monthly sales values.
- `plt.plot(months, sales)`
  Creates a line graph.
- `marker='o'`
  Displays circular markers at each data point.

- `color='blue'`
  Sets the line color to blue.
- `plt.title()`
  Adds the graph title.
- `plt.xlabel()`
  Labels the x-axis.
- `plt.ylabel()`
  Labels the y-axis.
- `plt.grid(True)`
  Displays grid lines.
- `plt.show()`
  Displays the graph.

Output



Output Description

- The graph shows the **monthly sales trend**.
- The **x-axis** represents the months.
- The **y-axis** represents sales.
- Blue line and circular markers indicate each month's sales.
- Grid lines improve readability.

Advantages of Time Series Graph

- Shows trends over time.
- Easy to compare different periods.
- Helps in forecasting future values.

- Easy to understand.

2. Geographical Data

Definition

Geographical data represents the locations of places on a map using **latitude and longitude coordinates.**

Matplotlib can represent geographical locations using a **scatter plot**, where:

- X-axis → Longitude
- Y-axis → Latitude

Applications of Geographical Data

- GPS navigation
- Weather stations
- Earthquake locations
- Population distribution
- Delivery tracking
- Tourist locations

Steps to Represent Geographical Data

1. Import Matplotlib.
2. Create latitude and longitude values.
3. Use `plt.scatter()` to plot the locations.

4. Add labels and title.
5. Display the graph.

Program

```
import matplotlib.pyplot as plt


# Sample Data

longitude = [78.48, 77.59, 80.27, 72.87]

latitude = [17.38, 12.97, 13.08, 19.07]


cities = ["Hyderabad", "Bengaluru", "Chennai", "Mumbai"]


# Scatter Plot

plt.scatter(longitude, latitude, color='red')


# Add city names

for i in range(len(cities)):

    plt.text(longitude[i], latitude[i], cities[i])
```

```
# Title and Labels

plt.title("Geographical Locations")

plt.xlabel("Longitude")

plt.ylabel("Latitude")


# Grid

plt.grid(True)


# Display

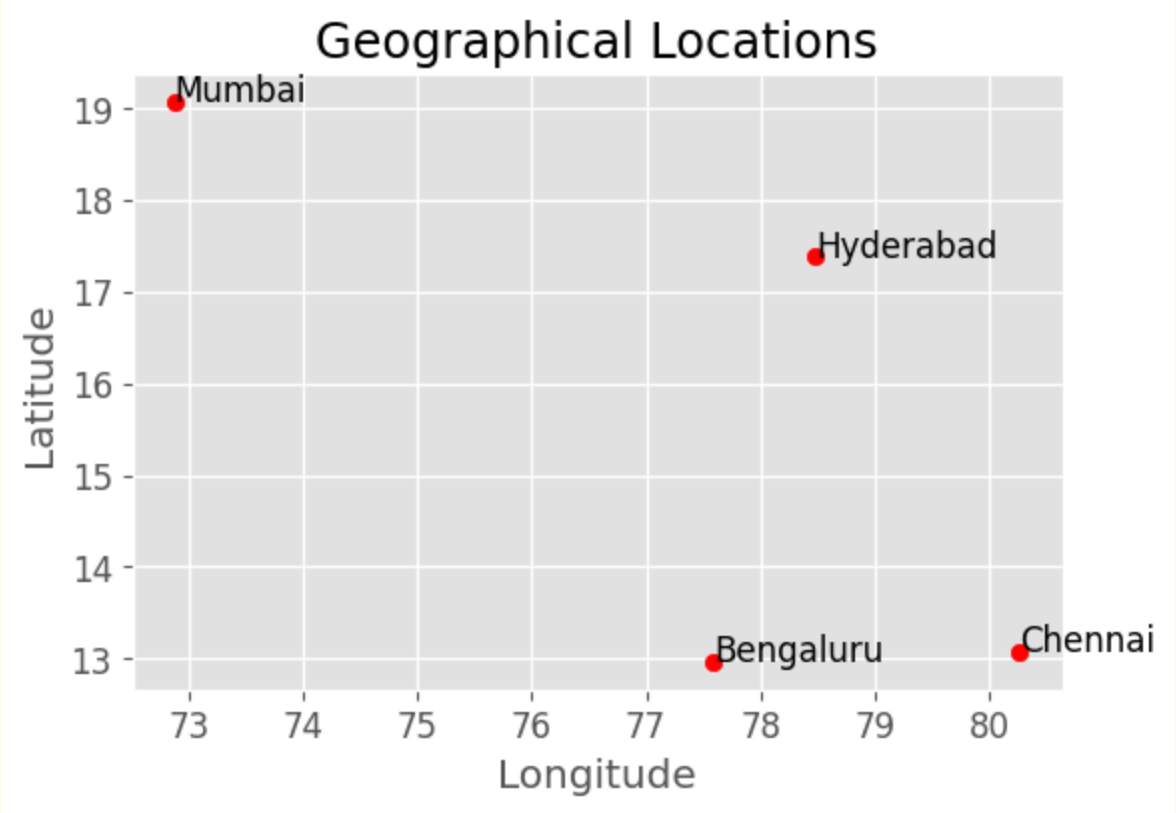
plt.show()
```

Explanation of the Code

- `longitude`
  Stores the longitude values.
- `latitude`
  Stores the latitude values.
- `cities`
  Stores city names.
- `plt.scatter()`
  Plots each city as a point.
- `plt.text()`
  Displays the city name near each point.

- `plt.title()`
  Adds the graph title.
- `plt.xlabel()`
  Labels the longitude axis.
- `plt.ylabel()`
  Labels the latitude axis.
- `plt.grid(True)`
  Displays grid lines.
- `plt.show()`
  Displays the graph.

Output



Output Description

- The graph shows the locations of four cities.
- The **x-axis** represents longitude.
- The **y-axis** represents latitude.
- Each red point represents a city.
- City names are displayed beside the corresponding points.

Difference Between Time Series and Geographical Data

Time Series Data	Geographical Data
Represents data over time	Represents data based on location
Uses time on the x-axis	Uses longitude on the x-axis
Usually represented using a line plot	Usually represented using a scatter plot
Used for trend analysis	Used for location analysis

Advantages

Time Series Data

- Shows trends clearly.
- Helps in forecasting.
- Easy to compare different time periods.
- Useful in business and finance.

Geographical Data

- Shows locations visually.
- Easy to compare different places.
- Useful for navigation and mapping.
- Helps in location-based analysis.

Applications

Time Series

- Stock market analysis
- Weather forecasting
- Sales analysis
- Population growth
- Website traffic monitoring

Geographical Data

- GPS systems
- Disaster management

- Delivery services
- Tourism
- Population mapping

Important Functions

Function	Purpose
`plt.plot()`	Draws a line graph
`plt.scatter()`	Draws a scatter plot
`plt.text()`	Adds labels near points
`plt.title()`	Adds a title
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis
`plt.grid()`	Displays grid lines
`plt.show()`	Displays the graph

Viva Questions

1. What is time series data?
Time series data is data collected at regular intervals over time.

2. Which graph is commonly used for time series data?
A line graph (`plt.plot()`).

3. What is geographical data?
Geographical data represents locations using latitude and longitude coordinates.

4. Which Matplotlib function is used for geographical data?
`plt.scatter()`.

5. Why is `plt.text()` used in geographical plots?
It is used to display labels or names of locations near the plotted points.

Conclusion

Matplotlib provides simple and effective methods for representing both **time series** and **geographical data**. Time series data is generally visualized using **line plots** to show trends over time, while geographical data is represented using **scatter plots** with latitude and longitude coordinates. These visualizations help users analyze data efficiently and make better decisions.

Exam Definition (2 Marks)

Time series data represents values recorded over time and is usually displayed using a line plot (`plt.plot()`). Geographical data represents locations using latitude and longitude coordinates and is commonly displayed using a scatter plot (`plt.scatter()`).

14Q) Explain How the Display Image Feature Can Improve the Readability of a Plot.

Ans:

In Matplotlib, the **display image feature** is used to display images inside a plot using the `imshow()` function. It allows images such as photographs, scanned documents, medical images, satellite images, and heat maps to be displayed clearly.

Displaying images helps users understand visual information more effectively than text or numerical values. It improves the readability of a plot by making data easier to interpret.

Definition

The **display image feature** in Matplotlib is the process of displaying an image inside a plotting window using the `imshow()` function.

It is commonly used to visualize images, matrices, and two-dimensional data.

Function Used

plt.imshow(image)

Syntax

plt.imshow(image)

plt.show()

Parameters

- **image** → The image or matrix to be displayed.

Why Does the Display Image Feature Improve Readability?

Displaying images improves the readability of plots because it:

- Makes information easier to understand.
- Represents visual data clearly.
- Helps identify patterns quickly.
- Improves the appearance of reports and presentations.
- Reduces the need for lengthy textual explanations.
- Makes graphs attractive and informative.

Applications of Display Images

The display image feature is used in:

- Medical image analysis (X-rays, MRI scans)
- Satellite and weather images
- Face recognition
- Computer vision
- Image processing
- Scientific research
- Machine learning

Steps to Display an Image in Matplotlib

Step 1: Import the Required Libraries

Import the Matplotlib library and image reading module.

Step 2: Load the Image

Read the image from the computer using `plt.imread()`.

Step 3: Display the Image

Use the `imshow()` function to display the image.

Step 4: Add a Title (Optional)

Use `plt.title()` to give a title to the image.

Step 5: Display the Figure

Use `plt.show()` to display the image.

Program

```
import matplotlib.pyplot as plt


# Read Image

image = plt.imread("flower.jpg")


# Display Image

plt.imshow(image)


# Title

plt.title("Flower Image")


# Display

plt.show()
```

Explanation of the Program

Import Library

import matplotlib.pyplot as plt

Imports the Matplotlib library.

Read the Image

image = plt.imread("flower.jpg")

Reads the image from the specified file.

Display the Image

plt.imshow(image)

Displays the image in the plotting window.

Add Title

plt.title("Flower Image")

Displays the title above the image.

Show the Image

plt.show()

Displays the final output.

Output



Output Description

- The image is displayed inside the Matplotlib figure.
- The title "**Flower Image**" appears at the top.
- The image is clear and easy to understand.
- The plot window displays the image without distortion.

Improving Readability Using `imshow()`

The `imshow()` function improves readability by:

- Displaying images in high quality.
- Showing color details clearly.

- Making visual comparison easier.
- Helping users understand image-based data quickly.
- Supporting scientific and medical image visualization.

Advantages of Display Image Feature

- Displays images with good clarity.
- Easy to use with a single function.
- Supports many image formats (JPG, PNG, BMP, etc.).
- Useful for image processing applications.
- Improves visualization and understanding.
- Can display grayscale and color images.

Limitations

- Requires an image file or image array.
- Large images consume more memory.
- Not suitable for plotting numerical trends (line, bar, scatter).

Applications

The display image feature is widely used in:

- Medical imaging
- Satellite imaging
- Digital photography
- Computer vision

- Artificial Intelligence
- Deep Learning
- Image processing
- Remote sensing

Important Functions

Function	Purpose
`plt.imread()`	Reads an image file
`plt.imshow()`	Displays the image
`plt.title()`	Adds a title
`plt.axis()`	Shows or hides axes
`plt.show()`	Displays the figure

Viva Questions

1. Which function is used to display an image in Matplotlib?
`plt.imshow()`

2. Which function is used to read an image?
`plt.imread()`

3. What is the purpose of `imshow()`?
It displays an image or a 2D data array in a Matplotlib figure.

4. Name any four applications of image display in Matplotlib.

- Medical imaging
- Satellite imaging
- Computer vision
- Image processing

5. What are the advantages of displaying images in Matplotlib?
It improves readability, provides clear visualization, and helps users understand image data easily.

Conclusion

The **display image feature** in Matplotlib is an effective way to visualize image data. By using the **`imshow()`** function, images can be displayed clearly inside a plotting window, making data easier to understand and analyze. This feature is widely used in image processing, medical imaging, computer vision, and scientific research because it enhances the readability and presentation of visual information.

Exam Definition (2 Marks)

The display image feature in Matplotlib uses the `imshow()` function to display images in a plot. It improves the readability of plots by presenting visual information clearly, making it easier to understand patterns, details, and image-based data.

15Q) Explain Box Plot in Detail and Create a Box Plot that Shows the Distribution of Five Sets of Data.

Ans:

A **Box Plot** (also called a **Box-and-Whisker Plot**) is a statistical graph used to **summarize the distribution of numerical data**. It displays important statistical values such as the **minimum value, first quartile (Q1), median (Q2), third quartile (Q3), maximum value, and outliers**.

A box plot provides a quick summary of the spread, center, and variability of a dataset.

Definition

A **Box Plot** is a graph that represents the distribution of numerical data using a rectangular box and whiskers. It helps in identifying the median, quartiles, spread of data, and outliers.

Components of a Box Plot

A box plot consists of the following parts:

1. Minimum Value

- The smallest value in the dataset (excluding outliers).

2. First Quartile (Q1)

- Represents the **25th percentile**.
- 25% of the data lies below Q1.

3. Median (Q2)

- Represents the **50th percentile**.
- Divides the data into two equal halves.

4. Third Quartile (Q3)

- Represents the **75th percentile**.
- 75% of the data lies below Q3.

5. Maximum Value

- The largest value in the dataset (excluding outliers).

6. Whiskers

- Lines extending from the box to the minimum and maximum values.

7. Outliers

- Values that are much smaller or larger than the rest of the data.

- Displayed as separate points outside the whiskers.

Sample Box Plot Structure

Minimum Q1 Median Q3 Maximum

|-------|=========|=========|-------|

<------ Box ------->

Why Do We Use a Box Plot?

A box plot is used to:

- Summarize data distribution.
- Compare multiple datasets.
- Detect outliers.
- Measure data spread.
- Find the median and quartiles.
- Identify skewness.

Applications

Box plots are widely used in:

- Student marks analysis.
- Salary comparison.
- Medical research.
- Scientific experiments.

- Business analytics.
- Machine Learning and Data Science.

Function Used

plt.boxplot(data)

Syntax

plt.boxplot(data)

Where:

- **data** → A list containing one or more datasets.

Steps to Create a Box Plot

Step 1: Import the Matplotlib Library

import matplotlib.pyplot as plt

Imports the Matplotlib plotting library.

Step 2: Create Five Sets of Data

data = [

[10,12,14,16,18],

[20,22,24,26,28],

```
    [15,18,20,22,25],
    [30,32,35,36,40],
    [12,15,17,19,21]
]
```

Each list represents one dataset.

Step 3: Create the Box Plot

```
plt.boxplot(data)
```

Creates box plots for all five datasets.

Step 4: Add Title and Labels

```
plt.title("Distribution of Five Data Sets")
plt.xlabel("Data Sets")
plt.ylabel("Values")
```

Adds the graph title and axis labels.

Step 5: Display the Plot

```
plt.show()
```

Displays the box plot.

Complete Program

```
import matplotlib.pyplot as plt


# Five datasets
data = [
    [10,12,14,16,18],
    [20,22,24,26,28],
    [15,18,20,22,25],
    [30,32,35,36,40],
    [12,15,17,19,21]
]


# Create box plot
plt.boxplot(data)


# Title and labels
plt.title("Distribution of Five Data Sets")
plt.xlabel("Data Sets")
plt.ylabel("Values")
```

```
# Display plot
plt.show()
```

Explanation of the Program

Import Library

```
import matplotlib.pyplot as plt
```

Imports the plotting module.

Create Data

```
data = [...]
```

Creates five different datasets.

Create Box Plot

```
plt.boxplot(data)
```

Draws five box plots, one for each dataset.

Add Title

```
plt.title("Distribution of Five Data Sets")
```

Adds a title to the graph.

Label the Axes

plt.xlabel("Data Sets")

plt.ylabel("Values")

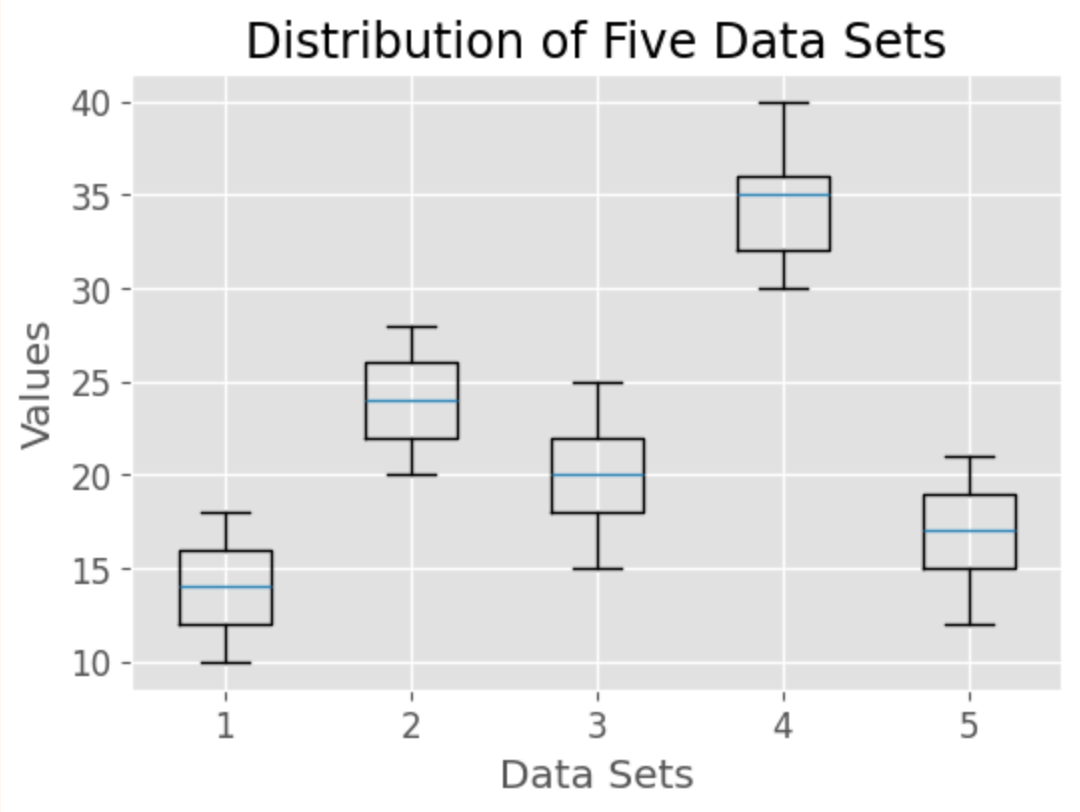
Labels the x-axis and y-axis.

Display the Plot

plt.show()

Displays the graph.

Output



Output Description

The output displays **five box plots**, each representing one dataset.

- The **x-axis** represents the five datasets.
- The **y-axis** represents the data values.
- Each box shows the **first quartile (Q1), median (Q2), and third quartile (Q3)**.
- The whiskers show the **minimum and maximum values**.
- Any outliers (if present) appear as separate points.

Advantages of Box Plot

- Summarizes data using five statistical values.
- Easy to compare multiple datasets.
- Detects outliers.
- Shows data spread and variability.
- Helps identify skewness.

Limitations

- Does not show the exact values of all data points.
- Does not display the frequency distribution.
- Less suitable for very small datasets.

Real-Life Applications

- Comparing students' exam marks.
- Salary analysis of employees.
- Comparing rainfall in different cities.

- Healthcare data analysis.
- Manufacturing quality control.
- Machine Learning and Data Science.

Important Functions

Function	Purpose
`plt.boxplot()`	Creates a box plot
`plt.title()`	Adds a title
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis
`plt.show()`	Displays the graph

Viva Questions

1. What is a box plot?

A box plot is a graph that summarizes the distribution of numerical data using quartiles, median, whiskers, and outliers.

2. Which function is used to create a box plot in Matplotlib?
`plt.boxplot()`

3. What does the box represent?
The box represents the interquartile range (Q1 to Q3).

4. What does the line inside the box represent?
It represents the median (Q2).

5. What are whiskers in a box plot?
Whiskers are lines that extend from the box to the minimum and maximum values (excluding outliers).

Conclusion

A **box plot** is a simple and effective statistical graph used to summarize the distribution of numerical data. It clearly displays the **minimum, maximum, quartiles, median, and outliers**, making it easy to compare multiple datasets. In Matplotlib, the `plt.boxplot()` function is used to create box plots quickly and efficiently.

Exam Definition (2 Marks)

A box plot is a statistical graph that represents the distribution of numerical data using the minimum value, first quartile (Q1), median (Q2), third quartile (Q3), maximum value, and outliers. In Matplotlib, it is created using the `plt.boxplot()` function.

16Q) Analyze the Importance of Customization Options in Matplotlib for Enhancing the Visual Appeal and Interpretability of Plots.

Ans:

Introduction

Matplotlib is one of the most popular Python libraries used for creating graphs and charts. By default, Matplotlib produces simple plots, but these may not always be attractive or easy to understand. **Customization options** allow users to modify the appearance of plots by changing colors, fonts, line styles, markers, labels, legends, grid lines, figure size, and other elements.

Customization makes plots more **attractive, informative, and easier to interpret**, which helps in effective communication of data.

Definition

Customization in Matplotlib refers to modifying the appearance and style of a graph using different formatting options to improve its readability, visual appeal, and clarity.

Why is Customization Important?

Customization is important because it:

- Makes graphs attractive and professional.

- Improves readability.
- Highlights important information.
- Makes comparisons easier.
- Helps communicate data effectively.
- Makes reports and presentations more impressive.

Importance of Customization in Matplotlib

1. Improves Visual Appeal

Customization allows us to use attractive colors, fonts, markers, and styles, making graphs more appealing.

Example:

- Using blue, green, or red colors.
- Using different marker styles.
- Adding grid lines.

2. Enhances Readability

Proper labels, titles, and font sizes make graphs easier to understand.

Example:

- Large title
- Clear axis labels
- Proper font size

This helps viewers quickly understand the graph.

3. Highlights Important Information

Different colors and line styles help emphasize important data.

Example:

- Red line for maximum values.
- Green bars for profit.
- Thick lines for important trends.

4. Makes Data Comparison Easy

Using different colors, legends, and markers helps compare multiple datasets in one graph.

Example:

- Comparing sales of two companies.
- Comparing rainfall of different cities.

5. Improves Data Interpretation

Well-designed plots allow users to identify:

- Trends
- Patterns
- Relationships
- Outliers

more easily.

6. Creates Professional Reports

Customized graphs are suitable for:

- Research papers
- Business reports
- Dashboards
- Academic presentations

Common Customization Features in Matplotlib

1. Title

Used to give a meaningful heading to the graph.

plt.title("Monthly Sales")

Purpose:

- Describes the graph.
- Helps users understand the graph quickly.

2. Axis Labels

Used to label the x-axis and y-axis.

plt.xlabel("Months")

```
plt.ylabel("Sales")
```

Purpose:

- Explains what each axis represents.
- Improves clarity.

3. Colors

Different colors make graphs attractive and distinguish different datasets.

```
plt.plot(x, y, color='blue')
```

Common colors:

- Blue
- Red
- Green
- Black
- Orange

4. Line Style

Changes the appearance of the line.

```
plt.plot(x, y, linestyle='--')
```

Common line styles:

- Solid (`-`)
- Dashed (`--`)
- Dotted (`:`)
- Dash-dot (`-.`)

5. Markers

Markers highlight individual data points.

plt.plot(x, y, marker='o')

Common markers:

- Circle (`o`)
- Square (`s`)
- Triangle (`^`)
- Star (`*`)
- Plus (`+`)

6. Grid

Grid lines make values easier to read.

plt.grid(True)

Benefits:

- Improves readability.
- Helps compare values accurately.

7. Legend

Legends identify multiple datasets.

plt.legend()

Benefits:

- Identifies each line or graph.
- Avoids confusion.

8. Figure Size

Changes the size of the graph.

plt.figure(figsize=(8,5))

Benefits:

- Larger graphs improve visibility.
- Better for presentations.

9. Font Size

Changes the size of titles and labels.

plt.title("Sales", fontsize=16)

Benefits:

- Makes text easy to read.
- Improves presentation.

10. Style Sheets

Applies predefined themes.

plt.style.use("ggplot")

Benefits:

- Professional appearance.
- Consistent formatting.
- Saves time.

Demonstration Program

```
import matplotlib.pyplot as plt


# Data
```

```
months = ["Jan","Feb","Mar","Apr","May"]
sales = [20,30,40,35,50]

# Figure Size
plt.figure(figsize=(7,5))

# Plot
plt.plot(months, sales,
         color='blue',
         linestyle='--',
         marker='o',
         linewidth=2,
         label='Sales')

# Title and Labels
plt.title("Monthly Sales Report", fontsize=15)
plt.xlabel("Months")
plt.ylabel("Sales")
```

```
# Grid
plt.grid(True)


# Legend
plt.legend()

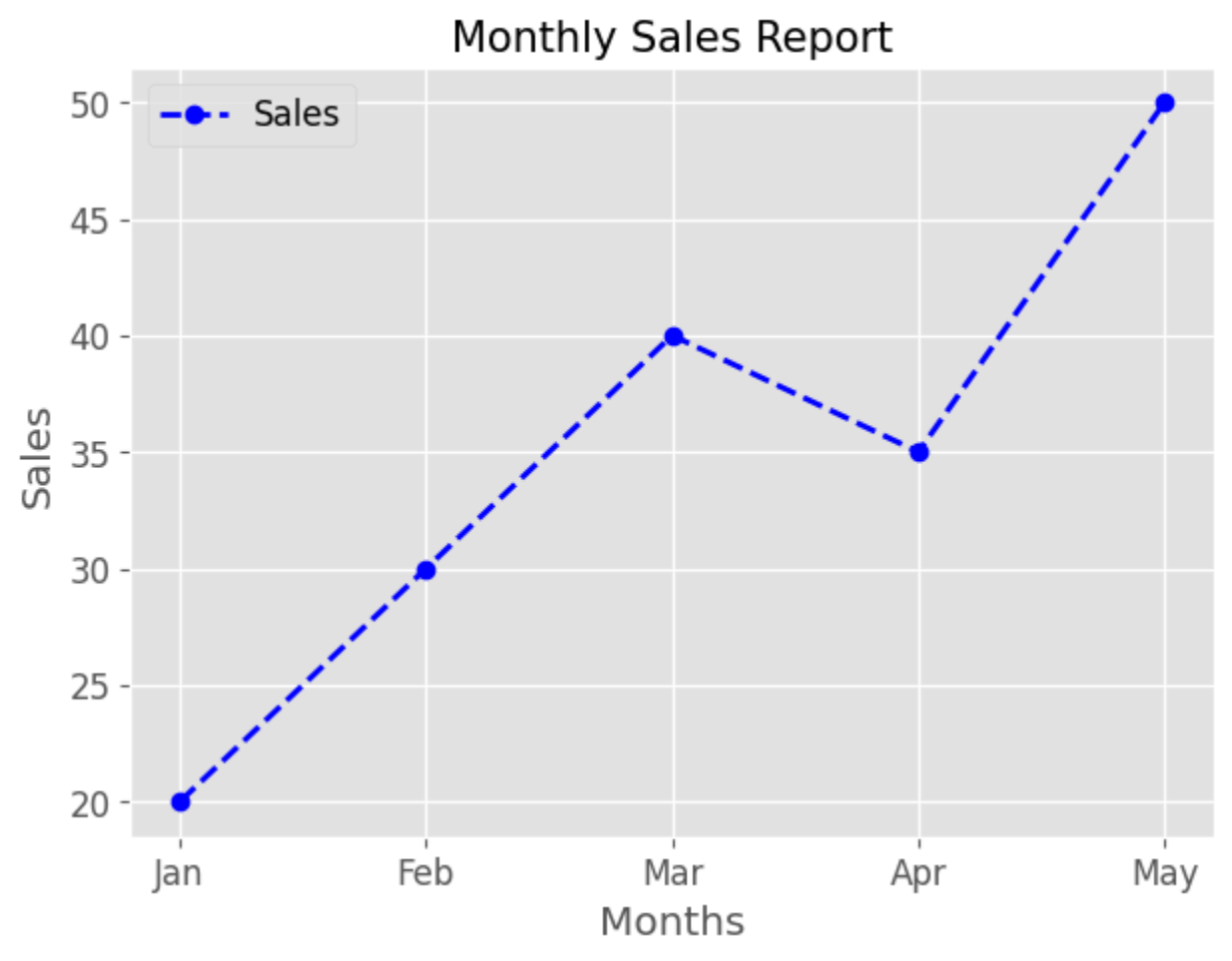

# Display
plt.show()
```

Explanation of the Program

- `plt.figure(figsize=(7,5))` → Sets the figure size.
- `color='blue'` → Changes the line color to blue.
- `linestyle='--'` → Displays a dashed line.
- `marker='o'` → Adds circular markers.
- `linewidth=2` → Makes the line thicker.
- `plt.title()` → Adds a graph title.
- `plt.xlabel()` → Labels the x-axis.
- `plt.ylabel()` → Labels the y-axis.
- `plt.grid(True)` → Displays grid lines.
- `plt.legend()` → Displays the legend.

- `plt.show()` → Displays the graph.

Output



Output Description

The graph displays:

- A **blue dashed line** representing monthly sales.
- **Circular markers** highlighting each data point.
- A **title** at the top.

- Clearly labeled x-axis and y-axis.
- A **legend** identifying the dataset.
- **Grid lines** to improve readability.
- A larger figure size for better visibility.

Advantages of Customization

- Improves the appearance of graphs.
- Makes graphs easier to understand.
- Highlights important information.
- Helps compare multiple datasets.
- Makes reports and presentations professional.
- Improves communication of data.

Applications

Customization is widely used in:

- Business reports
- Data Science
- Machine Learning
- Financial analysis
- Scientific research
- Academic projects
- Dashboards

Important Customization Functions

Function	Purpose
`plt.title()`	Adds a title
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis
`color`	Changes graph color
`linestyle`	Changes line style
`marker`	Adds markers
`plt.grid()`	Displays grid lines
`plt.legend()`	Shows the legend
`plt.figure()`	Sets figure size
`plt.style.use()`	Applies a style sheet
`fontsize`	Changes text size

Viva Questions

1. What is customization in Matplotlib?
Customization is the process of modifying the appearance of plots to improve readability and presentation.

2. Why is customization important?
It improves visual appeal, readability, and effective communication of data.

3. Which function is used to add a title?
`plt.title()`

4. Which function displays the legend?
`plt.legend()`

5. Name any four customization options in Matplotlib.

- Title
- Colors
- Markers
- Grid
- Legend
- Line style

Conclusion

Customization options in Matplotlib play a vital role in creating **clear, attractive, and informative visualizations**. Features such as **titles, axis**

labels, colors, markers, line styles, legends, grids, figure size, and style sheets improve both the **visual appeal** and the **interpretability** of plots. By using these customization features effectively, users can communicate data more clearly and make graphs suitable for reports, presentations, research, and business analysis.

Exam Definition (2 Marks)

Customization in Matplotlib refers to modifying the appearance of plots using options such as colors, titles, labels, markers, legends, grids, fonts, and style sheets to improve the visual appeal, readability, and effective communication of data.

17Q) Analyze the Role of Matplotlib in the Python Ecosystem for Data Visualization

Ans:

Introduction

Matplotlib is one of the most popular and widely used **data visualization libraries** in Python. It helps users convert numerical data into visual forms such as **line graphs, bar charts, scatter plots, histograms, pie charts, box plots, and many other types of graphs**.

It is considered the **foundation of data visualization in Python** because many other visualization libraries, such as **Seaborn** and **Pandas plotting**, are built on top of Matplotlib.

Definition

Matplotlib is an open-source Python library used to create **high-quality graphs, charts, and plots** for data visualization. It helps users analyze, understand, and present data in a clear and meaningful way.

Role of Matplotlib in the Python Ecosystem

Matplotlib plays an important role in Python because it provides powerful tools for visualizing data.

Its major roles include:

1. Data Visualization

Matplotlib converts numerical data into visual graphs, making it easier to understand patterns and trends.

Example:

- Monthly sales graph
- Student marks graph
- Temperature changes over time

2. Data Analysis

Visual graphs help users analyze data quickly.

For example, a line graph can easily show whether sales are increasing or decreasing over time.

3. Scientific Research

Scientists and researchers use Matplotlib to display:

- Experimental results
- Statistical analysis
- Scientific observations

4. Machine Learning and Data Science

In Machine Learning, Matplotlib is used to:

- Explore datasets
- Visualize training data
- Compare model performance
- Display prediction results

5. Business Reporting

Companies use Matplotlib to create:

- Sales reports
- Profit analysis
- Employee performance charts
- Financial dashboards

6. Education

Students and teachers use Matplotlib to:

- Learn data visualization
- Create graphs for assignments
- Present project results

Advantages of Matplotlib

1. Easy to Learn

Matplotlib has simple syntax, making it suitable for beginners.

2. Wide Variety of Graphs

It supports many types of plots, including:

- Line Plot
- Bar Chart
- Scatter Plot
- Histogram
- Pie Chart
- Box Plot
- Area Plot

3. Highly Customizable

Almost every part of a graph can be customized.

Users can change:

- Colors
- Fonts
- Titles
- Labels
- Markers
- Line styles
- Figure size
- Grid
- Legends

4. Works with Other Python Libraries

Matplotlib works very well with:

- NumPy
- Pandas
- SciPy
- Scikit-learn
- Jupyter Notebook

This makes it an important part of the Python data science ecosystem.

5. Produces High-Quality Graphs

Graphs created using Matplotlib can be saved in formats such as:

- PNG
- JPG
- PDF
- SVG

These are suitable for reports and presentations.

6. Open Source

Matplotlib is free to use and supported by a large community.

Limitations of Matplotlib

Although Matplotlib is very powerful, it also has some limitations.

1. More Code Required

Creating complex graphs often requires many lines of code.

2. Basic Default Appearance

Default graphs look simple and may need customization to become attractive.

3. Less Interactive

Matplotlib mainly creates **static plots**.

For highly interactive visualizations, libraries such as **Plotly** are more suitable.

4. Slower for Very Large Datasets

Rendering extremely large datasets can be slower than some modern visualization tools.

Comparison with Other Visualization Tools

Feature	Matplotlib	Seaborn	Plotly
Ease of Use	Moderate	Easy	Easy
Built on Matplotlib	No	Yes	No
Attractive Default Style	Basic	Excellent	Excellent
Interactive Graphs	Limited	Limited	Yes
Customization	Very High	High	High
Best For	General-purpose plotting	Statistical visualization	Interactive dashboards

Scenarios Where Matplotlib Excels

Matplotlib performs especially well in the following situations:

1. Educational Purposes

Students learning Python can easily create graphs for assignments and projects.

2. Scientific Research

Researchers can generate precise graphs for experiments and publications.

3. Engineering Applications

Engineers use Matplotlib to visualize measurements, signals, and test results.

4. Business Reports

Companies create sales charts, profit analysis, and financial reports.

5. Data Analysis

Data analysts use Matplotlib to identify trends, patterns, and relationships.

6. Machine Learning

Machine learning engineers visualize:

- Training data
- Model accuracy
- Loss curves
- Prediction results

Example Program

```
import matplotlib.pyplot as plt


# Data

months = ["Jan", "Feb", "Mar", "Apr", "May"]

sales = [20, 35, 30, 45, 50]


# Plot

plt.plot(months, sales, marker='o', color='blue')


# Title and Labels

plt.title("Monthly Sales")
```

```
plt.xlabel("Months")

plt.ylabel("Sales")

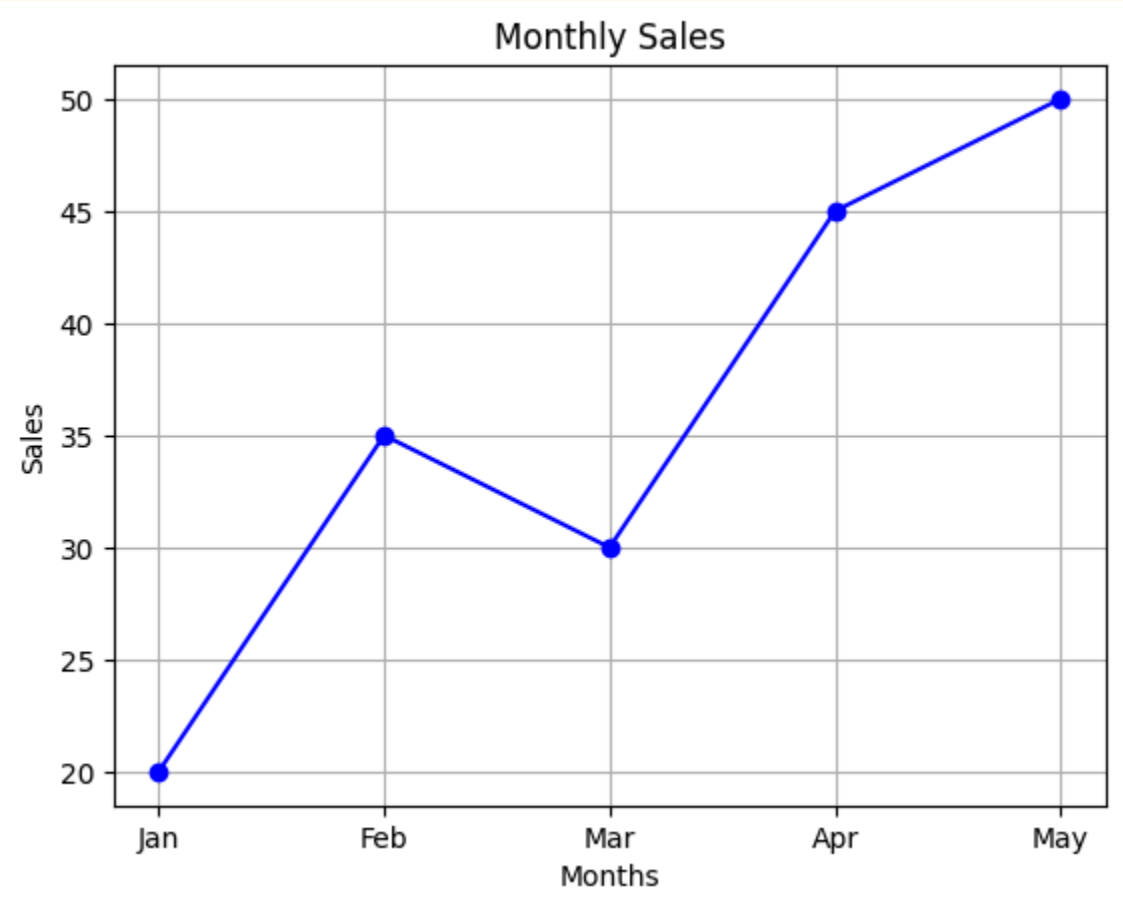

# Grid

plt.grid(True)


# Display

plt.show()
```

Output



Output Description

- The graph displays the monthly sales trend.
- The x-axis represents the months.
- The y-axis represents the sales values.
- A blue line connects all the data points.
- Circular markers highlight each value.
- Grid lines improve readability.

Why Matplotlib is Popular

Matplotlib is popular because:

- It is free and open source.
- It supports many graph types.
- It is highly customizable.
- It integrates well with NumPy and Pandas.
- It is widely used in Data Science, AI, and Machine Learning.
- It has strong community support and documentation.

Applications of Matplotlib

Matplotlib is widely used in:

- Data Science
- Machine Learning
- Artificial Intelligence
- Scientific Research
- Business Analytics
- Financial Analysis
- Weather Forecasting
- Healthcare
- Education
- Engineering

Important Functions

Function	Purpose

`plt.plot()`	Creates a line plot
`plt.bar()`	Creates a bar chart
`plt.scatter()`	Creates a scatter plot
`plt.hist()`	Creates a histogram
`plt.pie()`	Creates a pie chart
`plt.boxplot()`	Creates a box plot
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis
`plt.title()`	Adds a title
`plt.grid()`	Displays grid lines
`plt.legend()`	Displays a legend
`plt.show()`	Displays the graph

Viva Questions

1. What is Matplotlib?
Matplotlib is a Python library used for creating graphs and charts.

2. Why is Matplotlib important in Python?
It helps visualize data, making it easier to analyze and understand.

3. Name any four graphs created using Matplotlib.

- Line Plot
- Bar Chart
- Scatter Plot
- Histogram

4. What are the main advantages of Matplotlib?
Easy to use, highly customizable, supports many graph types, and integrates well with other Python libraries.

5. What is one limitation of Matplotlib?
It creates mostly static plots and may require more code for complex visualizations.

Conclusion

Matplotlib is one of the **core libraries for data visualization in Python**. It plays an essential role in **Data Science, Machine Learning, Scientific Research, Business Analytics, and Education** by converting numerical data into meaningful visual graphs. Its flexibility, extensive customization options, and compatibility with other Python libraries make it an excellent choice for creating professional and informative

visualizations. Although it has some limitations, such as limited interactivity and more coding for complex plots, its advantages make it one of the most widely used visualization tools in the Python ecosystem.

Exam Definition (2 Marks)

Matplotlib is an open-source Python library used for creating high-quality graphs and charts. It plays a key role in the Python ecosystem by helping users visualize, analyze, and communicate data effectively through various types of plots.

18Q) How to add Grid to a plot in matplotlib?

Ans:

Introduction

A **grid** is a set of horizontal and vertical lines drawn in the background of a graph. These lines help users read and compare the values on the graph more easily.

In Matplotlib, a grid can be added using the `plt.grid()` function. It improves the readability and appearance of plots, making them easier to understand.

Definition

A **grid** is a pattern of horizontal and vertical reference lines displayed in the background of a graph to make it easier to locate and compare data values.

Why Do We Use a Grid?

A grid is used to:

- Improve the readability of a graph.
- Help identify the exact values of data points.
- Make comparisons between data points easier.
- Give the graph a neat and professional appearance.
- Assist in analyzing trends and patterns.

Function Used

plt.grid()

Syntax

plt.grid(True)

or

plt.grid(False)

Parameters

- **True** → Displays the grid.
- **False** → Hides the grid.

Steps to Add a Grid

Step 1: Import the Matplotlib Library

import matplotlib.pyplot as plt

Imports the Matplotlib plotting module.

Step 2: Create Sample Data

x = [1, 2, 3, 4, 5]

y = [10, 20, 15, 25, 30]

Creates values for the x-axis and y-axis.

Step 3: Draw the Plot

plt.plot(x, y)

Creates a line graph.

Step 4: Add the Grid

```
plt.grid(True)
```

Displays horizontal and vertical grid lines.

Step 5: Add Title and Labels

```
plt.title("Sales Report")
plt.xlabel("Days")
plt.ylabel("Sales")
```

Adds the title and axis labels.

Step 6: Display the Graph

```
plt.show()
```

Displays the graph.

Complete Program

```
import matplotlib.pyplot as plt

# Sample Data
```

```
x = [1, 2, 3, 4, 5]

y = [10, 20, 15, 25, 30]


# Create Line Plot

plt.plot(x, y, marker='o', color='blue')


# Add Grid

plt.grid(True)


# Add Title and Labels

plt.title("Sales Report")

plt.xlabel("Days")

plt.ylabel("Sales")


# Display Plot

plt.show()
```

Explanation of the Program

Import Library

import matplotlib.pyplot as plt

Imports the plotting library.

Create Data

x = [1, 2, 3, 4, 5]

y = [10, 20, 15, 25, 30]

Creates sample data for plotting.

Create Plot

plt.plot(x, y, marker='o', color='blue')

Draws a blue line graph with circular markers.

Add Grid

plt.grid(True)

Displays horizontal and vertical grid lines in the background.

Add Title

```
plt.title("Sales Report")
```

Displays the graph title.

Add Axis Labels

```
plt.xlabel("Days")
plt.ylabel("Sales")
```

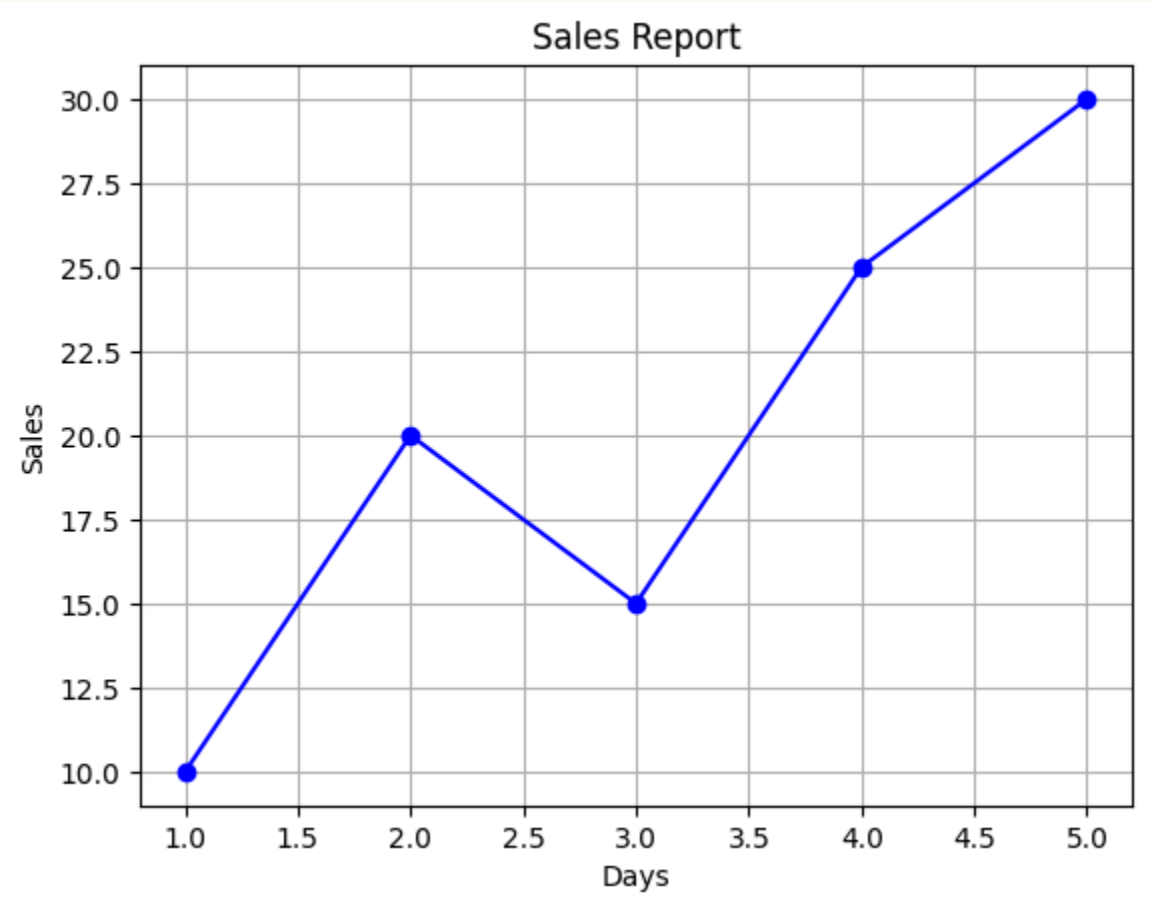
Labels the x-axis and y-axis.

Display the Graph

```
plt.show()
```

Displays the final graph.

Output



Output Description

The graph displays:

- A **blue line** connecting the data points.
- **Circular markers** at each point.
- The **title** "Sales Report".
- **Days** on the x-axis.
- **Sales** on the y-axis.
- **Horizontal and vertical grid lines** in the background, making it easier to read and compare values.

Grid Customization

Matplotlib allows us to customize the grid.

1. Change Grid Color

plt.grid(color='red')

Changes the grid color to red.

2. Change Grid Line Style

plt.grid(linestyle='--')

Displays dashed grid lines.

3. Change Grid Line Width

plt.grid(linewidth=2)

Makes the grid lines thicker.

4. Apply Multiple Customizations

plt.grid(color='gray', linestyle='--', linewidth=1)

Displays gray dashed grid lines with a width of 1.

Example of Customized Grid

```
import matplotlib.pyplot as plt


x = [1,2,3,4,5]

y = [5,8,6,10,12]


plt.plot(x, y, marker='o')


plt.grid(color='gray', linestyle='--', linewidth=1)

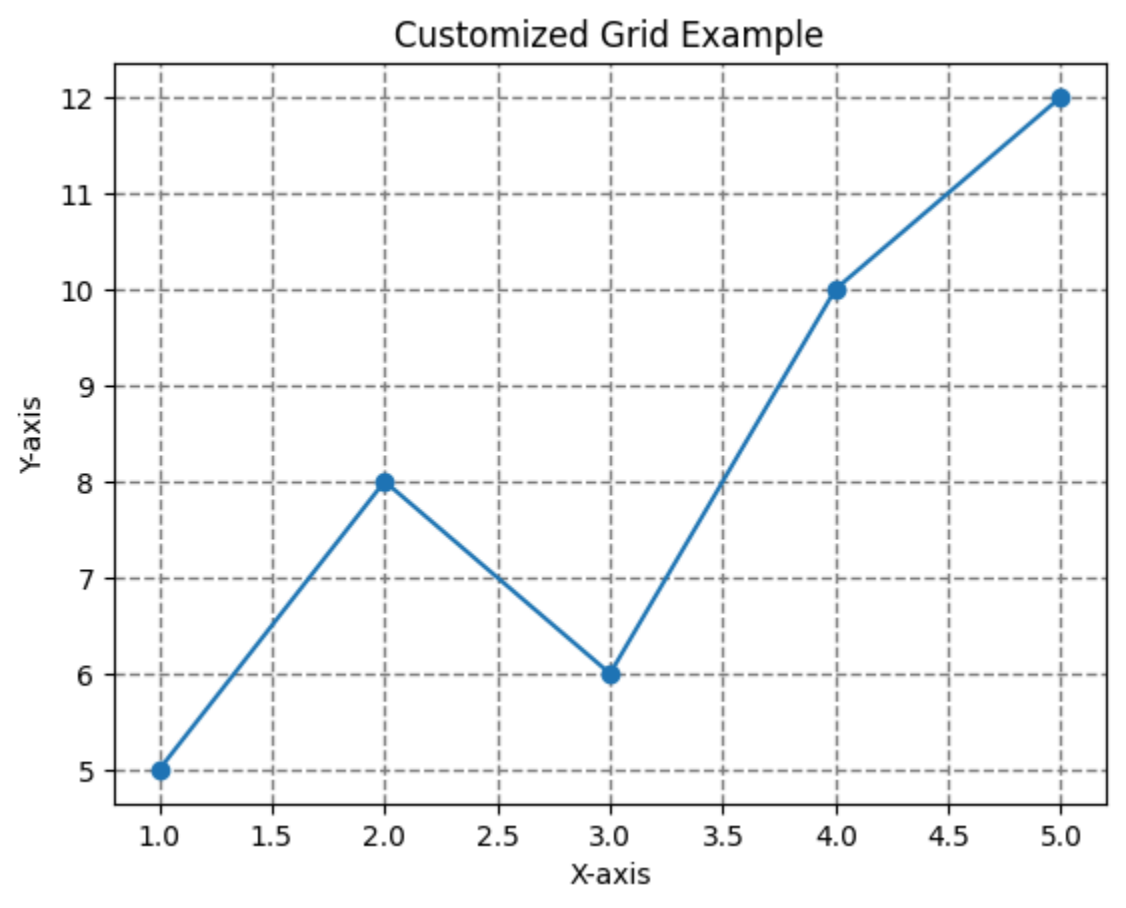

plt.title("Customized Grid Example")

plt.xlabel("X-axis")

plt.ylabel("Y-axis")


plt.show()
```

Output



Output Description

The graph displays:

- A blue line with circular markers.
- **Gray dashed grid lines.**
- Clearly labeled axes.
- A title at the top.
- Improved readability due to the customized grid.

Advantages of Using a Grid

- Improves readability.
- Helps identify exact values.
- Makes comparison of data easier.
- Enhances the appearance of the graph.
- Useful for presentations and reports.

Applications

Grids are commonly used in:

- Business reports
- Scientific research
- Engineering graphs
- Weather analysis
- Stock market analysis
- Machine Learning visualizations
- Educational projects

Important Functions

Function	Purpose
`plt.grid(True)`	Displays the grid

`plt.grid(False)`	Hides the grid
`color`	Changes the grid color
`linestyle`	Changes the grid line style
`linewidth`	Changes the thickness of the grid lines
`plt.plot()`	Creates a line graph
`plt.show()`	Displays the graph

Viva Questions

1. What is a grid in Matplotlib?
A grid is a set of horizontal and vertical reference lines displayed in the background of a graph.

2. Which function is used to add a grid?
`plt.grid()`

3. What does `plt.grid(True)` do?
It displays the grid lines on the graph.

4. Can we change the color of grid lines?
Yes, by using the `color` parameter.

5. Name any two benefits of using a grid.

- Improves readability.
- Makes data comparison easier.

Conclusion

Adding a grid to a plot in Matplotlib is a simple but powerful way to improve the readability and interpretation of graphs. The `plt.grid()` function allows users to display and customize grid lines by changing their color, style, and width. Grids make graphs easier to analyze, compare values, and present data professionally, making them an essential feature in data visualization.

Exam Definition (2 Marks)

A grid in Matplotlib is a set of horizontal and vertical reference lines added to a graph using the `plt.grid()` function. It improves the readability and interpretation of plots by making it easier to locate and compare data values.

19Q) How to add Title to a plot in matplotlib?

Ans:

Introduction

In Matplotlib, images can be displayed using the `imshow()` function. To make the image more informative, we can add a **title** above it using the `plt.title()` function.

A title helps users understand what the image represents without needing additional explanation. It improves the readability, presentation, and interpretation of the displayed image.

Definition

A **title** is a descriptive heading displayed at the top of an image or graph. In Matplotlib, a title is added using the `plt.title()` function.

Why Do We Add a Title?

A title is added to:

- Describe what the image represents.
- Improve readability.
- Help viewers understand the image quickly.
- Make the figure attractive.
- Improve presentation in reports and projects.

Function Used

plt.title("Title Name")

Syntax

plt.imshow(image)

plt.title("Image Title")

plt.show()

Steps to Add a Title to an Image

Step 1: Import the Matplotlib Library

import matplotlib.pyplot as plt

Imports the Matplotlib plotting module.

Step 2: Read the Image

image = plt.imread("flower.jpg")

Reads the image from the computer.

Step 3: Display the Image

plt.imshow(image)

Displays the image inside the figure.

Step 4: Add the Title

plt.title("Flower Image")

Displays the title at the top of the image.

Step 5: Display the Figure

plt.show()

Displays the final output.

Complete Program

import matplotlib.pyplot as plt

Read Image

image = plt.imread("flower.jpg")

```
# Display Image

plt.imshow(image)


# Add Title

plt.title("Flower Image")


# Display

plt.show()
```

Explanation of the Program

Import Library

```
import matplotlib.pyplot as plt
```

Imports the Matplotlib library.

Read the Image

```
image = plt.imread("flower.jpg")
```

Reads the image file named **flower.jpg**.

Display the Image

plt.imshow(image)

Displays the image inside the plotting window.

Add the Title

plt.title("Flower Image")

Adds the title **"Flower Image"** above the displayed image.

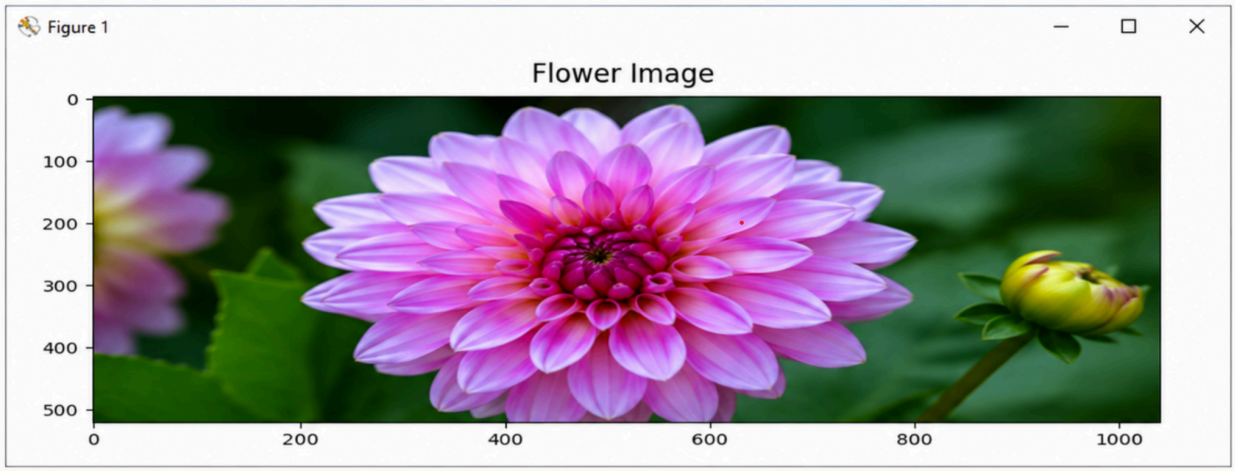
Display the Figure

plt.show()

Displays the image along with its title.

Output

Output 1: Basic Title Added to an Image



Output Description

The output displays:

- A **flower image** inside the Matplotlib figure.
- The title **"Flower Image"** appears at the top.
- The image is displayed clearly with its descriptive heading.
- The title helps identify the image quickly.

Customizing the Title

Matplotlib allows us to customize the title in several ways.

1. Change the Font Size

plt.title("Flower Image", fontsize=16)

Makes the title larger.

2. Change the Title Color

```
plt.title("Flower Image", color="blue")
```

Displays the title in blue.

3. Make the Title Bold

```
plt.title("Flower Image", fontweight="bold")
```

Makes the title bold.

4. Change the Font Style

```
plt.title("Flower Image", fontstyle="italic")
```

Displays the title in italic style.

5. Combine Multiple Customizations

```
plt.title(
    "Flower Image",
    fontsize=16,
    color="green",
```

```
    fontweight="bold"
)
```

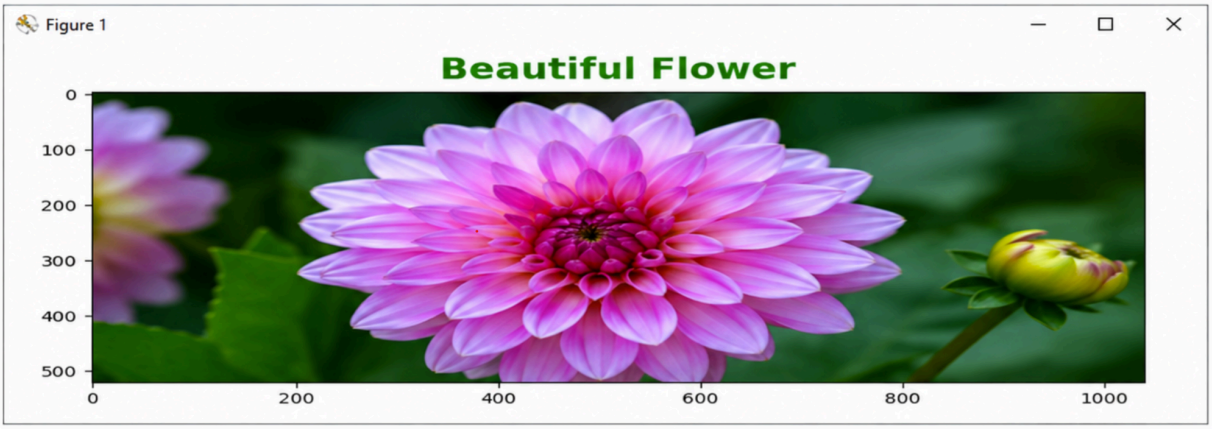
Displays a large, bold, green title.

Example with Customized Title

```
import matplotlib.pyplot as plt
image = plt.imread("flower.jpg")
plt.imshow(image)
plt.title(
    "Beautiful Flower",
    fontsize=18,
    color="green",
    fontweight="bold"
)
plt.show()
```

Output

Output 2: Customized Title



Output Description

The graph displays:

- A flower image.
- A **large bold green title** at the top.
- The customized title makes the image more attractive and easier to identify.

Advantages of Adding a Title

- Clearly describes the image.
- Improves readability.
- Makes the figure more attractive.
- Helps viewers understand the content quickly.
- Useful for reports, presentations, and projects.
- Gives a professional appearance.

Applications

Adding titles to images is useful in:

- Educational presentations
- Scientific research
- Medical image analysis
- Computer vision
- Machine learning
- Business reports
- Project documentation

Important Functions

Function	Purpose
`plt.imread()`	Reads an image file
`plt.imshow()`	Displays an image
`plt.title()`	Adds a title to the image

`fontsize`	Changes title size
`color`	Changes title color
`fontweight`	Makes the title bold
`fontstyle`	Changes the font style
`plt.show()`	Displays the figure

Viva Questions

1. Which function is used to add a title in Matplotlib?
`plt.title()`

2. Which function is used to display an image?
`plt.imshow()`

3. Which function is used to read an image?
`plt.imread()`

4. Can we change the color of the title?
Yes, by using the `color` parameter.

5. Name any three title customization options.

- `fontsize`
- `color`
- `fontweight`

Conclusion

Adding a title to an image in Matplotlib is a simple but important feature that improves the readability and presentation of the figure. Using the **`plt.title()`** function, we can add meaningful headings and customize them with different font sizes, colors, and styles. A well-chosen title helps viewers quickly understand the purpose of the image, making it especially useful in reports, presentations, educational materials, and scientific research.

Exam Definition (2 Marks)

A title is a descriptive heading displayed above an image or graph in Matplotlib. It is added using the `plt.title()` function and helps improve the readability, identification, and presentation of the displayed image.

20Q) Illustrate Any Three 2D Plots in Python with Their Significance

Ans:

Introduction

A **2D (Two-Dimensional) plot** is a graph that represents data using two axes:

- **X-axis (Horizontal axis)**
- **Y-axis (Vertical axis)**

Matplotlib provides many types of 2D plots to visualize and analyze data effectively.

Some commonly used 2D plots are:

- Line Plot
- Bar Graph
- Scatter Plot
- Histogram
- Pie Chart
- Box Plot

Below are **three important 2D plots** explained with examples.

1. Line Plot

Definition

A **Line Plot** is a graph in which data points are connected using straight lines. It is mainly used to show **changes or trends over time**.

Significance

- Shows trends over time.
- Easy to compare data.
- Useful for continuous data.
- Helps identify increasing or decreasing patterns.

Example Program

```
import matplotlib.pyplot as plt


x = [1,2,3,4,5]

y = [10,15,12,18,20]


plt.plot(x, y, marker='o')

plt.title("Line Plot")

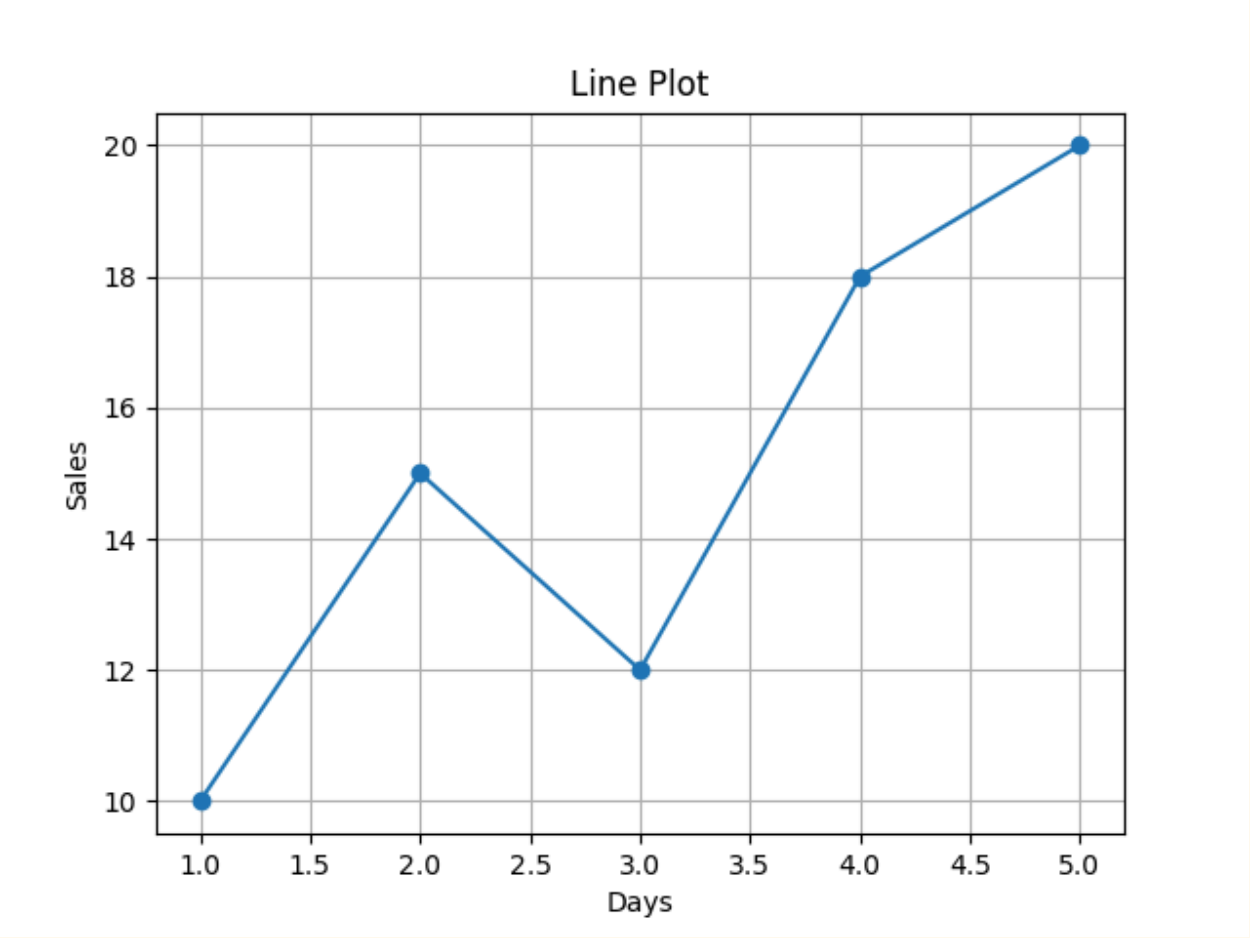
plt.xlabel("Days")

plt.ylabel("Sales")

plt.grid(True)
```

```
plt.show()
```

Output



Output Description

- A blue line connects all the data points.
- Circular markers indicate each value.
- The graph shows the sales trend over five days.

2. Bar Graph

Definition

A **Bar Graph** represents data using rectangular bars. The height of each bar is proportional to its value.

Significance

- Compares different categories easily.
- Clearly shows differences between values.
- Easy to understand.
- Suitable for categorical data.

Example Program

```
import matplotlib.pyplot as plt


subjects = ["Math", "Science", "English", "Computer"]

marks = [85, 90, 78, 95]


plt.bar(subjects, marks, color="skyblue")

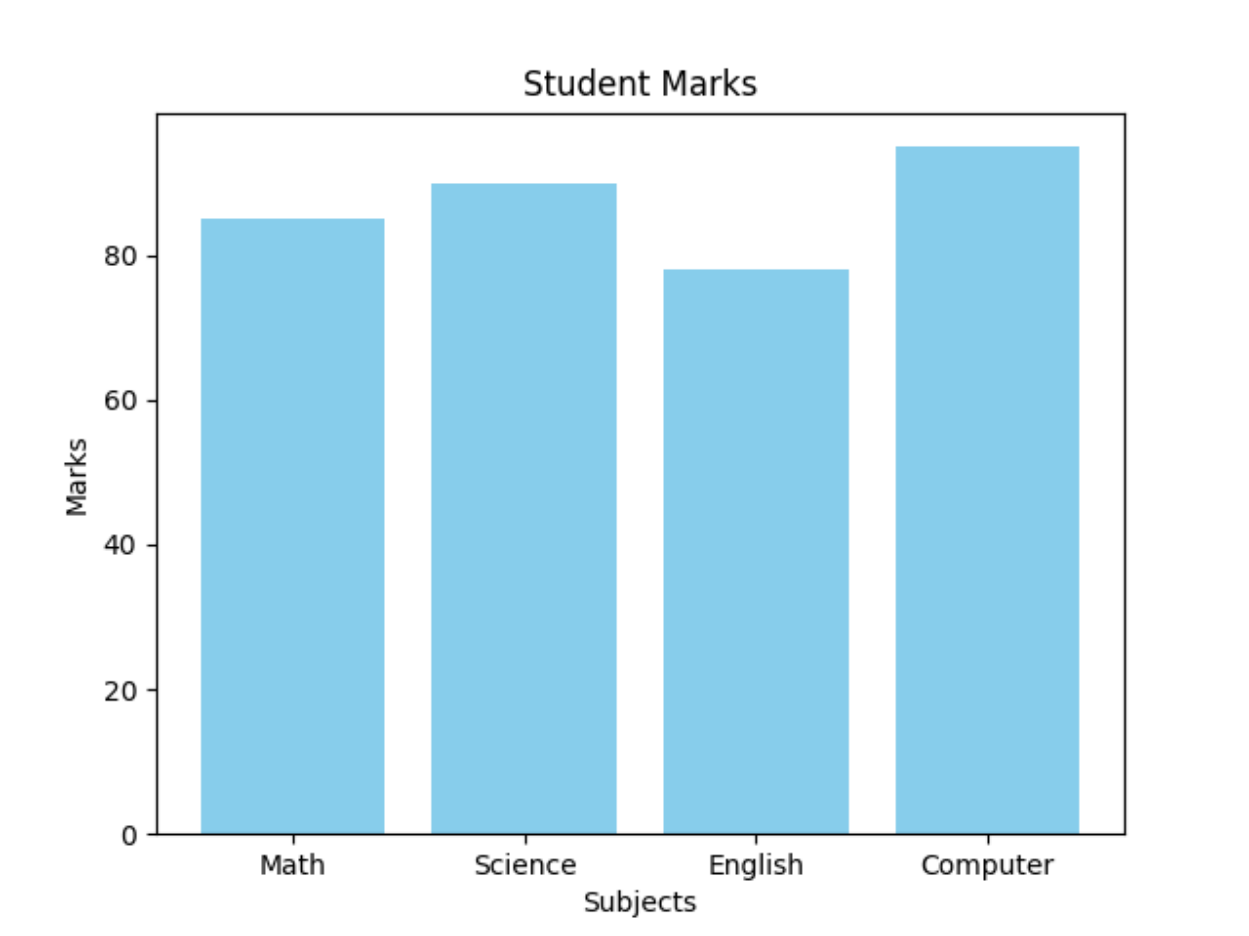
plt.title("Student Marks")

plt.xlabel("Subjects")
```

```
plt.ylabel("Marks")

plt.show()
```

Output



Output Description

- Four vertical bars represent the marks obtained in each subject.
- The tallest bar represents **Computer (95 marks)**.
- The shortest bar represents **English (78 marks)**.

3. Scatter Plot

Definition

A **Scatter Plot** displays data using individual points. It is mainly used to show the relationship between two numerical variables.

Significance

- Shows the relationship between variables.
- Helps identify patterns and trends.
- Detects outliers.
- Useful in correlation analysis.

Example Program

```
import matplotlib.pyplot as plt


hours = [1,2,3,4,5,6]

marks = [35,45,50,65,75,90]


plt.scatter(hours, marks, color="red")

plt.title("Study Hours vs Marks")

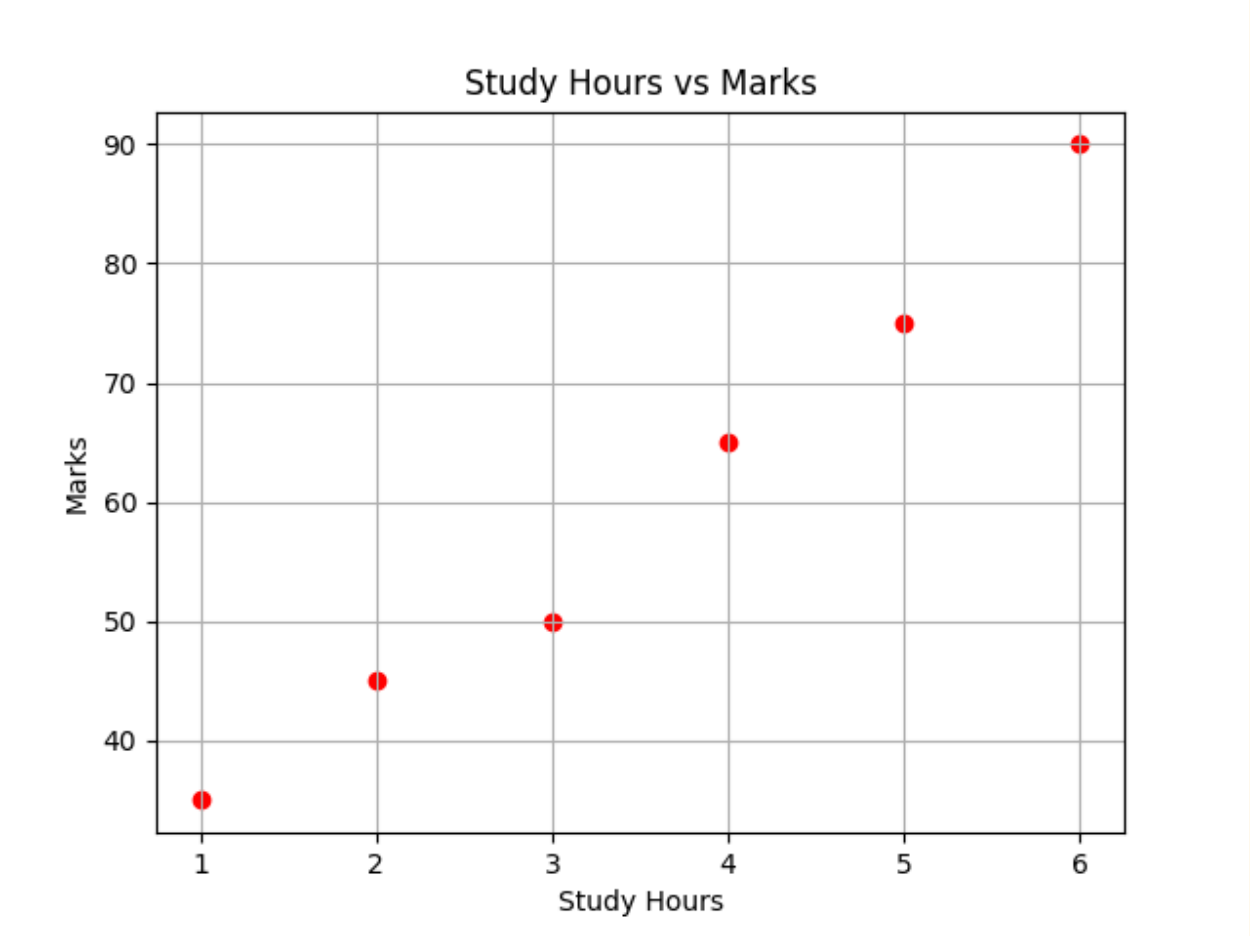
plt.xlabel("Study Hours")
```

```
plt.ylabel("Marks")

plt.grid(True)

plt.show()
```

Output



Output Description

- Red dots represent each student's study hours and marks.
- The graph shows that as study hours increase, marks also increase.

- It indicates a **positive correlation**.

Comparison of Three 2D Plots

Plot Type	Purpose	Best Used For
Line Plot	Shows trends over time	Sales, temperature, stock prices
Bar Graph	Compares categories	Student marks, product sales
Scatter Plot	Shows relationship between two variables	Study hours vs marks, height vs weight

Advantages of 2D Plots

- Easy to understand.
- Represents data visually.
- Helps identify trends and patterns.
- Makes comparison easier.
- Improves data presentation.
- Useful for reports and research.

Applications

2D plots are widely used in:

- Business reports
- Education
- Scientific research
- Machine Learning
- Data Science
- Healthcare
- Finance
- Weather forecasting

Conclusion

2D plots are one of the most important visualization tools in Python. **Line plots** are best for showing trends, **bar graphs** are ideal for comparing categories, and **scatter plots** are useful for identifying relationships between variables. These plots help users analyze data quickly and communicate information effectively.

Exam Definition (2 Marks)

A 2D plot is a graph that represents data using two axes (X-axis and Y-axis). Common 2D plots in Python include line plots, bar graphs,

and scatter plots, which are used to visualize trends, comparisons, and relationships between data.

21Q) What are the main differences between pie chart and bar chart?

Ans:

Pie Chart	Bar Chart
A **pie chart** is a circular graph divided into slices.	A **bar chart** is a graph that uses rectangular bars.
It shows the **proportion or percentage** of each category in the whole.	It shows the **comparison** between different categories.
The whole pie represents **100%** of the data.	There is no concept of the whole being 100%; each bar represents an independent value.
Each slice represents a category.	Each bar represents a category.
Best suited for showing **part-to-whole relationships**.	Best suited for comparing **individual values** across categories.

Difficult to compare values when there are many categories.	Easy to compare values even with many categories.
Usually used for **percentage data**.	Can be used for **counts, quantities, frequencies, or percentages**.
Labels are usually displayed on the slices.	Categories are displayed along the x-axis or y-axis.
Created using `plt.pie()` in Matplotlib.	Created using `plt.bar()` in Matplotlib.
Example: Market share of smartphone brands.	Example: Marks scored by students in different subjects.

Example of a Pie Chart

Purpose: Shows the percentage contribution of each category to the whole.

Example of a Bar Chart

Purpose: Compares the values of different categories.

Key Points to Remember

Pie Chart

- Circular graph.
- Shows **parts of a whole**.
- Total always represents **100%**.
- Best for percentage distribution.

Bar Chart

- Uses rectangular bars.
- Compares **different categories**.
- No requirement for values to total 100%.
- Best for comparing quantities.

Exam Definition (2 Marks)

- **Pie Chart:** A pie chart is a circular graph divided into slices that represent the percentage or proportion of each category in the whole.
- **Bar Chart:** A bar chart is a graph that uses rectangular bars to compare values among different categories.

22Q) How to add an image to a plot in matplotlib?

Ans:

Introduction

Sometimes, instead of displaying only graphs, we may want to **display an image** or **add an image to a Matplotlib figure**. Matplotlib provides functions to read and display images, making it useful in **image processing, computer vision, machine learning, medical imaging, and scientific visualization**.

The two main functions used are:

- **`plt.imread()`** → Reads (loads) an image.
- **`plt.imshow()`** → Displays the image.

Definition

Adding an image to a plot means displaying an image inside a Matplotlib figure using the `imshow()` function after reading it with `imread()`.

Why Do We Add Images?

Images are added to plots to:

- Display photographs.
- Show satellite images.
- Display medical scans.

- Visualize machine learning datasets.
- Show maps and diagrams.
- Improve presentations and reports.

Functions Used

1. Read an Image

plt.imread("image.jpg")

Reads the image from the computer.

2. Display an Image

plt.imshow(image)

Displays the image in the figure.

Syntax

import matplotlib.pyplot as plt

image = plt.imread("image.jpg")

plt.imshow(image)

```
plt.show()
```

Steps to Add an Image

Step 1: Import the Library

```
import matplotlib.pyplot as plt
```

Imports Matplotlib.

Step 2: Read the Image

```
image = plt.imread("flower.jpg")
```

Loads the image named **flower.jpg**.

Step 3: Display the Image

```
plt.imshow(image)
```

Displays the image.

Step 4: Display the Figure

```
plt.show()
```

Shows the image on the screen.

Complete Program

```
import matplotlib.pyplot as plt


# Read image

image = plt.imread("flower.jpg")


# Display image

plt.imshow(image)


# Show figure

plt.show()
```

Explanation of the Program

Import Library

```
import matplotlib.pyplot as plt
```

Imports Matplotlib.

Read Image

image = plt.imread("flower.jpg")

Reads the image file and stores it in the variable `image`.

Display Image

plt.imshow(image)

Displays the image inside the Matplotlib figure.

Show Figure

plt.show()

Displays the final output window.

Output

Output Description

The output shows:

- A flower image displayed inside a Matplotlib figure.
- The image is shown without any graph.
- The image occupies the plotting area.

Adding a Title to the Image

We can add a title using `plt.title()`.

Example

```
import matplotlib.pyplot as plt


image = plt.imread("flower.jpg")


plt.imshow(image)


plt.title("Flower Image")


plt.show()
```

Output

Output Description

- The flower image is displayed.
- The title **"Flower Image"** appears at the top.

Removing the Axis

Sometimes, we don't want the x-axis and y-axis around the image.

plt.axis("off")

Example

```
import matplotlib.pyplot as plt

image = plt.imread("flower.jpg")

plt.imshow(image)

plt.axis("off")

plt.title("Flower Image")

plt.show()
```

Output

Output Description

- The image is displayed without x-axis and y-axis.
- Only the image and title are visible.
- The figure looks cleaner and more attractive.

Applications

Adding images in Matplotlib is useful in:

- Computer Vision

- Medical Image Analysis
- Satellite Image Processing
- Machine Learning
- Scientific Research
- Image Editing
- Educational Presentations

Advantages

- Displays images easily.
- Supports many image formats (JPG, PNG, BMP, etc.).
- Can combine images with graphs.
- Supports titles and labels.
- Useful for image processing applications.

Limitations

- Basic image editing features.
- Not designed for advanced photo editing.
- Large images may require more memory.

Important Functions

Function	Purpose
`plt.imread()`	Reads an image

`plt.imshow()`	Displays an image
`plt.title()`	Adds a title
`plt.axis("off")`	Hides the axes
`plt.show()`	Displays the figure

Viva Questions

1. Which function is used to read an image?
`plt.imread()`

2. Which function displays an image?
`plt.imshow()`

3. Which function hides the axes?
`plt.axis("off")`

4. Can we add a title to an image?
Yes, using `plt.title()`.

5. Name any two applications of displaying images in Matplotlib.

- Computer Vision
- Medical Image Analysis

Conclusion

Matplotlib provides a simple and effective way to display images using the `plt.imread()` and `plt.imshow()` functions. Images can also be enhanced by adding titles, hiding axes, and combining them with plots. This feature is widely used in computer vision, machine learning, scientific research, and educational applications.

Exam Definition (2 Marks)

An image can be added to a Matplotlib plot by reading it with `plt.imread()` and displaying it using `plt.imshow()`. Additional functions such as `plt.title()` and `plt.axis("off")` can be used to improve the presentation of the displayed image.

23Q) How to set a Single Main Title for all the subplots in matplotlib?

Ans:

Introduction

When we create **multiple subplots** in Matplotlib, each subplot can have its own title using `plt.title()`. However, sometimes we want **one common title** that describes the entire figure instead of each individual graph.

Matplotlib provides the **`plt.suptitle()`** function to add a **single main title** for all subplots.

Definition

A **main title** (also called a **super title**) is a single heading displayed at the top of a figure that applies to all the subplots. In Matplotlib, it is added using the **`plt.suptitle()`** function.

Why Do We Use a Main Title?

A single main title is used to:

- Describe the entire figure.
- Avoid repeating the same title for every subplot.
- Make the figure neat and professional.
- Improve readability.
- Help viewers understand the purpose of all graphs together.

Function Used

plt.suptitle("Main Title")

Syntax

plt.suptitle("Main Title")

Steps to Set a Single Main Title

Step 1: Import the Matplotlib Library

import matplotlib.pyplot as plt

Imports the Matplotlib plotting module.

Step 2: Create Sample Data

x = [1,2,3,4]

y1 = [2,4,6,8]

y2 = [1,3,5,7]

Creates data for two graphs.

Step 3: Create the First Subplot

plt.subplot(1,2,1)

plt.plot(x,y1)

plt.title("Graph 1")

Creates the first subplot and adds its own title.

Step 4: Create the Second Subplot

plt.subplot(1,2,2)

plt.plot(x,y2)

plt.title("Graph 2")

Creates the second subplot.

Step 5: Add the Main Title

plt.suptitle("Comparison of Two Graphs")

Adds one title for the entire figure.

Step 6: Display the Figure

plt.show()

Displays both subplots with the common title.

Complete Program

```
import matplotlib.pyplot as plt


# Data

x = [1,2,3,4]

y1 = [2,4,6,8]

y2 = [1,3,5,7]


# First subplot

plt.subplot(1,2,1)

plt.plot(x,y1)

plt.title("Graph 1")


# Second subplot

plt.subplot(1,2,2)

plt.plot(x,y2)

plt.title("Graph 2")
```

```
# Main Title
plt.suptitle("Comparison of Two Graphs")


# Display
plt.show()
```

Explanation of the Program

Import Library

```
import matplotlib.pyplot as plt
```

Imports Matplotlib.

Create Data

```
x = [1,2,3,4]
```

Stores x-axis values.

```
y1 = [2,4,6,8]
```

Stores data for the first graph.

y2 = [1,3,5,7]

Stores data for the second graph.

First Subplot

plt.subplot(1,2,1)

Creates the left subplot.

Draw First Graph

plt.plot(x,y1)

Draws the first line graph.

Add First Subplot Title

plt.title("Graph 1")

Adds the title for the first graph.

Second Subplot

plt.subplot(1,2,2)

Creates the right subplot.

Draw Second Graph

plt.plot(x,y2)

Draws the second line graph.

Add Second Subplot Title

plt.title("Graph 2")

Adds the title for the second graph.

Add Main Title

plt.suptitle("Comparison of Two Graphs")

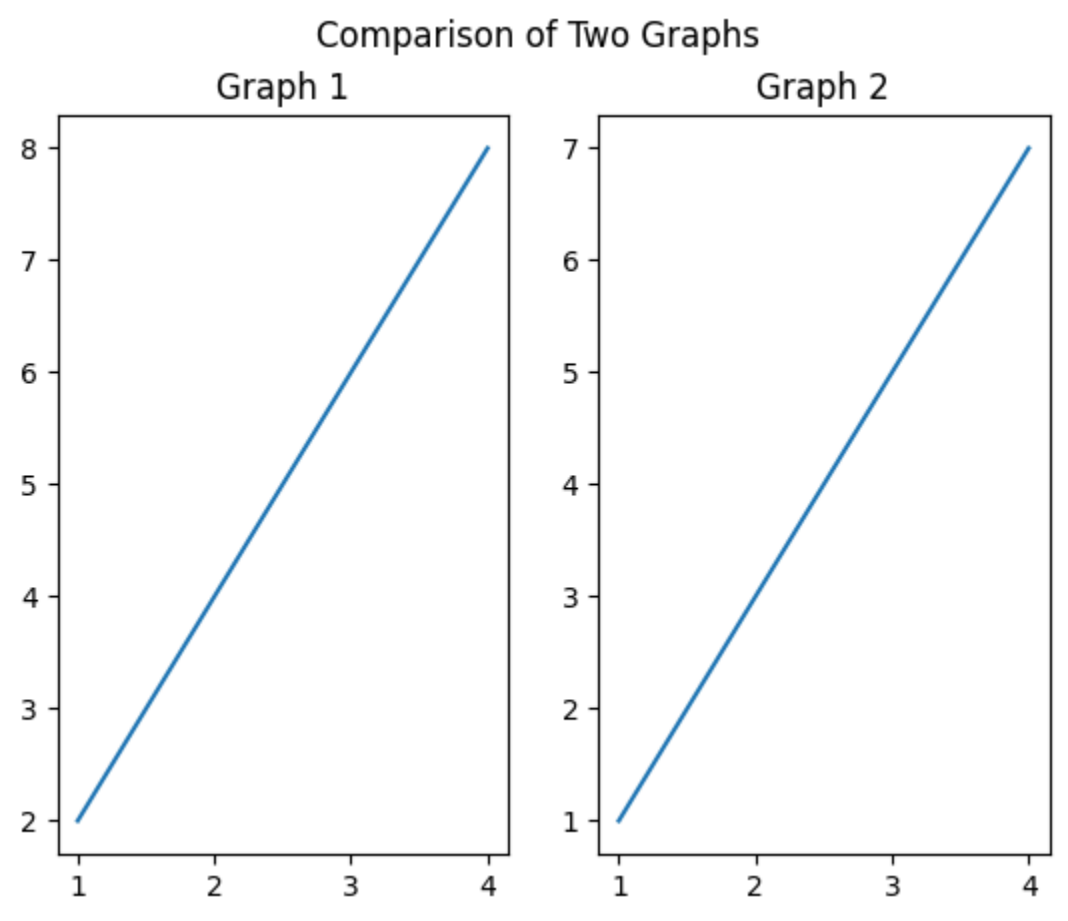
Adds one common title above both graphs.

Display the Figure

plt.show()

Displays the complete figure.

Output



Output Description

The output displays:

- A **single figure** containing **two line graphs**.
- The **left subplot** is titled **"Graph 1"**.
- The **right subplot** is titled **"Graph 2"**.
- A **main title "Comparison of Two Graphs"** appears at the top of the figure.

- The main title applies to **both subplots**.

Customizing the Main Title

We can change the appearance of the main title.

Change Font Size

```
plt.suptitle("Comparison of Two Graphs", fontsize=16)
```

Change Title Color

```
plt.suptitle("Comparison of Two Graphs", color="blue")
```

Make the Title Bold

```
plt.suptitle("Comparison of Two Graphs", fontweight="bold")
```

Customized Example

```
plt.suptitle(
    "Comparison of Two Graphs",
    fontsize=18,
    color="darkblue",
    fontweight="bold"
)
```

Output

Output Description

The graph displays:

- Two subplots side by side.
- Individual titles for each subplot.
- A **large, bold, dark blue main title** at the top.
- The customized title makes the figure more attractive and professional.

Advantages of Using a Main Title

- Describes the entire figure.
- Makes the graph more organized.
- Improves readability.
- Avoids repeating the same title.
- Gives a professional appearance.
- Useful for reports and presentations.

Applications

Main titles are commonly used in:

- Business reports
- Scientific research
- Data analysis
- Educational presentations
- Machine Learning

- Dashboards
- Project reports

Important Functions

Function	Purpose
`plt.subplot()`	Creates subplots
`plt.plot()`	Draws a line graph
`plt.title()`	Adds a title to an individual subplot
`plt.suptitle()`	Adds a single main title for all subplots
`plt.show()`	Displays the figure

Viva Questions

1. Which function is used to add a common title to all subplots?
`plt.suptitle()`

2. What is the difference between `plt.title()` and `plt.suptitle()`?

- `plt.title()` adds a title to an individual subplot.
- `plt.suptitle()` adds one main title to the entire figure.

3. Can each subplot have its own title?
Yes, using `plt.title()`.

4. Can the main title be customized?
Yes, by changing its font size, color, and font weight.

5. Why is a main title useful?
It describes the overall purpose of all subplots and improves readability.

Conclusion

A **single main title** helps describe the overall purpose of multiple subplots in a figure. In Matplotlib, it is added using the **`plt.suptitle()`** function. It improves the presentation, readability, and organization of graphs, making it especially useful in reports, research papers, dashboards, and presentations.

Exam Definition (2 Marks)

A single main title for all subplots in Matplotlib is called a super title and is added using the `plt.suptitle()` function. It provides a common heading for the entire figure, making multiple graphs easier to understand and present.

24Q) Give short note on the following:

1. Customizing ticks in matplotlib.
2. Customizing plot legends.

Ans:

1. Customizing Ticks in Matplotlib (Short Note)

Definition

Ticks are the small marks and labels shown along the **x-axis and y-axis** of a graph. They indicate the scale and values of the axes. Matplotlib allows us to customize these ticks to improve the readability and appearance of the plot.

Common Functions

plt.xticks()

plt.yticks()

Customization Options

- Change the tick positions.
- Change the tick labels.
- Change the font size.

- Change the font color.
- Rotate tick labels.

Example

```
import matplotlib.pyplot as plt


x = [1,2,3,4]

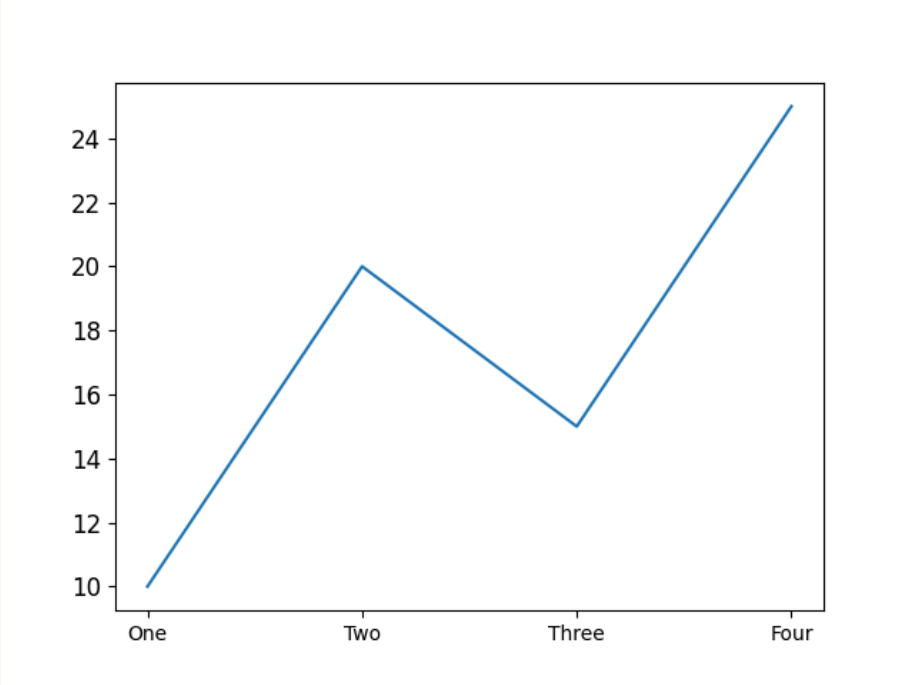
y = [10,20,15,25]


plt.plot(x,y)


plt.xticks([1,2,3,4], ['One','Two','Three','Four'])

plt.yticks(fontsize=12)


plt.show()
```



Advantages

- Improves readability.
- Makes the graph more attractive.
- Helps display meaningful labels.
- Makes axis values easier to understand.

2. Customizing Plot Legends (Short Note)

Definition

A **legend** is a box that identifies the meaning of different lines, colors, or markers in a graph. It helps distinguish multiple datasets displayed in the same plot.

Matplotlib allows legends to be customized using the **`plt.legend()`** function.

Function Used

plt.legend()

Customization Options

- Change the legend location.
- Change the font size.
- Change the title.
- Change the background color.
- Add a border around the legend.

Example

import matplotlib.pyplot as plt

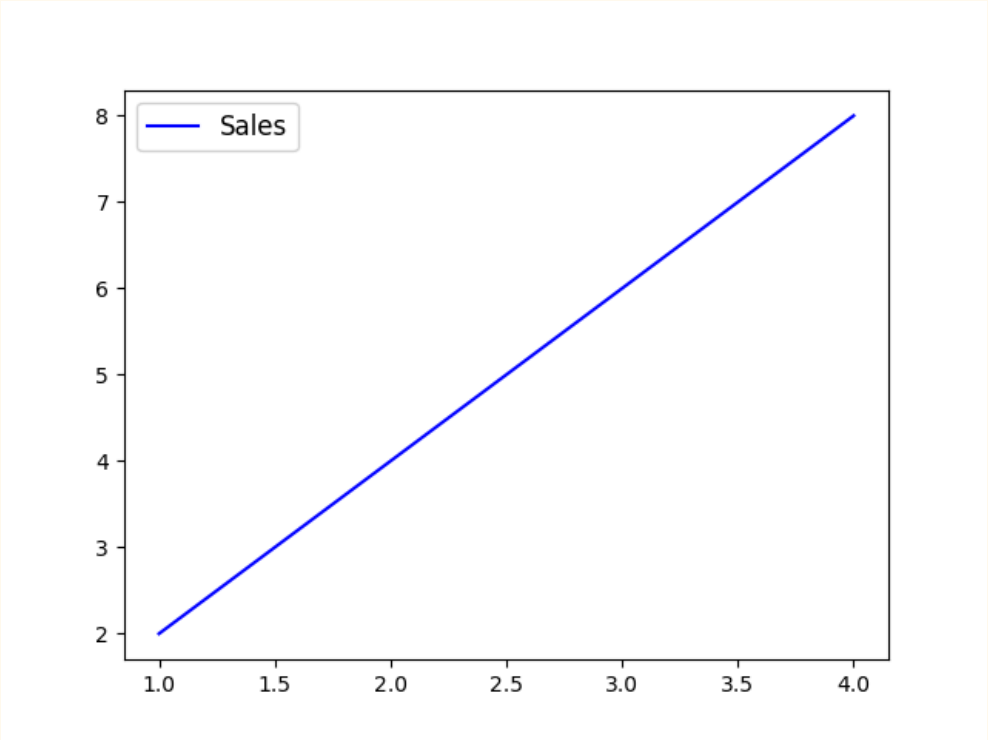
x = [1,2,3,4]

y = [2,4,6,8]

plt.plot(x, y, label="Sales", color="blue")

```
plt.legend(loc="upper left", fontsize=12)


plt.show()
```



Advantages

- Identifies different datasets.
- Improves readability.
- Makes graphs easier to understand.
- Gives a professional appearance.

Exam Definitions (2 Marks)

Customizing Ticks

Customizing ticks means modifying the position, labels, size, color, or orientation of the tick marks on the x-axis and y-axis using functions like `plt.xticks()` and `plt.yticks()`.

Customizing Plot Legends

Customizing plot legends means changing the appearance or position of the legend using the `plt.legend()` function to make the graph clearer and more informative.

25Q) How to Plot Two Histograms Together in Matplotlib?

Ans:

Introduction

A **histogram** is a graph used to show the **distribution (frequency)** of numerical data. Sometimes, we need to compare the distributions of **two different datasets** in the same figure. This can be done by plotting **two histograms together** using the `plt.hist()` function.

Displaying two histograms together helps us compare how two datasets are distributed, identify similarities or differences, and analyze overlapping values.

Definition

Plotting two histograms together means displaying the frequency distributions of two different datasets on the same graph using the `plt.hist()` function with different colors and labels.

Why Do We Plot Two Histograms Together?

We use two histograms together to:

- Compare two datasets.
- Analyze differences in distribution.
- Identify overlapping values.
- Detect patterns and variations.
- Make comparisons easier.

Function Used

plt.hist()

Syntax

plt.hist(data1, color='blue', alpha=0.5, label='Dataset 1')

plt.hist(data2, color='red', alpha=0.5, label='Dataset 2')

plt.legend()

Parameters

- **data1, data2** → Numerical datasets.
- **color** → Color of each histogram.
- **alpha** → Transparency (0 to 1).
- **label** → Name shown in the legend.
- **legend()** → Displays the legend.

Steps to Plot Two Histograms Together

Step 1: Import Matplotlib

import matplotlib.pyplot as plt

Imports the Matplotlib library.

Step 2: Create Two Datasets

data1 = [10,20,20,30,30,30,40,40,50]

data2 = [15,25,25,35,35,45,45,55,55]

Creates two sets of numerical data.

Step 3: Plot the First Histogram

plt.hist(data1,

color='blue',

```
        alpha=0.5,
        label='Dataset 1')
```

Draws the first histogram.

Step 4: Plot the Second Histogram

```
plt.hist(data2,
        color='red',
        alpha=0.5,
        label='Dataset 2')
```

Draws the second histogram on the same graph.

Step 5: Add Title and Labels

```
plt.title("Comparison of Two Histograms")
plt.xlabel("Values")
plt.ylabel("Frequency")
```

Adds the title and axis labels.

Step 6: Display the Legend

plt.legend()

Shows the labels for both datasets.

Step 7: Display the Graph

plt.show()

Displays the histogram.

Complete Program

```
import matplotlib.pyplot as plt

# Dataset 1
data1 = [10,20,20,30,30,30,40,40,50]

# Dataset 2
data2 = [15,25,25,35,35,45,45,55,55]
```

```
# First Histogram
plt.hist(data1,
         color='blue',
         alpha=0.5,
         label='Dataset 1')

# Second Histogram
plt.hist(data2,
         color='red',
         alpha=0.5,
         label='Dataset 2')

# Title and Labels
plt.title("Comparison of Two Histograms")
plt.xlabel("Values")
plt.ylabel("Frequency")
```

```
# Legend

plt.legend()


# Display

plt.show()
```

Explanation of the Program

Import Library

```
import matplotlib.pyplot as plt
```

Imports Matplotlib.

Dataset 1

```
data1 = [10,20,20,30,30,30,40,40,50]
```

Stores the first dataset.

Dataset 2

data2 = [15,25,25,35,35,45,45,55,55]

Stores the second dataset.

First Histogram

plt.hist(data1,

color='blue',

alpha=0.5,

label='Dataset 1')

Creates a **blue histogram**.

Second Histogram

plt.hist(data2,

color='red',

alpha=0.5,

label='Dataset 2')

Creates a **red histogram**.

The **alpha=0.5** makes both histograms semi-transparent so overlapping bars can be seen.

Title

plt.title("Comparison of Two Histograms")

Adds the graph title.

Axis Labels

plt.xlabel("Values")

plt.ylabel("Frequency")

Labels the x-axis and y-axis.

Legend

plt.legend()

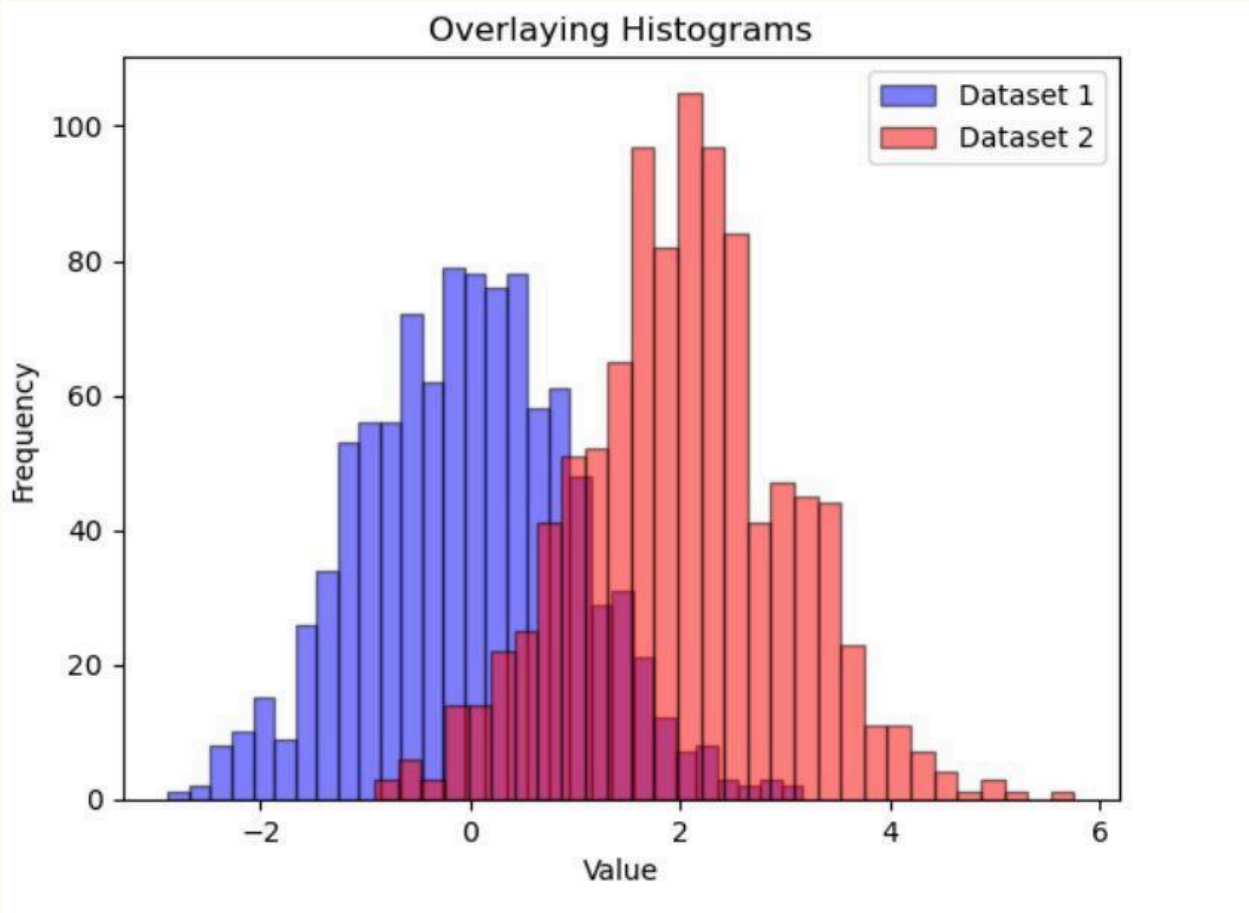
Shows which color represents each dataset.

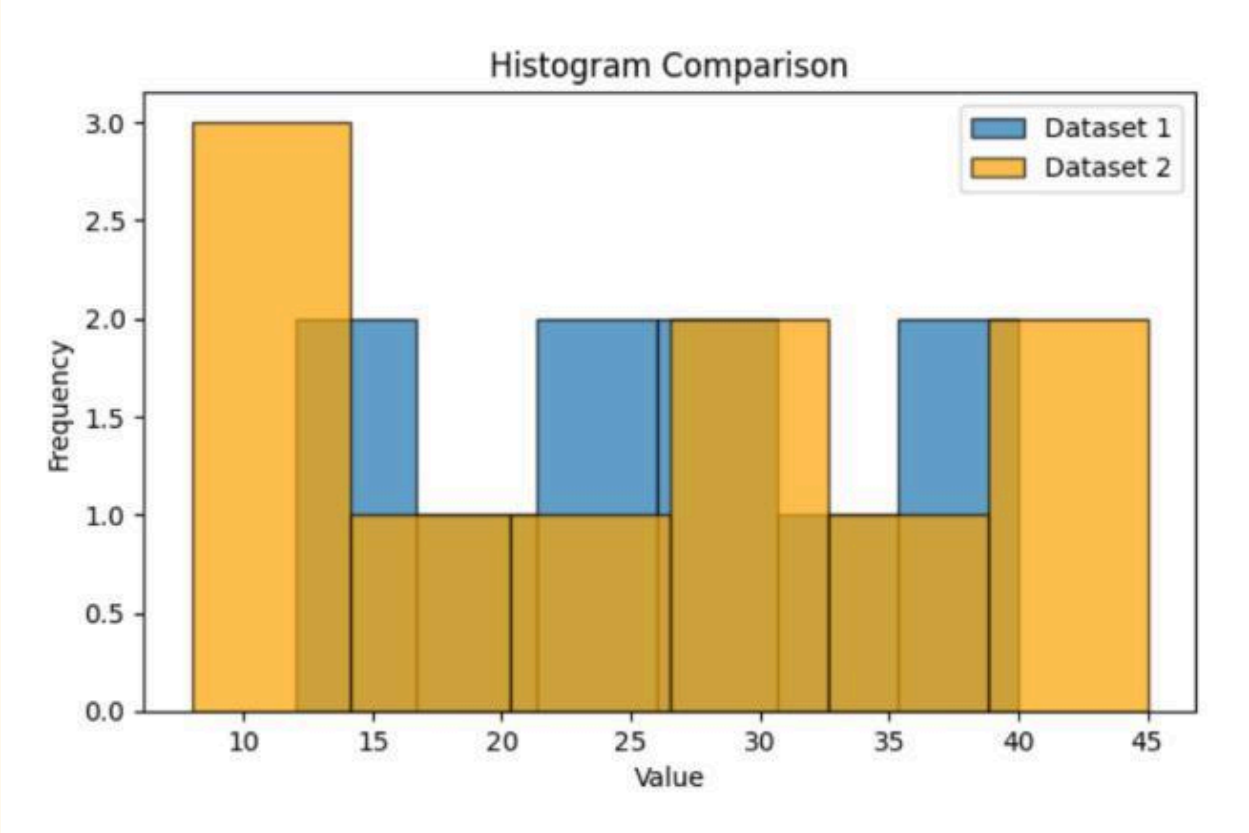
Display

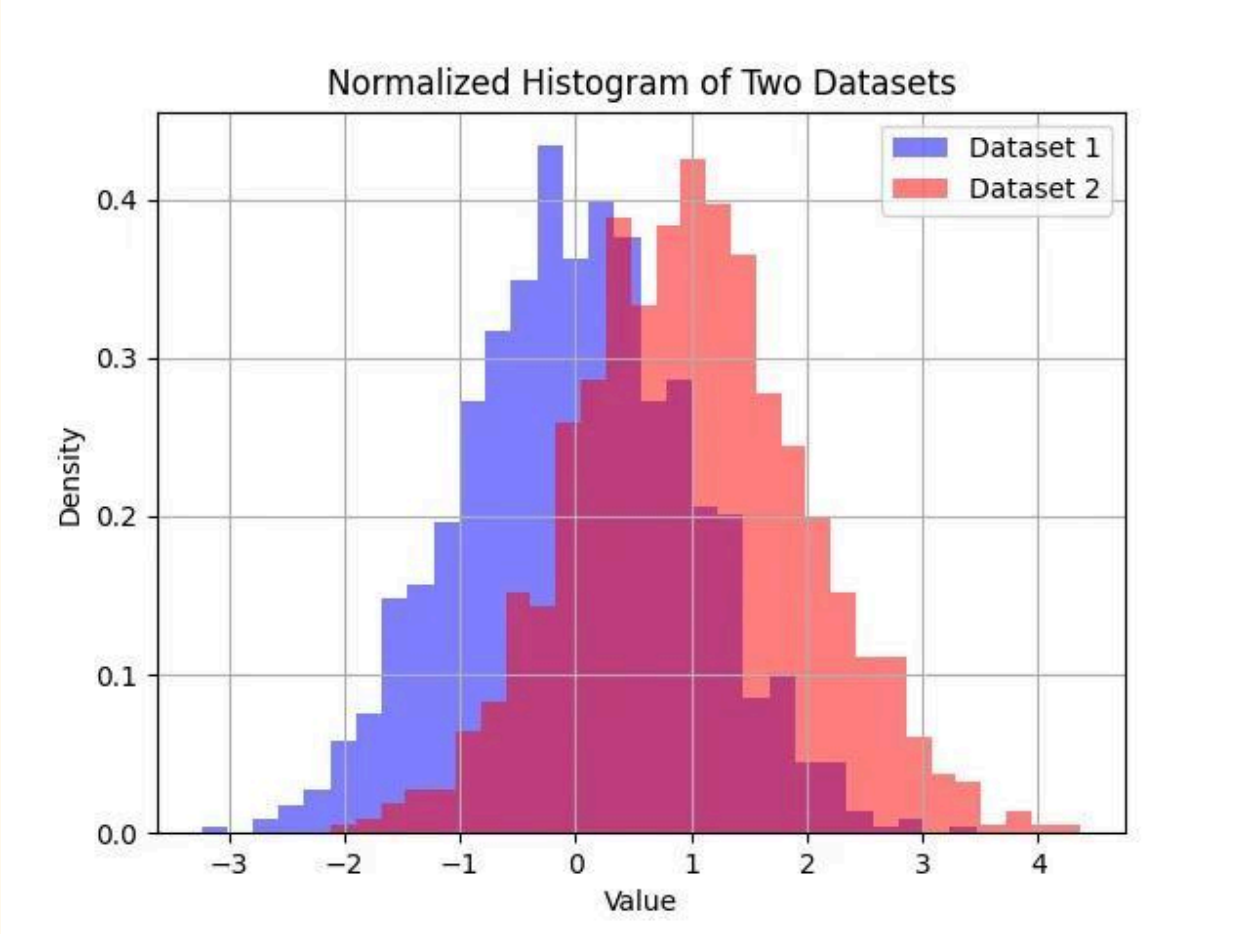
plt.show()

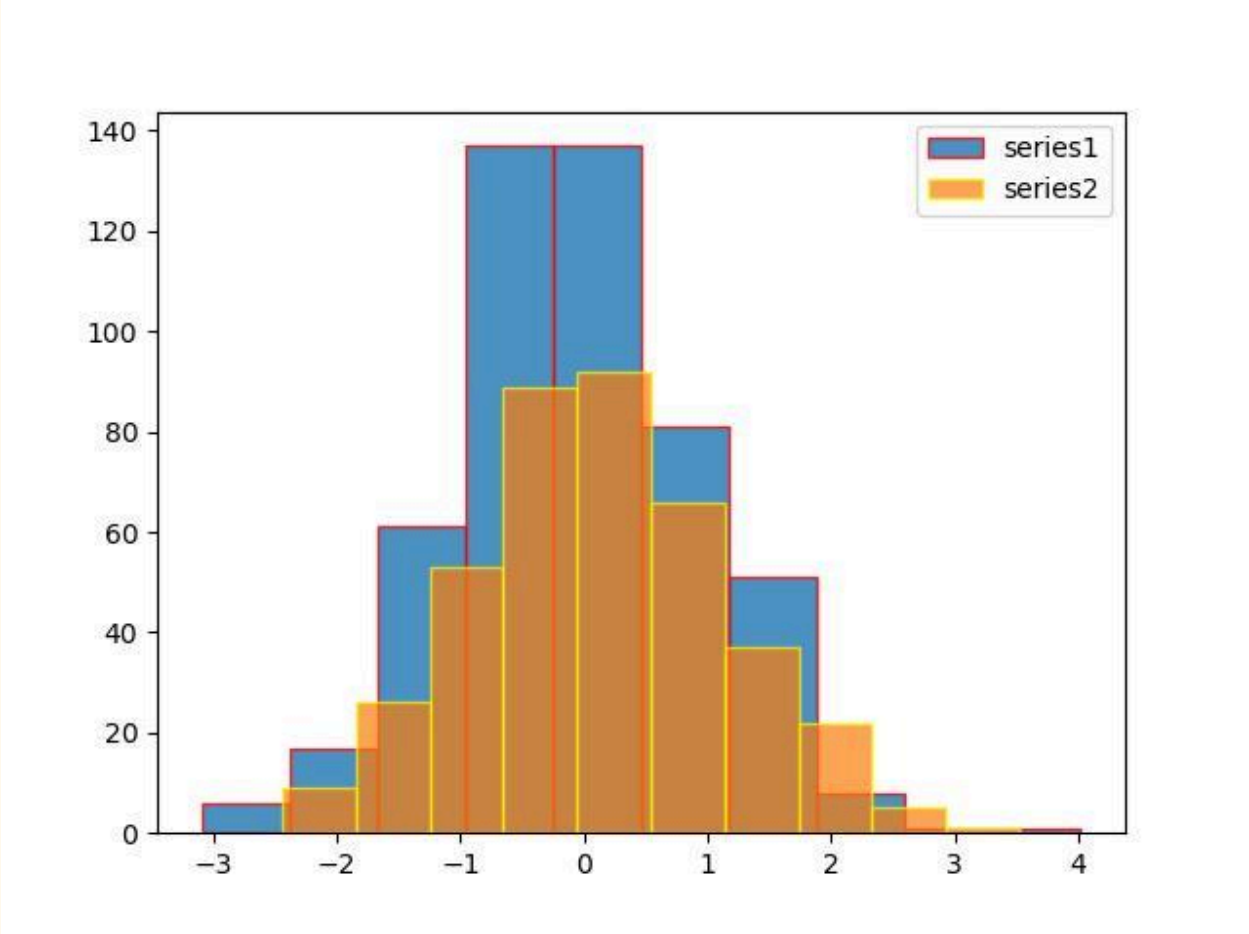
Displays the graph.

Output


Overlaying Histograms
Dataset 1
Dataset 2
Frequency
Value
0
20
40
60
80
100
−2
0
2
4
6


Histogram Comparison
Dataset 1
Dataset 2
Frequency
Value
0.0
0.5
1.0
1.5
2.0
2.5
3.0
10
15
20
25
30
35
40
45


Normalized Histogram of Two Datasets
Dataset 1
Dataset 2
Density
Value
0.0
0.1
0.2
0.3
0.4
−3
−2
−1
0
1
2
3
4



Output Description

The graph displays:

- A **blue histogram** representing **Dataset 1**.
- A **red histogram** representing **Dataset 2**.
- Some bars overlap because of the transparency (`alpha=0.5`).
- A legend identifies each dataset.
- The x-axis shows **Values**.
- The y-axis shows **Frequency**.

Advantages

- Compares two datasets in one graph.
- Saves space by avoiding separate figures.
- Makes differences easy to identify.
- Shows overlapping distributions clearly.
- Improves data analysis.

Applications

Two histograms are commonly used in:

- Comparing students' marks from two classes.
- Comparing salaries of employees in two companies.
- Comparing heights of male and female students.
- Comparing sales of two products.
- Machine Learning data analysis.

Important Functions

Function	Purpose
`plt.hist()`	Creates a histogram
`color`	Changes bar color

`alpha`	Sets transparency
`label`	Names the dataset
`plt.legend()`	Displays the legend
`plt.title()`	Adds a title
`plt.xlabel()`	Labels the x-axis
`plt.ylabel()`	Labels the y-axis
`plt.show()`	Displays the graph

Viva Questions

1. Which function is used to create a histogram?
`plt.hist()`

2. How can two histograms be shown on the same graph?
By calling `plt.hist()` separately for each dataset before using `plt.show()`.

3. Why is `alpha` used?
To make the histograms transparent so overlapping bars are visible.

4. Which function displays the legend?
`plt.legend()`

5. What is the advantage of plotting two histograms together?
It allows easy comparison of the distributions of two datasets.

Conclusion

Plotting two histograms together in Matplotlib is an effective way to compare the distributions of two datasets. By using different colors, transparency (`alpha`), and a legend, both datasets can be displayed clearly in the same figure. This technique is widely used in statistics, business analysis, education, and machine learning for comparing numerical data.

Exam Definition (2 Marks)

Two histograms can be plotted together in Matplotlib by calling the `plt.hist()` function for each dataset with different colors, labels, and transparency (`alpha`). This helps compare the frequency distributions of multiple datasets in a single graph.

26Q) How to Plot Mean, Median, and Mode on Graphs Using `axvline()` and `axhline()`

Ans:

Introduction

In data visualization, it is often useful to highlight important statistical values such as the **Mean**, **Median**, and **Mode**. Matplotlib provides two functions:

- `plt.axvline()` – Draws a **vertical reference line.**
- `plt.axhline()` – Draws a **horizontal reference line.**

These lines help identify the positions of the mean, median, and mode on a graph, making the data easier to analyze.

Definition

- **Mean**: The average of all data values.
- **Median**: The middle value when the data is arranged in order.
- **Mode**: The value that occurs most frequently.

Using `axvline()` and `axhline()`, these statistical measures can be highlighted on a graph.

Why Do We Use `axvline()` and `axhline()`?

They are used to:

- Highlight important values.
- Improve readability.
- Compare data with statistical measures.
- Make graphs more informative.
- Analyze distributions easily.

Functions Used

Vertical Line

plt.axvline()

Draws a **vertical line.**

Horizontal Line

plt.axhline()

Draws a **horizontal line.**

Syntax

Vertical Line

```
plt.axvline(x=value,
            color='red',
            linestyle='--',
            label='Mean')
```

Horizontal Line

```
plt.axhline(y=value,
            color='green',
            linestyle='--',
            label='Average')
```

Parameters

Parameter	Description
`x`	X-coordinate where the vertical line is drawn
`y`	Y-coordinate where the horizontal line is drawn
`color`	Line color

`linestyle`	Solid, dashed, dotted, etc.
`linewidth`	Thickness of the line
`label`	Text displayed in the legend

Example 1: Plot Mean, Median, and Mode on a Histogram

import matplotlib.pyplot as plt

import statistics

Sample data

data = [2,3,4,4,5,6,6,6,7,8,9]

Calculate statistics

mean = statistics.mean(data)

median = statistics.median(data)

mode = statistics.mode(data)

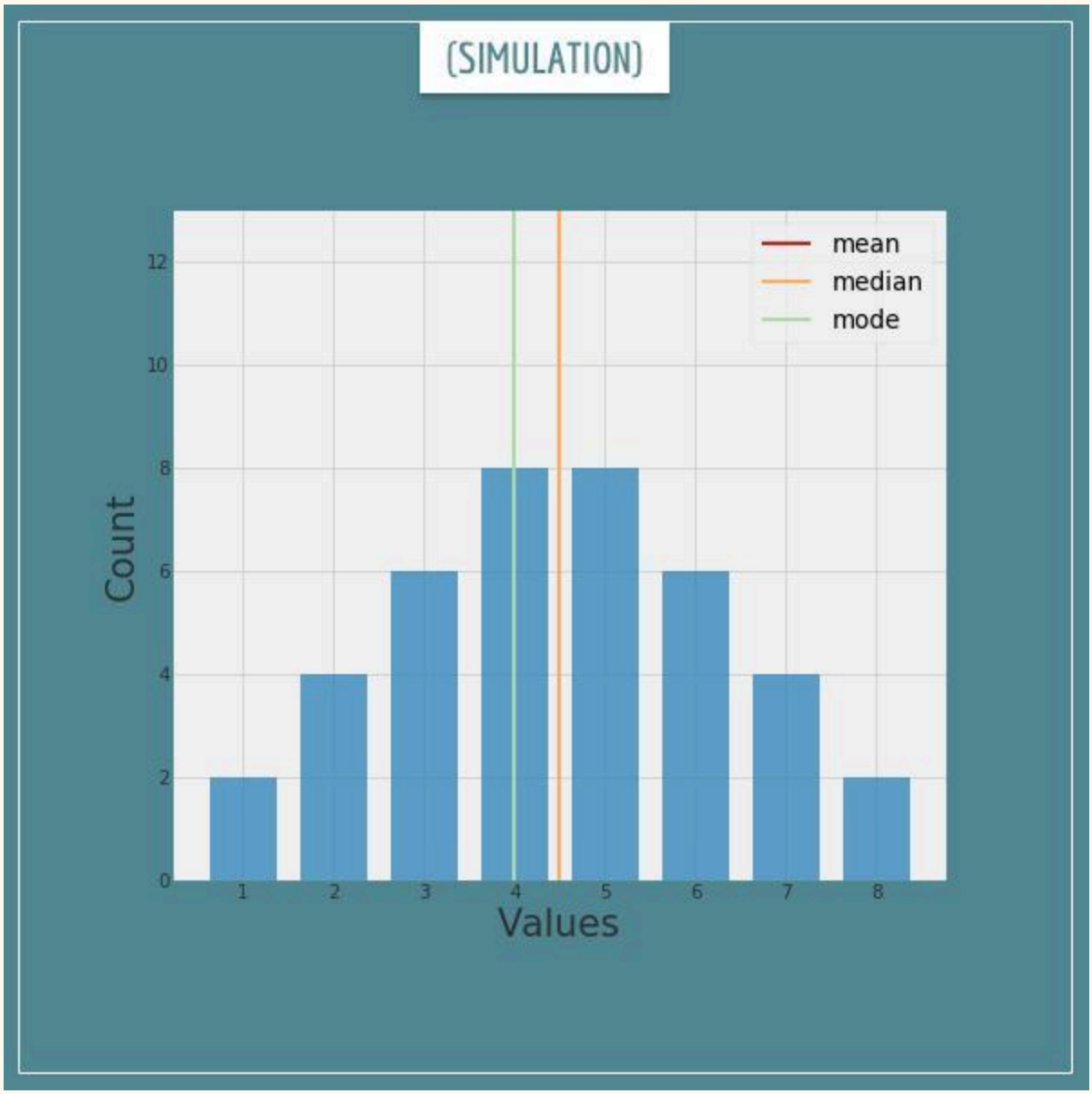
```
# Histogram
plt.hist(data, bins=7, color='skyblue', edgecolor='black')

# Vertical lines
plt.axvline(mean, color='red', linestyle='--', label='Mean')
plt.axvline(median, color='green', linestyle='-.', label='Median')
plt.axvline(mode, color='blue', linestyle=':', label='Mode')

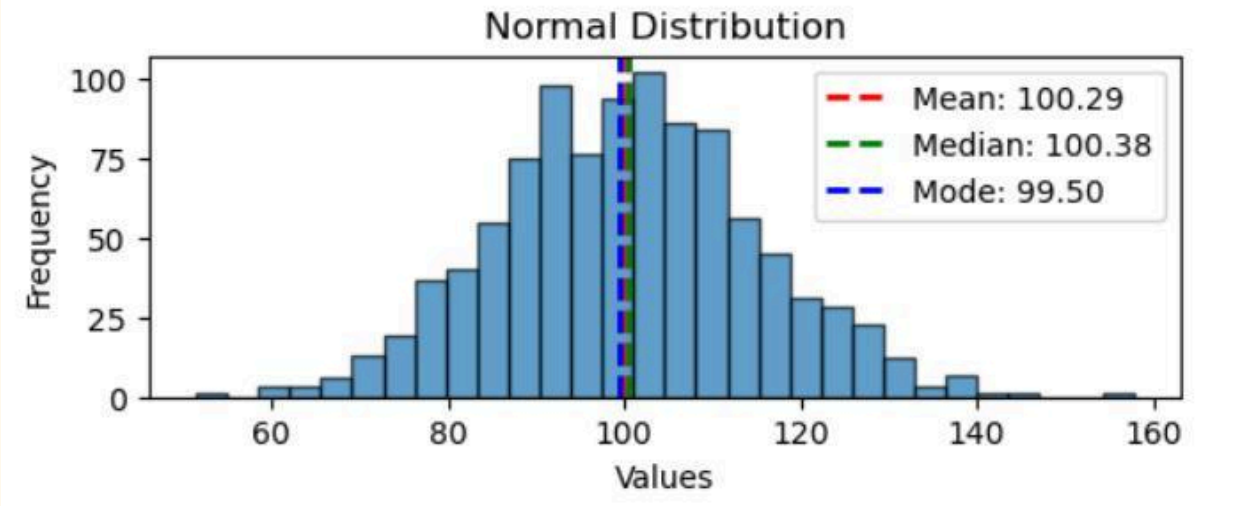
plt.title("Histogram with Mean, Median and Mode")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.legend()

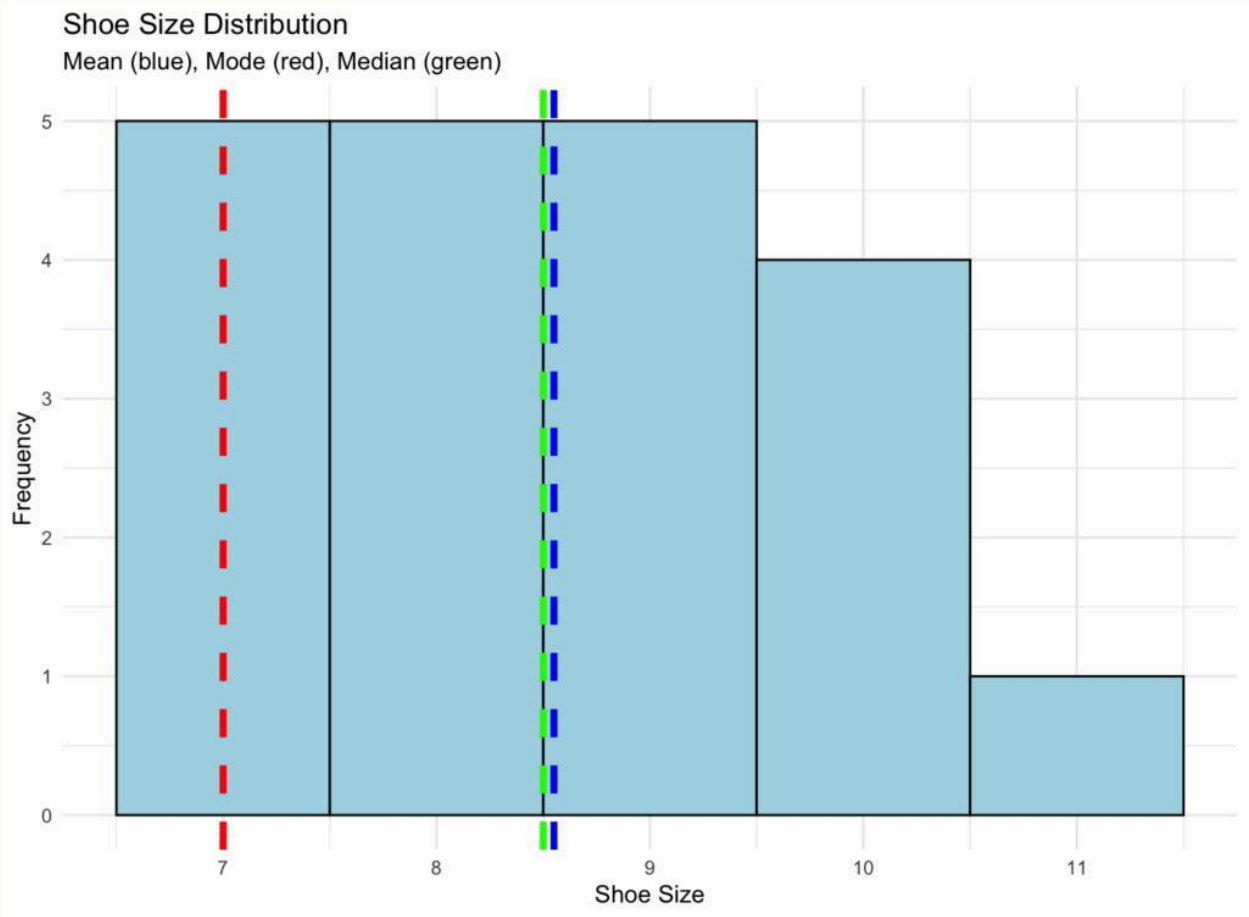
plt.show()
```

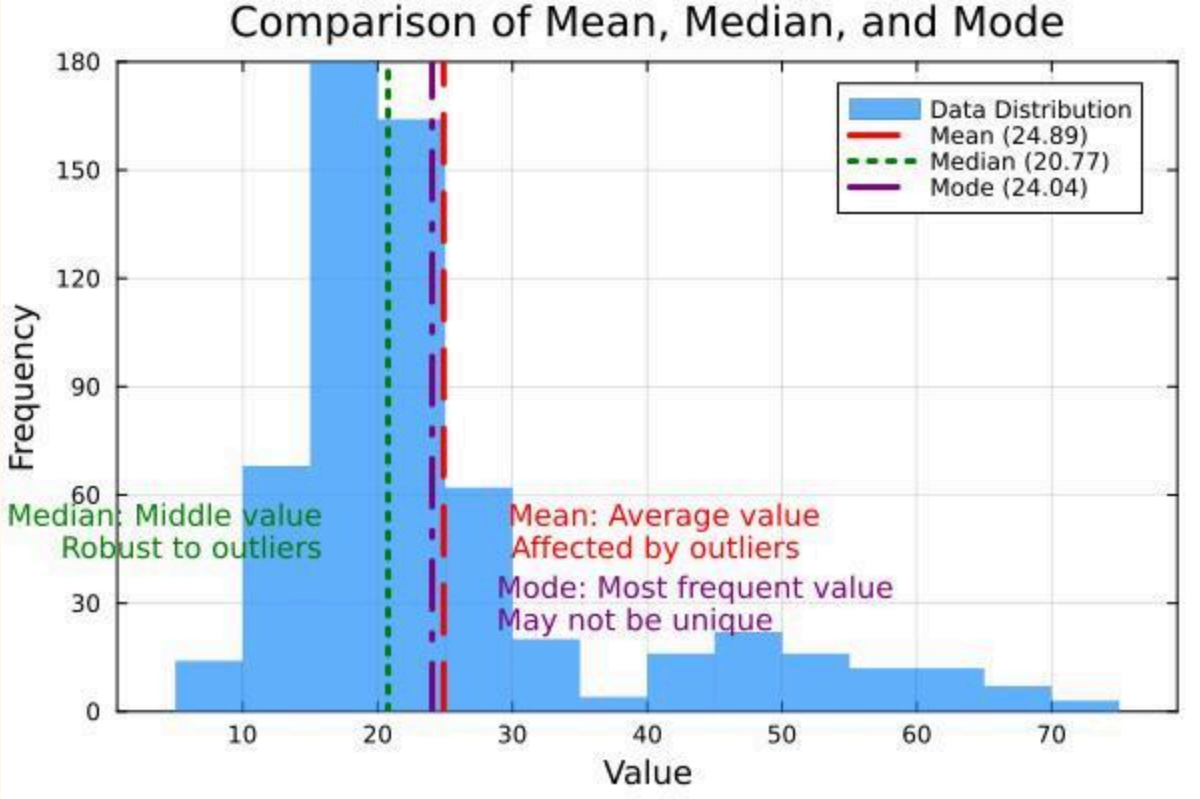
Output

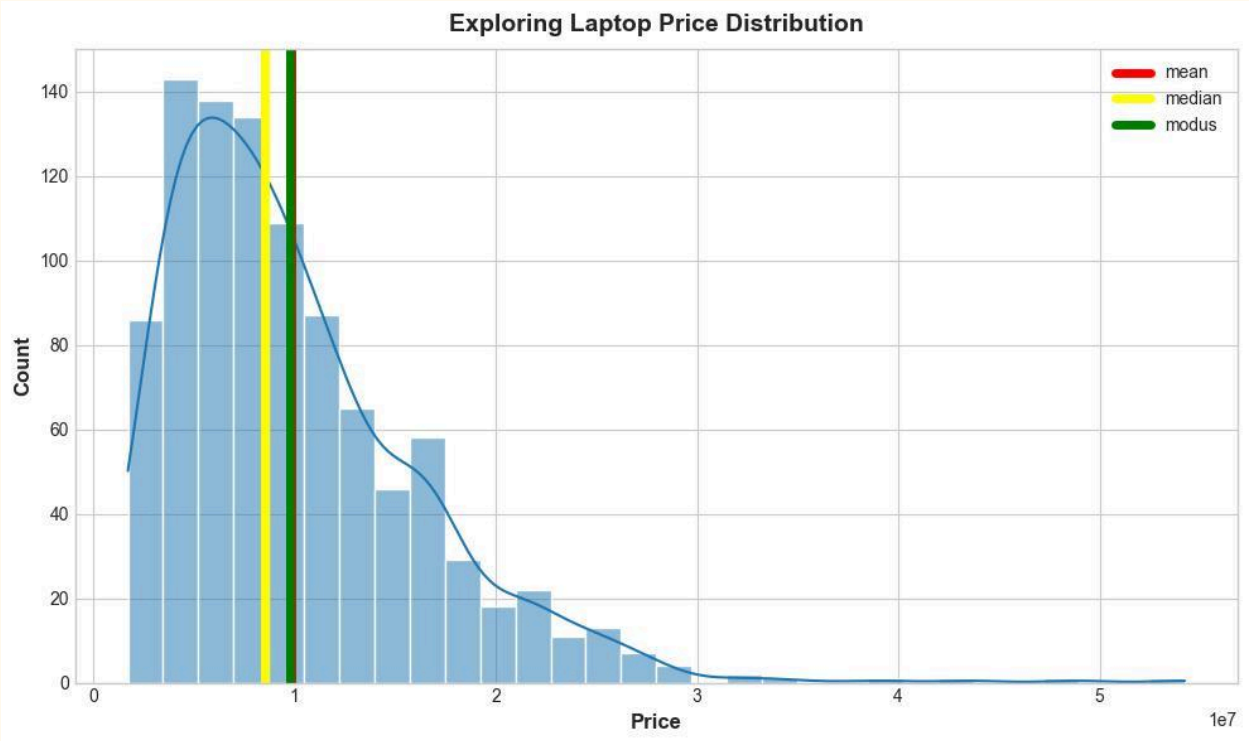

(SIMULATION)
mean
median
mode
Count
Values
0
2
4
6
8
10
12
1
2
3
4
5
6
7
8

```
Mean: 100.29
Median: 100.38
Mode: 99.50
```






Comparison of Mean, Median, and Mode
Data Distribution
Mean (24.89)
Median (20.77)
Mode (24.04)
Frequency
Value
Median: Middle value
Robust to outliers
Mean: Average value
Affected by outliers
Mode: Most frequent value
May not be unique


Exploring Laptop Price Distribution
mean
median
modus
Count
Price
1e7

Output Description

- Histogram showing the data distribution.
- **Red dashed line** → Mean.
- **Green dash-dot line** → Median.
- **Blue dotted line** → Mode.
- Legend identifies each statistical value.

Example 2: Using `axhline()` in a Line Graph

Suppose the average sales value is **18**.

import matplotlib.pyplot as plt

x = [1,2,3,4,5]

y = [10,15,18,20,27]

plt.plot(x, y, marker='o')

Horizontal line at average value

plt.axhline(y=18,

```
        color='red',
        linestyle='--',
        label='Average Sales')

plt.title("Sales Analysis")
plt.xlabel("Days")
plt.ylabel("Sales")
plt.legend()

plt.show()
```

Output

Output Description

- A line graph of sales.
- A **horizontal red dashed line** at **18**.
- The line represents the average sales value.

Difference Between `axvline()` and `axhline()`

`axvline()`	`axhline()`
Draws a **vertical line**	Draws a **horizontal line**
Uses the **x** parameter	Uses the **y** parameter
Highlights x-values such as mean, median, mode	Highlights y-values such as average, threshold, target
Commonly used in histograms and scatter plots	Commonly used in line graphs and bar charts

Which Graphs Can Use `axvline()` and `axhline()`?

Both functions can be used with many Matplotlib graphs:

- ✅ Histogram
- ✅ Line Plot
- ✅ Scatter Plot
- ✅ Bar Chart
- ✅ Box Plot

Advantages

- Highlights important statistical values.
- Makes graphs easy to interpret.
- Helps compare data with averages.
- Improves presentation quality.
- Useful in reports and research.

Applications

These functions are widely used in:

- Data Science
- Machine Learning
- Statistics
- Business Analytics
- Sales Analysis
- Scientific Research
- Financial Analysis

Important Functions

Function	Purpose
`plt.axvline()`	Draws a vertical reference line

`plt.axhline()`	Draws a horizontal reference line
`statistics.mean()`	Calculates the mean
`statistics.median()`	Calculates the median
`statistics.mode()`	Calculates the mode
`plt.legend()`	Displays the legend
`plt.show()`	Displays the graph

Viva Questions

1. Which function draws a vertical line?
`plt.axvline()`

2. Which function draws a horizontal line?
`plt.axhline()`

3. Why is `axvline()` used in histograms?
To highlight statistical values such as the mean, median, and mode.

4. What does `axhline()` represent in a line graph?
It shows a constant y-value, such as an average or target.

5. Can `axvline()` and `axhline()` be used in the same graph?
Yes. They can be used together whenever both vertical and horizontal reference lines are needed.

Conclusion

The `axvline()` and `axhline()` functions are powerful tools in Matplotlib for highlighting important statistical values and reference levels. `axvline()` is mainly used to display **mean, median, and mode** as vertical lines, while `axhline()` is useful for showing **averages, thresholds, or target values** as horizontal lines. These functions improve the readability and interpretability of graphs, making data analysis more effective.

Exam Definition (2 Marks)

`axvline()` is used to draw a vertical reference line at a specified x-value, while `axhline()` is used to draw a horizontal reference line at a specified y-value. They are commonly used to highlight statistical

values such as the mean, median, mode, or average in Matplotlib graphs.
